

title: "ScratchBird — Concepts and Getting Started"

ScratchBird — Concepts and Getting Started

ScratchBird documentation — draft

Who this book is for

New users, evaluators, and anyone forming a first understanding of ScratchBird.

About this book

This volume introduces ScratchBird as a Convergent Data Engine (CDE): what that product class is, the architecture and ideas behind it, and the first practical steps for creating a database and running a session. Read it before the reference and operations volumes.

Parts in this volume

- **Getting Started**
- **Concepts of a Convergent Data Engine**
- **Appendix: Functionality Support Matrix**

This is a **draft**. See *About This Documentation* at the end of this book for status and license. Confirm any specific behavior against the current build.

Getting Started Guide

Early Draft 06-14-2026

ScratchBird Getting Started Guide

This directory contains the draft ScratchBird Getting Started Guide for end users, evaluators, and operators who need a high-level understanding before reading the SBsql language reference or building the source tree.

This guide is intentionally cautious. It describes the architecture and intended use of the public source tree without promising production readiness, compatibility completeness, platform support, performance, security certification, or suitability for any particular workload. Always verify the current release notes, build configuration, test results, and component status before treating a feature as available.

Directory Map

Directory	Purpose
core_concepts	Plain-language explanations of databases, Convergent Data Engines, Multi-Generational Architecture, and the ScratchBird architecture.
operating_modes	How ScratchBird is intended to run as an embedded engine, IPC server, standalone server, or managed group deployment.
architecture	End-user architecture topics: parser separation, recursive schema, SBsql/SBLR, Git-oriented workflows, identity, and recovery.
using_scratchbird	First tasks: creating a database, connecting with SBsql, understanding schemas and reference-system compatibility.
administration	Deployment choice, configuration basics, diagnostics, support bundles, backup, restore, and data movement.
reference	Glossary and document map.

Reading Model

Start with [core_concepts/what_is_a_database.md](#) if you are new to database systems. Start with [operating_modes/choosing_a_mode_summary.md](#) if you already know databases and need to decide how ScratchBird fits into an application.

ScratchBird uses several branded component names:

Name	Role In This Guide
SBcore	The embedded engine library.
SBsrv	The local IPC server.
SBgate	The listener and parser-facing network entry point.
SBmgr	The single-node manager front door.
SBsql	The native ScratchBird SQL language and command-line interface.
SBParser	The native SBsql parser package.
Compatibility parser	A parser package that accepts one reference-system protocol or dialect surface and lowers admitted work to ScratchBird execution requests.

First Reading Path

1. [What Is A Database?](#)
2. [What Is A Convergent Data Engine?](#)
3. [How ScratchBird Implements A CDE](#)
4. [Understanding MGA](#)
5. [Choosing A Mode Summary](#)
6. [First Database](#)
7. [First SBsql Session](#)

Related Manuals

Topic	Manual
SBsql grammar, syntax, functions, operators, catalog tables, procedural SQL, and EBNF	SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX
Installation, configuration, service lifecycle, diagnostics, backup, restore, and release validation	Operations, Security, and Autonomy, page XXX
Schema tree and name resolution details	SBsql Language Reference — Syntax, page XXX
Operator precedence and result types	SBsql Language Reference — Syntax, page XXX
Procedural SQL	SBsql Language Reference — Syntax, page XXX

Document Map

Purpose

This document map helps readers move through the draft ScratchBird documentation without having to know the directory layout first. It is organized by reading goal: learn the concepts, choose an operating mode, run a first session, understand architecture, administer a deployment, or continue into the SBsql Language Reference.

This map is a navigation aid. It is not a release checklist, support statement, compatibility guarantee, or production-readiness claim.

Start Here

Reader Goal	Start With	Then Read
I am new to database systems.	What Is A Database?	What Is A Convergent Data Engine?
I know databases and want the ScratchBird model.	How ScratchBird Implements A CDE	Engine Parser Boundary
I need to choose how to run it.	Choosing A Mode Summary	The operating-mode page for the mode you intend to test.
I want to create a first database.	First Database	First SBsql Session
I am trying to understand names and schemas.	Schemas, Objects, And Names	Recursive Schema Tree
I need detailed SBsql syntax.	Language Reference (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)	The specific syntax, datatype, function, or catalog page for the command you are using.

Recommended First Reading Path

Read these pages in order if you are evaluating ScratchBird for the first time.

1. [What Is A Database?](#)
2. [What Is A Convergent Data Engine?](#)
3. [How ScratchBird Implements A CDE](#)
4. [Choosing A Mode Summary](#)
5. [First Database](#)
6. [First SBsql Session](#)
7. [Schemas, Objects, And Names](#)
8. [Configuration Basics](#)
9. [Diagnostics And Support Bundles](#)

Core Concepts

Page	Use It For
What Is A Database?	Plain-language explanation of durable data, metadata, transactions, identity, and why a database is more than a file.
What Is A Convergent Data Engine?	Explanation of the CDE category and the difference between shared engine authority and one universal syntax.
How ScratchBird Implements A CDE	ScratchBird-specific overview of SBcore, parsers, SBLR, recursive schema, sandboxing, resources, operating modes, and refusal behavior.
Understanding MGA	Plain-language explanation of Multi-Generational Architecture: transactions, snapshots, non-destructive change, cleanup, and how it differs from write-ahead logging.

Operating Modes

Page	Use It For
Choosing A Mode Summary	Compare embedded, single-node IPC, standalone server, and managed group deployment at a high level.
Embedded Engine	Understand direct SBcore use inside an application process.
Single-Node IPC Server	Understand local multi-user access through SBsrv without a network listener.
Standalone Server	Understand client access through SBgate, parser routing, and server process boundaries.
Managed Group Deployment	Understand SBmgr as a managed single-node front door with shared identity or policy integration.

Architecture Topics

Page	Use It For
Engine Parser Boundary	Understand what parsers do, what SBcore owns, and why raw SQL text is not durable authority.
Recursive Schema Tree	Understand schema branches, visible roots, workareas, catalog projections, and name resolution.
SBsql And SBLR	Understand the native language surface and the bound request representation submitted to engine authority.
Git-Oriented Workflows	Understand how Git-oriented lifecycle concepts fit around database authority without replacing transactions or recovery.
Identity, Authentication, And Authorization	Understand users, principals, groups, grants, policy checks, and sandboxed access at a high level.
Storage, Transactions, And Recovery	Understand the role of storage, transaction visibility, recovery, and refusal when a database state is uncertain.

Using ScratchBird

Page	Use It For
First Database	Create or open a disposable first database, run a smoke workload, test one controlled refusal, and verify reopen behavior.
First SBsql Session	Run a first native SBsql session with schema creation, table creation, multi-row insert, query, commit, error handling, and cleanup.
Schemas, Objects, And Names	Understand current schema, home schema, qualified names, durable UUID identity, object lifecycle, comments, and compatibility workareas.
Reference-System Compatibility	Understand standalone parser packages, compatibility scope, sandboxing, logical streams, denied physical or low-level actions, and refusal behavior.

Administration

Page	Use It For
Choosing A Deployment Mode	Translate application needs into an operating mode and deployment shape.
Configuration Basics	Understand configuration, resource files, policy defaults, parser registration, and output-tree expectations.
Diagnostics And Support Bundles	Understand message vectors, logs, redaction, support bundles, and troubleshooting evidence.
Backup, Restore, And Data Movement Overview	Understand logical backup and restore streams, denied physical copy behavior, import/export, replication, ETL, and migration boundaries.

Language Reference Entry Points

Use the Language Reference when you need exact syntax, types, operators, functions, catalog views, or procedural SQL details.

Area	Entry Point
Main language index	Language Reference — Foundations, Data Types, and Catalog, page XXX
Core engine concepts	Language Reference — Foundations, Data Types, and Catalog, page XXX
Parser pipeline	Language Reference — Foundations, Data Types, and Catalog, page XXX
Security and sandboxing	Language Reference — Foundations, Data Types, and Catalog, page XXX
UUID catalog identity	Language Reference — Foundations, Data Types, and Catalog, page XXX
Transactions and recovery	Language Reference — Foundations, Data Types, and Catalog, page XXX
Type system	Language Reference — Foundations, Data Types, and Catalog, page XXX
Numeric types	Language Reference — Foundations, Data Types, and Catalog, page XXX
Text, collation, and character sets	Language Reference — Foundations, Data Types, and Catalog, page XXX
Temporal types	Language Reference — Foundations, Data Types, and Catalog, page XXX
Documents, graph, vector, and multimodel values	Language Reference — Foundations, Data Types, and Catalog, page XXX
Operators	Language Reference — Syntax, page XXX
Operator precedence and result types	Language Reference — Syntax, page XXX
Procedural SQL overview	Language Reference — Syntax, page XXX
Procedural blocks	Language Reference — Syntax, page XXX
Procedural control flow	Language Reference — Syntax, page XXX
Procedural cursors	Language Reference — Syntax, page XXX
Procedural exceptions	Language Reference — Syntax, page XXX
Triggers and events	Language Reference — Syntax, page XXX
Functional reference index	Functions — Functional Reference Index, page XXX
Catalog reference index	Language Reference — Foundations, Data Types, and Catalog, page XXX

Syntax Reference By Task

Task	Pages
Create or manage databases	Language Reference — Syntax, page XXX, Language Reference — Syntax, page XXX
Create or manage schemas	Language Reference — Syntax, page XXX, Language Reference — Syntax, page XXX
Create or manage tables	Language Reference — Syntax, page XXX, Language Reference — Syntax, page XXX, Language Reference — Syntax, page XXX

Getting Started Guide Draft

Task	Pages
Create or manage views	Language Reference — Syntax, page XXX, Language Reference — Syntax, page XXX
Create or manage indexes	Language Reference — Syntax, page XXX
Create or manage routines	Language Reference — Syntax, page XXX Language Reference — Syntax, page XXX Language Reference — Syntax, page XXX
Create or manage sequences	Language Reference — Syntax, page XXX
Query data	Language Reference — Syntax, page XXX Language Reference — Syntax, page XXX), Where (SbSql Language Reference — Syntax, page XXX), Group By And Having (SbSql Language Reference — Syntax, page XXX), Order By Limit Offset (SbSql Language Reference — Syntax, page XXX), Window (SbSql Language Reference — Syntax, page XXX), With (SbSql Language Reference — Syntax, page XXX)
Change data	Insert (SbSql Language Reference — Syntax, page XXX), Update (SbSql Language Reference — Syntax, page XXX), Delete (SbSql Language Reference — Syntax, page XXX), Merge And Upsert (SbSql Language Reference — Syntax, page XXX), Copy (SbSql Language Reference — Syntax, page XXX)
Control transactions	Transaction Control (SbSql Language Reference — Syntax, page XXX)
Manage security	Security And Privilege Statements (SbSql Language Reference — Syntax, page XXX), Policy, Mask, And RLS (SbSql Language Reference — Syntax, page XXX)
Move data or migrate	Backup, Restore, Replication, Migration (SbSql Language Reference — Syntax, page XXX)
Work with multimodel statements	Multimodel Statements (SbSql Language Reference — Syntax, page XXX)
Understand diagnostics	Refusal Vectors (SbSql Language Reference — Syntax, page XXX), Management And Operations (SbSql Language Reference — Syntax, page XXX)
Work with agents	Agent (SbSql Language Reference — Syntax, page XXX)
Understand scripts and identifiers	Script Tokens And Identifiers (SbSql Language Reference — Syntax, page XXX)

Operations And Administration Entry Points

Use the Operations And Administration Guide when you need runbook, configuration, diagnostics, data movement, or release-validation guidance.

Area	Entry Point
Main operations index	Operations And Administration Guide (ScratchBird — Operations, Security, and Autonomy, page XXX)
Installation and output layout	Installation And Output Layout (ScratchBird — Operations, Security, and Autonomy, page XXX)
Configuration	Configuration Reference (ScratchBird — Operations, Security, and Autonomy, page XXX)
Operating modes runbook	Operating Modes Runbook (ScratchBird — Operations, Security, and Autonomy, page XXX)
Service lifecycle	Service Lifecycle (ScratchBird — Operations, Security, and Autonomy, page XXX)
Identity, security, and policy	Identity, Security, And Policy (ScratchBird — Operations, Security, and Autonomy, page XXX)
Parser routes	Parser Registration And Routes (ScratchBird — Operations, Security, and Autonomy, page XXX)
Database lifecycle	Database Lifecycle (ScratchBird — Operations, Security, and Autonomy, page XXX)
Filespaces and storage	Filespaces And Storage (ScratchBird — Operations, Security, and Autonomy, page XXX)
Backup, restore, and data movement	Backup, Restore, And Data Movement (ScratchBird — Operations, Security, and Autonomy, page XXX)
Diagnostics and support bundles	Diagnostics, Message Vectors, And Support Bundles (ScratchBird — Operations, Security, and Autonomy, page XXX)
Monitoring and readiness	Monitoring, Health, And Readiness (ScratchBird — Operations, Security, and Autonomy, page XXX)
Troubleshooting	Troubleshooting (ScratchBird — Operations, Security, and Autonomy, page XXX)
Upgrade and compatibility	Upgrade And Compatibility Policy (ScratchBird — Operations, Security, and Autonomy, page XXX)
Release validation	Release Validation Checklist (ScratchBird — Operations, Security, and Autonomy, page XXX)

Functional Reference By Namespace

Namespace	Page
Core functions	sb_core (SBsql Functions — SB Core Functional Reference, page XXX)
Cryptographic and protected-material helpers	sb_crypto (SBsql Functions — SB Crypto Functional Reference, page XXX)
Cursor helpers	sb_cursor (SBsql Functions — SB Cursor Functional Reference, page XXX)
Diagnostics	sb_diagnostic (SBsql Functions — SB Diagnostic Functional Reference, page XXX)
JSON	sb_json (SBsql Functions — SB JSON Functional Reference, page XXX)
Large objects	sb_lob (SBsql Functions — SB LOB Functional Reference, page XXX)
Operator helpers	sb_operator (SBsql Functions — SB Operator Functional Reference, page XXX)
Range values	sb_range (SBsql Functions — SB Range Functional Reference, page XXX)
Regular expressions	sb_regex (SBsql Functions — SB Regex Functional Reference, page XXX)
Row sets	sb_rowset (SBsql Functions — SB Rowset Functional Reference, page XXX)
Spatial values	sb_spatial (SBsql Functions — SB Spatial Functional Reference, page XXX)
Temporal values	sb_temporal (SBsql Functions — SB Temporal Functional Reference, page XXX)
Time-series values	sb_timeseries (SBsql Functions — SB Timeseries Functional Reference, page XXX)
UUID values	sb_uuid (SBsql Functions — SB UUID Functional Reference, page XXX)
Vector values	sb_vector (SBsql Functions — SB Vector Functional Reference, page XXX)
XML values	sb_xml (SBsql Functions — SB XML Functional Reference, page XXX)

If You Are Looking For A Term

Use [Glossary](#) for short definitions of component names, authority concepts, session concepts, data movement terms, diagnostics, and parser terminology.

What Is A Database?

Purpose

A database is a managed place for durable information. It stores data, describes that data, controls who may use it, and applies rules when the data changes.

The important part is the word managed. A spreadsheet, a text file, and a directory full of JSON files can all hold information. A database adds a system around the information so applications can ask questions, make changes, share access, and recover after failures with predictable rules.

This page explains the idea without assuming that you already know database terminology.

Data And Metadata

Data is the information users care about: accounts, orders, documents, sensor readings, vectors, audit events, configuration records, and similar values.

Metadata is information about the data:

Metadata	What It Describes
Tables and columns	The relational shape of rows.
Document descriptors	The expected shape or meaning of document values.
Types and domains	Which values are valid and how they compare or convert.
Constraints	Rules that values must satisfy.
Indexes	Search structures that help find rows or values.
Views	Named query projections over stored or derived data.
Procedures and functions	Stored routines that perform controlled work.
Grants and policies	Who can see or change each object or value.
Catalog entries	The durable records that describe database objects.

In ScratchBird, metadata is not just comments or loose documentation. It is engine-owned state. A parser may accept a text command such as `create table`, but the durable object is represented by catalog identity, descriptors, parent schema identity, grants, and transaction visibility.

Common Database Responsibilities

Most database systems provide a set of core responsibilities:

Responsibility	Meaning
Storage	Keeps data and metadata in durable files, memory-backed structures, or managed devices.
Type handling	Knows how values are encoded, compared, sorted, converted, and validated.
Query execution	Reads or changes data according to a request.
Transactions	Groups work so it can commit, roll back, and recover consistently.
Concurrency	Lets multiple sessions work at the same time without corrupting shared state.
Security	Controls who can connect and what each identity can see or change.
Recovery	Reopens the database after normal shutdown, refused startup, or failure paths according to documented rules.
Diagnostics	Explains success, failure, refusal, and recovery-required states to users and operators.

An application usually experiences these responsibilities through a language, driver, protocol, command-line tool, or embedded API.

A Simple Example

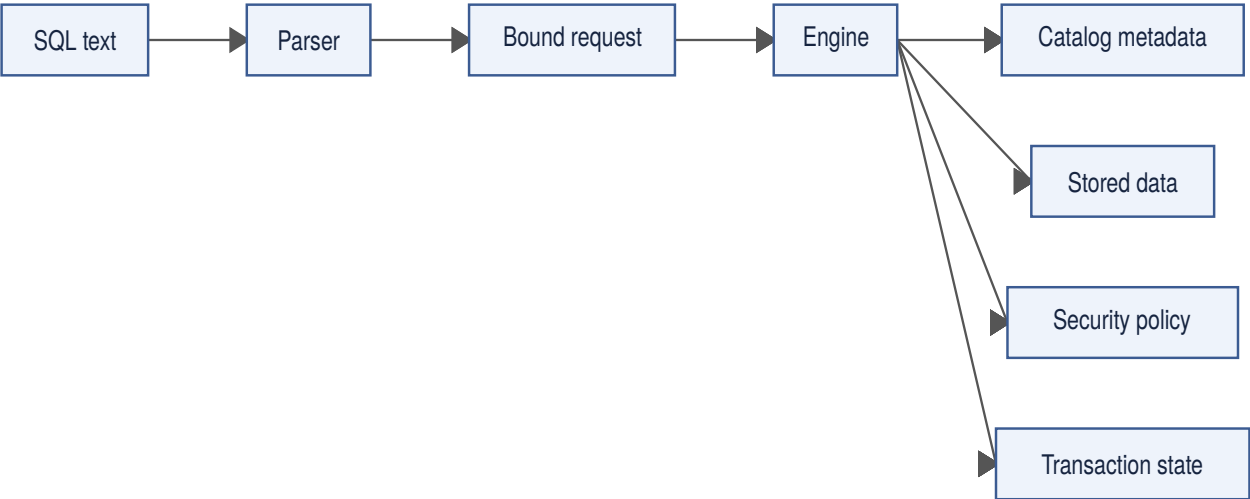
A table named `orders` might hold order rows:

```
create table app.orders (  
    order_id bigint not null,  
    account_id bigint not null,  
    status text not null,  
    total numeric(18, 2) not null  
);
```

That statement is user-facing text. After it is accepted, a database also needs durable metadata:

- the table object's internal identity;
- the parent schema identity;
- each column's descriptor;
- the constraints attached to the table;
- privileges and policies;
- transaction visibility for the catalog change.

The durable database is therefore more than the text statement.



Tables, Documents, And Other Shapes

Many people first learn databases through relational tables. Tables are still a central model because they give clear names, columns, constraints, indexes, and query behavior.

Modern applications often also need other shapes:

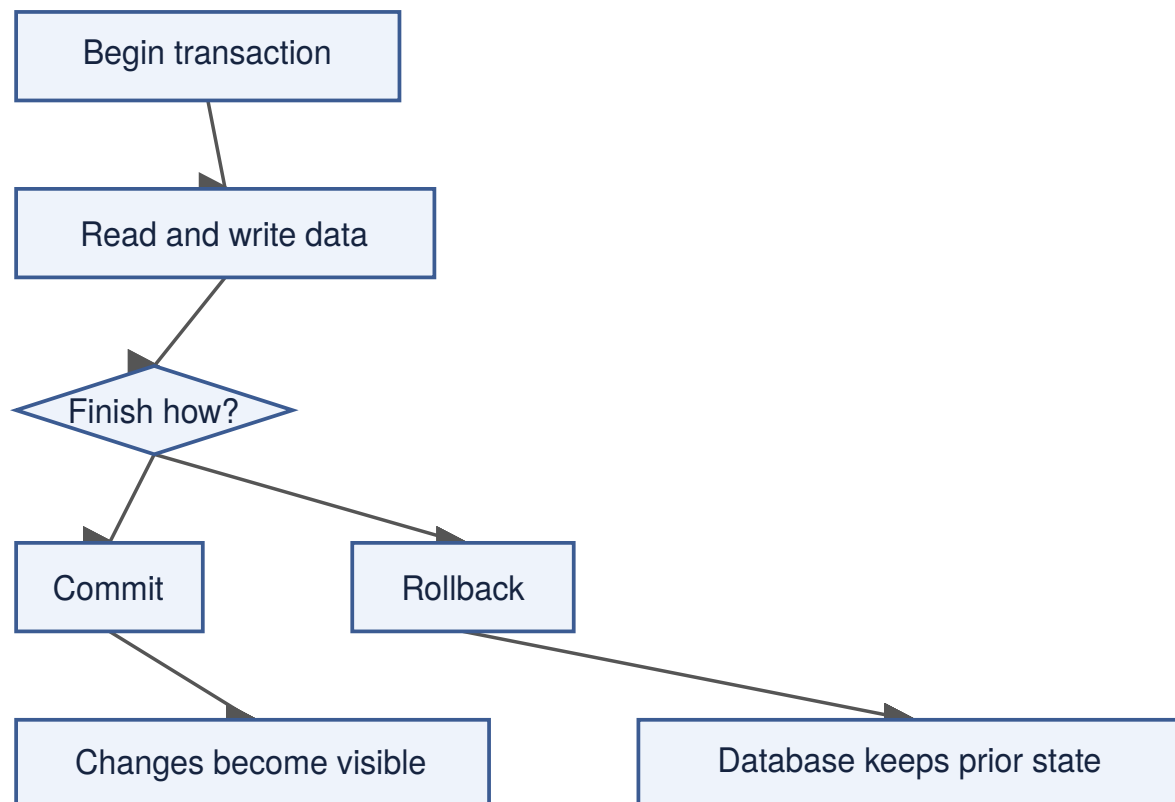
Shape	Example Use
Relational rows	Orders, invoices, users, permissions.
Documents	Flexible records, application payloads, nested values.
Key-value records	Fast lookup by key.
Graph relationships	Connected objects and relationship traversal.
Vector values	Embeddings and similarity search.
Time-series values	Measurements over time.

ScratchBird documentation uses database language broadly because the system is designed around a common engine authority model for multiple data shapes where those surfaces are implemented.

Transactions In Plain Language

A transaction is a boundary around work.

Within a transaction, a session can make changes that are not yet final. When the transaction commits, the engine makes the outcome visible according to transaction visibility rules. When the transaction rolls back, the engine discards the uncommitted outcome.



ScratchBird documentation refers to its transaction authority model as MGA. For a new reader, the practical rule is this: commit, rollback, visibility, cleanup, and recovery are engine decisions. Client tools and parser packages can request transaction actions, but they do not own finality.

Names And Durable Identity

Users work with names such as `app.orders`. Names are convenient and necessary, but names are not the deepest identity of a durable database object.

ScratchBird uses UUID-based internal identity for catalog objects. A table can be renamed, displayed differently through a compatibility parser, or reached through a schema branch, while the engine still tracks the underlying object through durable identity.

This distinction matters for:

- renaming objects;
- resolving dependencies;
- enforcing grants;
- supporting recursive schemas;
- projecting compatibility catalogs;
- diagnosing failures;
- preserving transaction visibility.

Database Files And Database Systems

It is tempting to describe a database as "the database file." That is incomplete.

A database file can hold pages, metadata, and stored values, but a database system also includes:

- the engine code that understands the file;
- resource files such as character sets, collations, policies, and configuration;
- tools that create, open, verify, back up, restore, or diagnose data;
- parser or protocol packages that accept client requests;
- operational rules for startup, shutdown, refusal, and recovery.

For that reason, copying files, replaying text, or translating syntax is not enough to reproduce a database safely.

How ScratchBird Uses The Word Database

ScratchBird documentation uses **database** in two related ways:

Use	Meaning
Durable database	The engine-owned stored data, metadata, catalog identity, security state, and transaction state.
User-facing database	The namespace and compatibility view a connected client sees after authentication and parser routing.

A native SBsql session, an administrator, and a compatibility client may see different parts of the same durable database because each session can have its own parser profile, identity, grants, and schema root.

What A Database Is Not

A database is not only:

- a file;
- a query language;
- a network protocol;
- a parser;
- a set of tables;
- a backup stream;
- a catalog dump.

Those can all be part of a database product, but the database itself is the managed combination of durable storage, metadata, identity, transaction rules, security rules, diagnostics, and recovery behavior.

Why This Matters For ScratchBird

ScratchBird separates language from durable authority.

The parser handles text, wire protocol, dialect rules, and compatibility presentation. The engine handles catalog identity, descriptors, transactions, storage, recovery, and materialized authorization.

That separation is one of the foundations for the Convergent Data Engine model described in the next page.

Where To Go Next

- [What Is A Convergent Data Engine?](#)
- [How ScratchBird Implements A CDE](#)
- [Engine Parser Boundary](#)
- Schema Tree And Name Resolution (SBsql Language Reference — Syntax, page XXX)

What Is A Convergent Data Engine?

Purpose

This guide uses Convergent Data Engine, or CDE, as a database-engine category. A CDE is an engine design that attempts to bring capabilities that are often split across separate products into one shared engine substrate.

The term is descriptive. It is not a certification, benchmark result, compatibility guarantee, or production-readiness claim. A feature is available only when the current source tree, build target, configuration, tests, and release notes prove that it is available.

Why The Category Exists

Application systems often grow by adding specialized data products:

- a relational database for transactions;
- a document store for flexible records;
- a search service for text search;
- a vector store for embeddings;
- a graph store for relationships;
- a time-series store for measurements;
- a separate cache;
- separate governance, audit, and backup tooling;
- separate compatibility gateways for old applications.

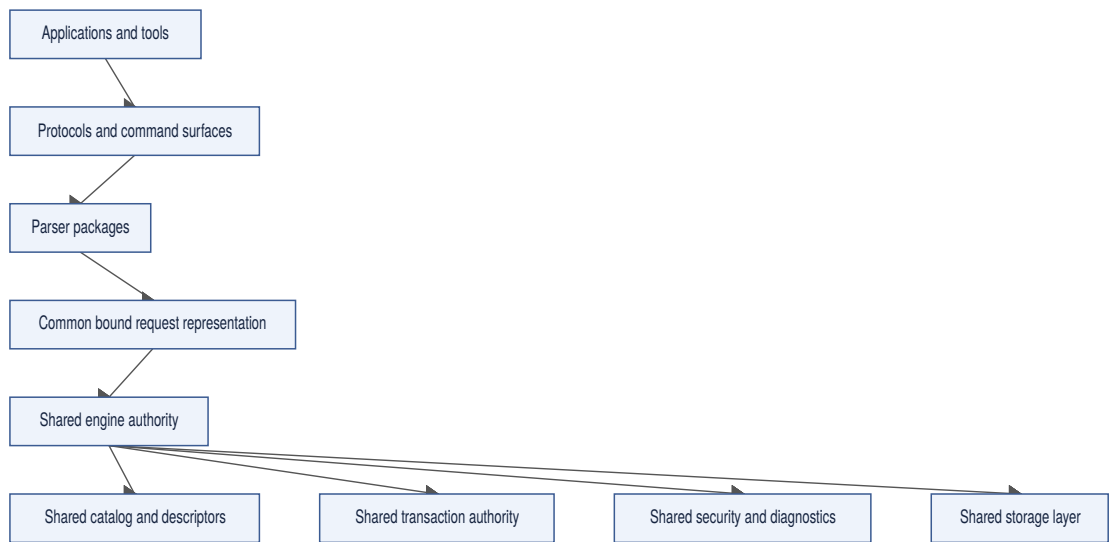
That can be the right architecture for many teams. It also creates work:

- data has to move between systems;
- security models diverge;
- transaction boundaries become harder to explain;
- backups and restores need product-specific handling;
- operators need several diagnostic systems;
- applications need several drivers and failure models.

A CDE design explores whether more of those capabilities can share one engine authority model.

The Basic CDE Idea

A CDE does not mean that every language becomes the same language. It means that different surfaces can lower into a common execution and authority layer.



The parser accepts a client language or wire protocol. The engine owns durable identity, security, transaction finality, recovery, and storage.

What Converges

The word convergent means several responsibilities can meet at one engine boundary.

Area	What Converges
Object identity	Objects can have stable engine identity even when surfaced through different names or parser profiles.
Data models	Relational, document, key-value, graph, vector, time-series, and other surfaces can share catalog and transaction authority where implemented.
Language surfaces	SBsql and parser packages can map different command styles into a shared bound request model.
Security	Authentication, authorization, protected material policy, masking, and row-level policy can be enforced by engine authority.
Transactions	Commit, rollback, visibility, and cleanup are not delegated to parser text.
Diagnostics	Refusals, unsupported features, policy failures, and runtime errors can be represented through shared message-vector behavior.
Operations	Startup, health checks, support bundles, configuration validation, and administrative commands can be handled as product surfaces instead of one-off utilities.

What Does Not Converge Automatically

A CDE is not a promise that every client sees the same language or the same behavior.

Non-Goal	Why
One universal SQL dialect	Different clients expect different syntax, defaults, catalogs, and diagnostic shapes.
Automatic compatibility with every engine	Each compatibility profile requires implementation and proof.
Parser authority over storage	Parser packages translate requests; they do not own transaction finality or durable catalog identity.
Unlimited feature availability	A surface must be implemented, configured, admitted by policy, and tested for the target build.
Silent behavior substitution	If a behavior is unsupported, unsafe, denied, or unlicensed, the system should refuse with a diagnostic rather than pretending success.

Examples Of Convergence

The examples below describe the architectural idea. They are not a statement that every listed surface is complete in every build.

User Need	CDE-Shaped Response
A native user creates a relational table.	SBsql parses the statement; the engine creates UUID-backed catalog objects and descriptors.
A compatibility client asks for its catalog tables.	The parser can project compatibility catalog rows while the engine remains the source of durable authority.
An application stores JSON and queries fields.	JSON descriptors and functions can be bound through the same expression and policy system as ordinary scalar values.
A vector query ranks candidates by distance.	Vector descriptors, functions, indexes, and execution operators can be tied back to engine catalog and policy rules.
A stored routine updates rows and emits diagnostics.	Procedural SQL lowers to engine-controlled operations and message vectors.

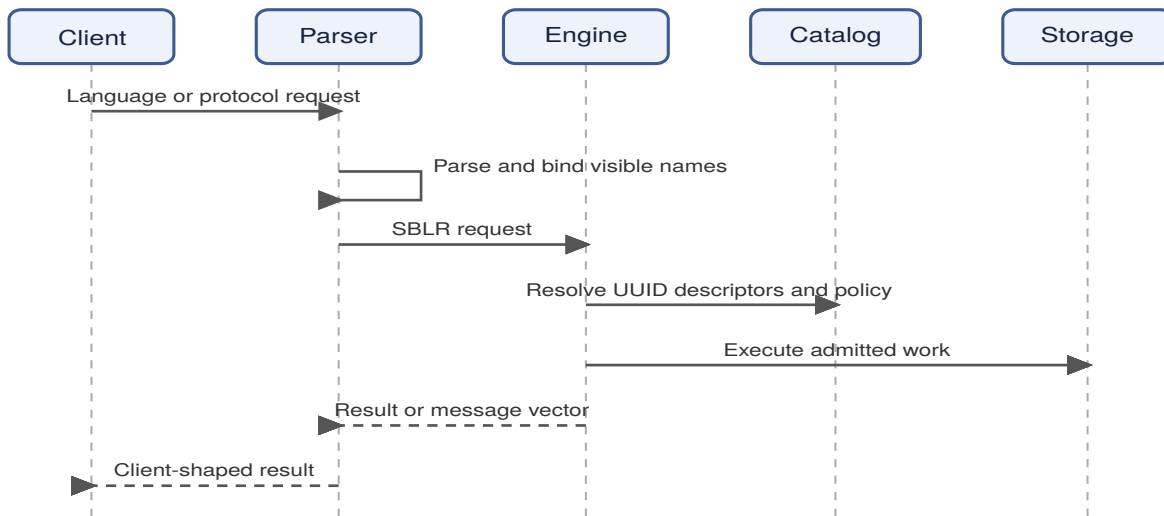
Why Parser Separation Matters

A CDE that supports multiple client families cannot let raw client text become the database engine.

ScratchBird uses parser packages as translators. A parser package can:

- accept the protocol or language it is designed for;
- apply parser-specific syntax rules;
- render parser-specific diagnostics where implemented;
- map names to visible object identities;
- lower accepted work into SBLR;
- refuse unsupported or unsafe behavior.

The engine then decides whether the bound request is authorized, type-correct, transactionally valid, and executable.



Compatibility Must Be Scoped

In this documentation, a parser profile means a compatibility route exists as a tracked product surface. It does not mean every behavior from that source ecosystem is complete.

Read compatibility cautiously:

- a parser should support only its intended client family;
- a compatibility parser should not silently accept unrelated dialects;
- reference-system compatibility defaults must be documented and tested per parser;
- unsupported or denied behavior should return a diagnostic;
- compatibility status depends on the current tests and release notes.

CDE And Operations

A CDE design also affects operators. If more behavior runs through one engine authority model, operations need shared answers to ordinary questions:

- Is the database ready?
- Is the session authorized?
- Which parser handled this request?
- Which object UUID was accessed?
- Which transaction controls visibility?
- Why was a request refused?
- Which support bundle evidence can diagnose the failure?

ScratchBird documentation treats those questions as part of the product, not as afterthoughts.

What To Verify Before Relying On A Feature

Before relying on a CDE feature in a specific build, verify:

- the component exists in the public source tree;
- the build target includes it;
- the parser or API route admits it;
- the engine has an implemented SBLR/runtime path;
- policy allows it;
- tests cover the behavior;
- documentation states any limitations;
- diagnostics are clear when it is refused.

Where To Go Next

- [How ScratchBird Implements A CDE](#)
- [Engine Parser Boundary](#)
- [SBsql And SBLR](#)
- [Choosing A Mode Summary](#)

How ScratchBird Implements A CDE

Purpose

ScratchBird is organized as a Convergent Data Engine by separating durable engine authority from parser, protocol, management, and operational surfaces. This page explains that model at a high level for users who are trying to understand what parts of the system they are interacting with.

This is an architectural overview, not a compatibility or deployment certification. A feature is usable only when it is implemented, built for the target platform, enabled by configuration and policy, and covered by the relevant tests for the release being used.

Design Summary

ScratchBird is built around a small number of boundaries:

- **SBcore** owns durable engine behavior.
- **Parsers** translate client language or protocol requests into engine requests.
- **SBLR** is the bound request representation passed toward engine authority.
- **Catalog identity** uses durable UUIDs rather than relying only on text names.
- **MGA transaction authority** belongs to the engine.
- **Security and policy** are materialized before work is admitted.
- **Operational tools** inspect, configure, diagnose, and manage the running system.

The result is a system where several client surfaces can exist without making any one client language the engine itself.



Main Components

The public names below are the user-facing names used in documentation and output. Internal file names may differ.

Component	User-Facing Role
ScratchBird Engine, or SBcore	The embedded engine library that owns catalog identity, descriptors, transaction authority, storage, recovery decisions, materialized security checks, and engine diagnostics.
ScratchBird IPC Server, or SBSrv	A local multi-user server process for clients on the same machine. It is useful when local processes need shared access without exposing a network listener.
ScratchBird Listener, or SBgate	The parser and parser-pool entry point used for network-facing client traffic. It routes accepted client work to the appropriate parser path.
ScratchBird Single Node Manager, or SBmgr	A single-node front door that can proxy authenticated connections to internal listener routes in managed deployments.
ScratchBird SQL, or SBsql	The native ScratchBird command language and script runner surface.
ScratchBird Core Parser, or SBParser	The parser package for native SBsql requests.
Compatibility parser packages	Standalone parser packages for specific client families. Each parser should understand only its intended client surface and lower accepted work to ScratchBird engine requests.
Administrative tools	Utilities for backup, security, diagnostics, conformance, policy, character set, collation, and related operational work where those tools are present in the release.
Resource files	Character sets, collations, time zones, policy files, configuration files, and other resources that make the built output usable as a product instead of only a binary.

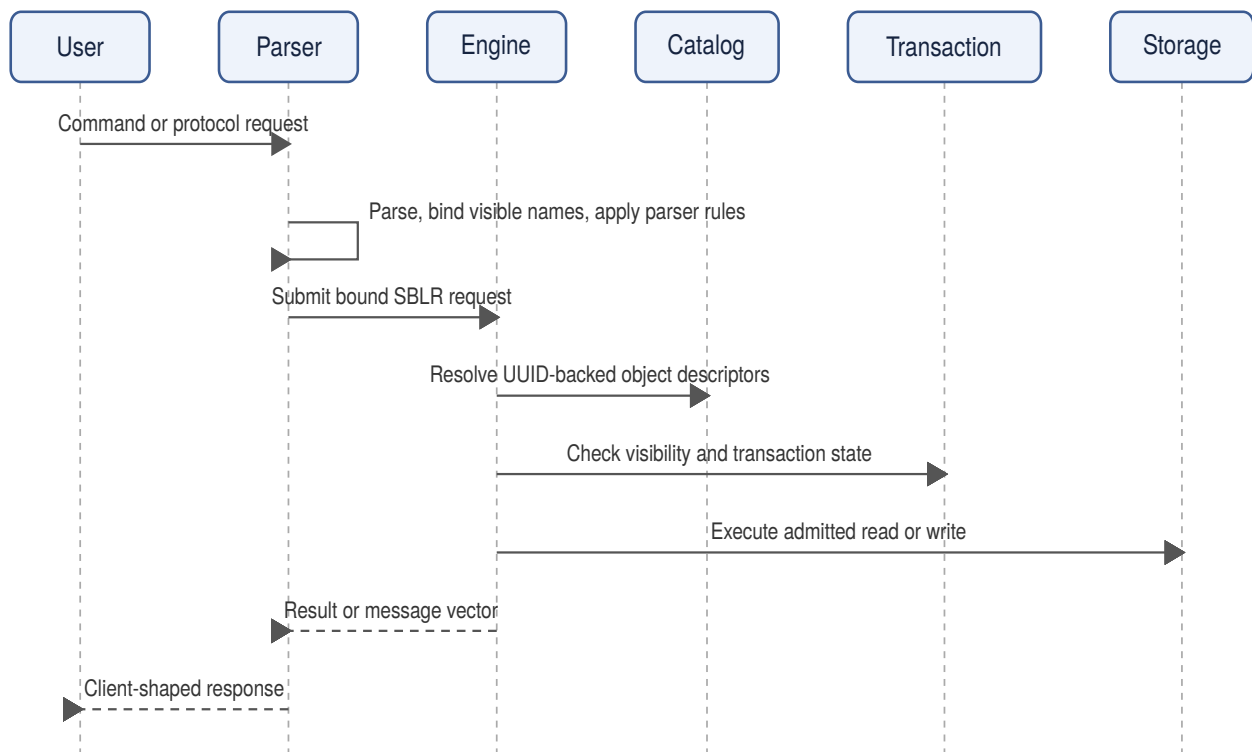
Engine Authority

ScratchBird treats the engine as the final authority for durable database behavior.

The engine owns:

- database create, open, close, and reopen behavior;
- object identity and catalog state;
- type descriptors and domain descriptors;
- transaction begin, commit, rollback, savepoint, visibility, and cleanup;
- storage pages, row storage, overflow storage, and filespace state;
- materialized authorization and policy checks;
- index identity and index maintenance;
- diagnostic message vectors;
- recovery and refusal decisions.

Parser packages can request work, but they do not own durable identity, final transaction state, storage recovery, or security authority.



Parser Separation

ScratchBird does not require every client to speak SBsql. Parser packages exist so different client families can be supported without moving storage authority into those parsers.

A parser package is responsible for:

- accepting the client language or wire protocol it is built for;
- applying that parser's syntax rules;
- mapping visible names to engine object identities;
- applying parser-local defaults before lowering the request;
- generating SBLR for supported operations;
- rendering results and diagnostics in the expected client shape where implemented;
- refusing unsupported, denied, unsafe, or out-of-scope behavior.

A parser package should not silently accept another parser's language. A compatibility parser is scoped to its reference-system client family. Native SBsql is the ScratchBird language surface.

SBLR Boundary

SBLR is the bridge between parsed client intent and engine execution. It is where text has already been parsed and bound into a structured request.

That boundary matters because it prevents raw client text from becoming durable authority. It also makes it possible for different parser packages to share engine behavior while preserving their own syntax and diagnostic presentation.

At a high level:

1. A client sends a request.
2. The selected parser accepts or refuses the request.
3. The parser binds names, values, types, and parameters that are visible to that session.
4. The parser emits SBLR for an engine-supported operation.
5. The engine admits, executes, or refuses the request based on engine authority.
6. The parser returns a client-shaped result or message vector.

Catalog Identity And Names

ScratchBird users work with names. The engine works with durable catalog identity.

Names are still important:

- they are what users type;
- they are what tools display;
- they are part of schema navigation;
- they can be projected differently by different parser surfaces.

Durable identity is deeper than the visible name. Engine catalog objects are tracked through UUID-backed identity and descriptors so the engine can preserve meaning across rename, dependency tracking, parser projection, security evaluation, and transaction visibility.

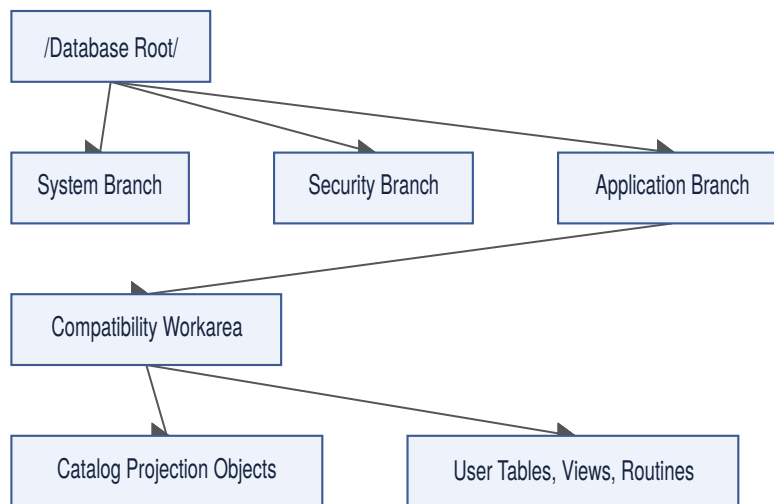
For example, a table may have a user-facing name inside a schema branch. If that table is renamed, dependencies and grants should follow the object identity rather than treating the renamed table as an unrelated object.

Recursive Schema Tree

ScratchBird schemas form a recursive tree. A schema can contain objects and child schemas. A session resolves names from its current authority, current schema, home schema, and parser-visible schema root.

This is important for compatibility and security:

- native SBsql can administer or query broader parts of the tree when authorized;
- a compatibility client can be sandboxed so its connected workarea appears to be the root;
- catalog projection objects can expose selected metadata without giving the user direct access outside the sandbox;
- object names can be resolved relative to the session instead of requiring one global flat namespace.



The exact objects visible to a session depend on parser route, authentication, authorization, policy, and schema root.

Data Model Convergence

ScratchBird's CDE model is intended to let different data shapes share engine authority where those surfaces are implemented.

Surface	How It Fits The CDE Model
Relational rows	Tables, columns, indexes, constraints, views, and query execution are represented through catalog descriptors and engine execution.
Documents	Document values and document functions can be described by types and operated on through admitted expression and query surfaces.
Graph-like relationships	Relationship-oriented objects can be modeled through catalog identity, references, indexes, and query surfaces where implemented.
Vector values	Vector descriptors and similarity operations can use engine type, function, and index machinery where implemented.
Time-series values	Time-oriented data can use ordinary storage, indexing, functions, and policy while retaining time-specific semantics where implemented.
Procedural SQL	Stored routines, triggers, functions, packages, and event-style behavior lower into engine-controlled requests instead of bypassing authority.
Operational data	Diagnostics, backup, restore, security, policy, and management surfaces are treated as product behavior with message-vector refusal paths.

The convergence point is not a single syntax. It is shared authority for identity, security, transactions, storage, and diagnostics.

Security And Sandboxing

A connected session is not automatically allowed to see every object in the database.

ScratchBird's model separates:

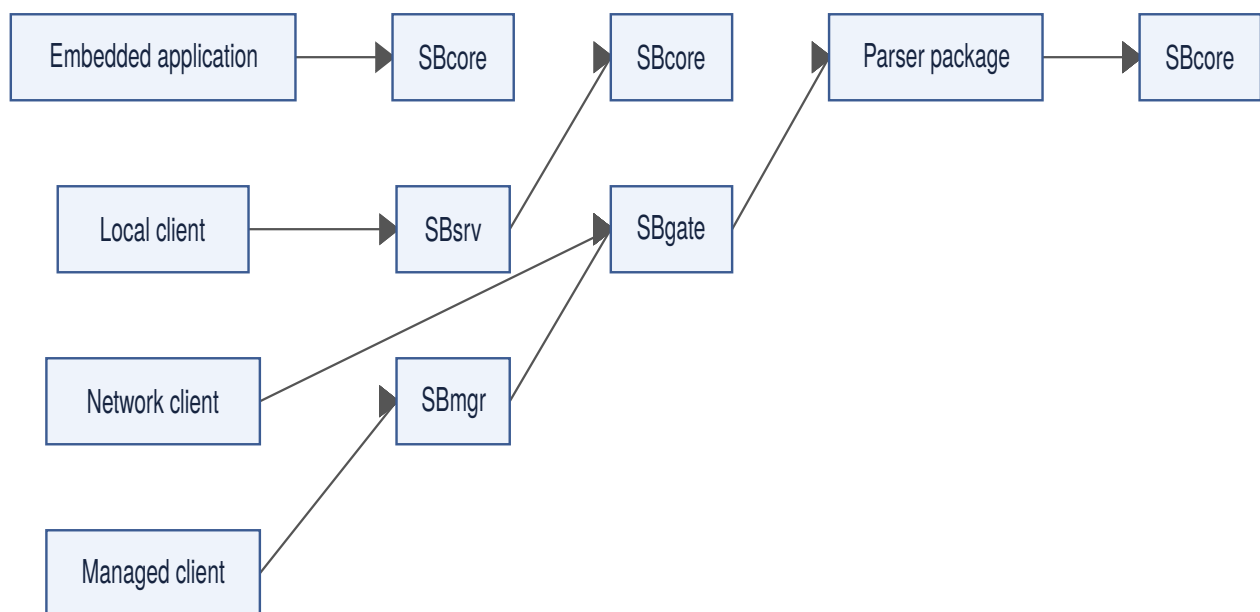
- authentication, which establishes the identity;
- authorization, which determines what that identity may do;
- parser-visible schema root, which shapes the namespace presented to the client;
- policy, which can further restrict rows, values, external access, operational commands, or protected material;
- catalog projections, which can show selected metadata without making the underlying objects directly accessible.

In a compatibility workarea, the client should experience that workarea as its database root. If metadata views show information that requires broader knowledge, that access belongs to the catalog projection object and its grants, not to unrestricted user authority.

Operating Modes

ScratchBird can be used through several operating modes. The correct mode depends on how the application connects, how many clients need access, where the trust boundary sits, and which binaries are available for the target platform.

Mode	Path	Typical Use
Embedded engine	Application -> SBcore	A process links to the engine and owns the local application boundary.
Single-node IPC server	Local client -> SBsrv -> SBcore	Several local processes need shared access without a network listener.
Standalone server	Client -> SBgate -> parser -> SBcore	Network clients use listener and parser routing.
Managed group deployment	Client -> SBmgr -> SBgate -> parser -> SBcore	Local installations need a managed front door and shared identity or policy integration.



Read the operating-mode pages before choosing a deployment shape.

Resources And Built Output

A usable ScratchBird build is more than a compiled executable.

Depending on the release and target platform, the output tree may need:

- runtime binaries;
- parser packages;
- character set definitions;
- collation definitions;
- time zone data;
- policy defaults;
- configuration templates;
- security provider configuration;
- diagnostics and support-bundle tooling;
- test and conformance assets used to prove the build.

Documentation should treat those resources as part of the product because a binary without its required resources may not behave like the tested release.

Git-Oriented Workflows

ScratchBird documentation may describe Git-oriented workflows and metadata where they are implemented. Git support is an operational and lifecycle concept; it does not replace database transaction authority, recovery behavior, or the engine catalog.

Use Git-oriented features only as documented for the release. Do not assume that a repository operation is the same thing as a database transaction, backup, restore, or repair operation.

Refusals And Diagnostics

An enterprise-style database surface must be able to say no clearly.

ScratchBird uses message-vector diagnostics so a request can distinguish between cases such as:

- syntax not accepted by the selected parser;
- feature unsupported by the selected parser;
- feature unsupported by the engine build;
- operation denied by policy;
- missing capability or unavailable component;
- authentication or authorization failure;
- unsafe request shape;
- recovery-required or operator-intervention state.

Clear refusal is part of the CDE model. It is better for the system to refuse an operation than to pretend to support behavior that has not been implemented or admitted.

What To Verify In A Release

Before relying on a ScratchBird capability, verify:

- the required component exists in the release output;
- the parser route is present and configured;
- the target platform is listed for that component;
- the feature is documented as implemented;
- the configuration and policy admit the operation;
- tests or conformance proof cover the behavior;
- refusal diagnostics are documented for unsupported cases.

This is especially important for compatibility parsers and operational commands, where the presence of a parser package does not automatically mean every possible behavior from that client family is implemented.

Where To Go Next

- [What Is A Database?](#)
- [What Is A Convergent Data Engine?](#)
- [Engine Parser Boundary](#)
- [SBsql And SBLR](#)
- [Recursive Schema Tree](#)
- [Embedded Engine](#)
- [Single-Node IPC Server](#)
- [Standalone Server](#)
- [Managed Group Deployment](#)
- Language Reference (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)

Understanding Multi-Generational Architecture (MGA)

Purpose

This page explains, in plain language, how ScratchBird keeps your data safe while many people read and write at the same time. The mechanism is called **Multi-Generational Architecture**, or **MGA**. You do not need to be a programmer to follow it; the analogies below build the intuition first, and the later sections connect that intuition to the real engine behavior.

MGA is the reason ScratchBird can let one person read a stable view of the data while another person is busy changing it, without either one waiting on the other. It is also the foundation for features described elsewhere in this guide, such as snapshot isolation, point-in-time history, and safe background cleanup.

For the architectural treatment of the same topic, see [Storage, Transactions, And Recovery](#). For the language-level transaction contract, see the Transaction Control (SBSql Language Reference — Syntax, page XXX) page in the Language Reference.

1. Always in a Transaction (The Safety Bubble)

Whenever you open the database, you are automatically placed inside a **Transaction**. Think of this as your own private safety bubble. Anything you do inside this bubble isn't real to the rest of the world until you hit "Save" (Commit). If something goes wrong, the bubble pops (Rolls back), and it's like it never happened.

2. Always in Snapshot Mode (The Time-Freeze Photo)

The moment your safety bubble opens, the database takes a digital snapshot of the entire system.

- You only see data that was officially saved **before** your snapshot was taken.
- Even if another user changes a row while you are working, your view remains frozen in time. You will never experience data shifting under your feet.

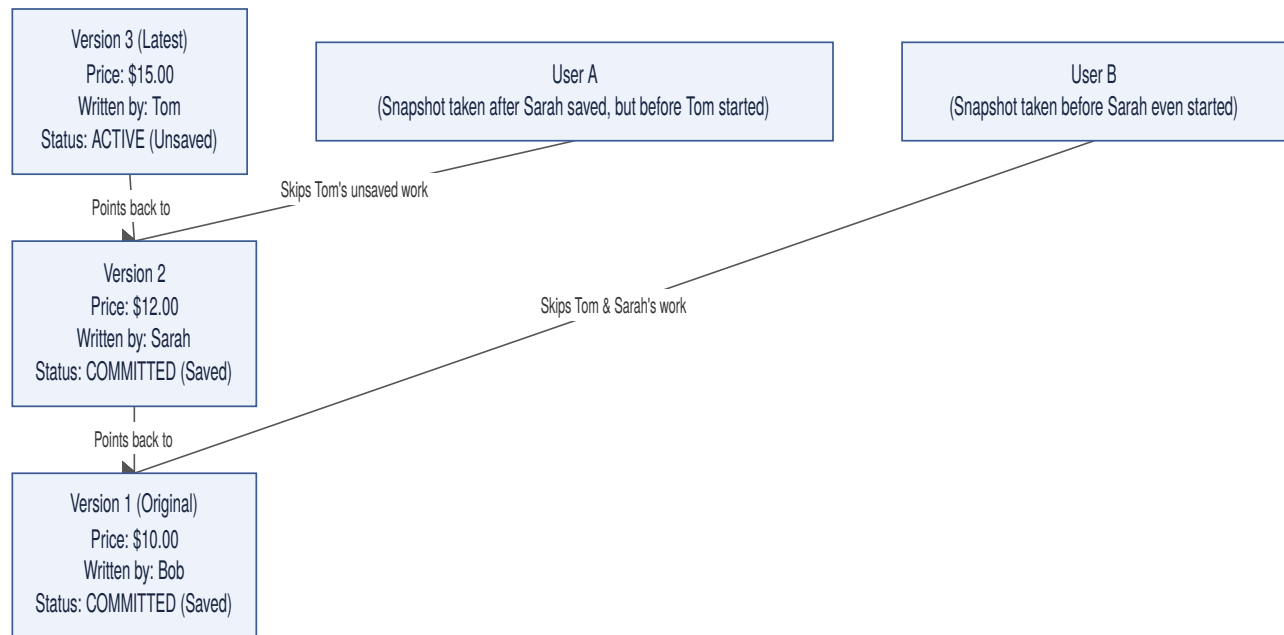
3. Non-Destructive Changes (The LiveJournal Page)

When you **Delete** or **Update** a row, the database *never* overwrites or destroys the original data on the disk.

- **An Update:** Writes a brand new version of the row right next to the old one, with a note saying: *"If you want the version before this, look over there."*
- **A Delete:** Doesn't erase anything. It just attaches a "Dead" sticky note to the row.

Visualizing the LiveJournal Page (Record History)

This diagram shows how different users see the exact same piece of data depending on when their "Snapshot Photo" was taken.



How the Database Decides What You See

When a non-programmer reads the "LiveJournal" page, the database acts as a smart filter by asking three simple questions:

1. **Is the latest version officially saved?** * Looking at the diagram, **Tom's Version 3** isn't saved yet (it's Active). The database immediately ignores it for other users.
2. **Was it saved *before* my snapshot photo was taken?**
 - **User A** took their photo after Sarah saved. They see **Version 2 (\$12.00)**.
 - **User B** took their photo a long time ago, before Sarah even started writing. The database rolls them all the way back to **Version 1 (\$10.00)**.
3. **What happens to the old stuff?**

Instead of blindly erasing old data to save space (which can corrupt active users' views), ScratchBird uses a careful, two-stage system: **Logical Cleanup** (deciding what is safe to delete) and **Physical Cleanup** (actually reclaiming the disk space).

It functions like a team of editors managing a shared historical journal. They won't delete an old draft until they are 100% sure no reader in the building is still looking at it.

1. Dealing with Long-Running Users (Transaction Pressure)

Because every user session is *always* inside a frozen "snapshot" transaction, a single user who leaves their connection open can block the system from cleaning up old data. ScratchBird manages this with a strict escalation policy.

The "Do Not Disturb" Boundary (The Horizon)

ScratchBird calculates a global **Cleanup Horizon**. This is the exact moment in time behind which data is safe to destroy.

- It checks active snapshots, unresolved data, and open sessions to find the absolute **oldest thing anyone cares about**.
- Anything newer than this horizon is strictly hands-off.

The Escalation Timer (Pressure Management)

If a user is idle but their open session is keeping the horizon from moving forward, a background manager starts a countdown clock to nudge them out of the way:

Time Elapsed	ScratchBird's Action
~5 Minutes	Warn: Sends a gentle warning to the client application.
~15 Minutes	Request Restart: Asks the application to recycle its transaction.
~20 Minutes	Request Reauth: Asks the user to re-authenticate.
~25 Minutes	Request Cancel: Asks to gracefully cancel the blocking work.
~30 Minutes	Force Replacement: If allowed by policy, it forcibly replaces the old transaction boundary with a fresh one to let cleanup proceed safely.

2. Dealing with Clutter (Page & Index Bloat)

ScratchBird handles old data versions incrementally in a series of careful checkpoints, ensuring background cleanup never slows down the user's active work.

Step 1: Logical Version Cleanup

The engine groups old row versions into a "candidate list." A row version is only eligible to be destroyed if:

1. It was explicitly deleted, rolled back, or replaced by a newer saved version.
2. The transaction that created it is older than the **Cleanup Horizon**.
3. It passes a rigorous safety check ensuring it isn't blocked by an active backup, a legal archive hold, or a recovery process.

Step 2: Physical Page Compaction

Once a row version is logically cleared, the physical cleanup coordinator steps in with the "reclaim evidence." It shrinks and packs the actual database pages, recycling the empty space. It refuses to do this unless it has airtight proof from Step 1.

Step 3: Index Maintenance

Indexes (the data phonebooks) are cleaned up just as carefully. ScratchBird checks the actual table snapshot before removing stale index entries from its ledger, ensuring search shortcuts never break.

Step 4: Debt Scheduling (The Budget)

To keep the database fast, cleanup is treated as a continuous, low-priority background task.

- It scores "cleanup debt" (where the most clutter is).
- If the user is doing heavy foreground work, the cleanup budget is automatically scaled back so it never starves the system of performance.

The Ultimate Rule of ScratchBird

Safety Over Space: If a long-running transaction or legal snapshot remains active and authorized, ScratchBird will **always** choose to retain the old versions and let the database grow rather than risk data corruption. It will pressure the blocker to move, but it will never compromise visibility rules.

How Do Other Database Engines Do This?

Most database systems use WAL - Write Ahead Logs, which is designed with a very different approach to operations. To understand the difference between these two systems without getting bogged down in computer science jargon, imagine you run a busy **Legal Archive Office** where team members are constantly changing and reading documents.

Standard WAL and **ScratchBird MGA** represent two completely different strategies for running this office so that files never get lost and workers don't trip over each other.

1. Standard WAL: The "Notebook and Eraser" Strategy

In a standard WAL (Write-Ahead Logging) office, there is one main master binder on the shelf, and one sequential logbook on the desk.

- **Making a Change:** When a worker wants to change a price from \$10 to \$12, they must write it in the logbook first: *"Entry 45: Changing page 5 from \$10 to \$12."*
- Only after it is written in the logbook are they allowed to go to the master binder, take an eraser, **scratch out \$10 and overwrite it with \$12.**
- **Changing Your Mind (Cancel):** If a worker gets halfway through a massive change and decides to cancel, it is a lot of work. They have to read the logbook backward and physically erase their mistakes in the master binder to restore the original numbers.
- **The Traffic Jam:** Because there is only one master copy of the page, if someone is in the middle of erasing and overwriting a line, nobody else is allowed to look at that page. Readers have to wait in line.

2. ScratchBird MGA: The "LiveJournal" Strategy

ScratchBird throws away the erasers. The binders act like a chronological journal where **nothing is ever crossed out or destroyed.**

- **Making a Change:** When a worker wants to change a price from \$10 to \$12, they leave the original line completely alone. They simply write a brand new line underneath it: *Price is \$12 (Written by Tom)*. Then, they draw a little arrow pointing back to the old line, creating a historical chain.
- **Changing Your Mind (Cancel):** Canceling is instant. If Tom decides to cancel his work, he doesn't have to erase anything. The office manager simply flips a switch marking Tom's name as "Invalid." The next person who reads the book sees Tom's line, sees his name is invalid, and completely ignores it.
- **No Traffic Jams:** Because old data is never destroyed, a reader can open a binder and read a frozen snapshot of the past while a writer is actively adding new lines at the bottom of the same page. No one ever has to wait in line to read.

3. How They Handle a Crisis

The biggest danger in a busy office is a worker who opens a book, starts a project, and then goes to lunch for three hours without closing it. Here is how both systems handle that crisis:

The WAL Crisis: The Logbook Explodes

Because the master binder only shows the *current* moment, the office cannot throw away or archive the desk logbook while a project is open—it might still need those logs to fix a mistake.

- **The Problem:** As other workers keep doing business, the desk logbook grows longer and longer, eventually spilling out of the room and consuming the entire building until there is no physical space left to stand.
- **The Result:** The office completely grinds to a halt.

The ScratchBird Crisis: The Binders Get Heavy

Because ScratchBird keeps historical lines on the page, if a worker keeps a project open for three hours, the office cannot clean up any old lines written during those three hours, because that worker might still look at them.

- **The Problem:** The pages get cluttered with old versions, and the binders get thick and heavy (page bloat).
- **The ScratchBird Solution:** ScratchBird has a built-in "Pressure Manager" that acts like an attentive office supervisor. If it sees a worker has gone to lunch and left a project open, a timer starts:
 1. **5 Mins:** The supervisor gives them a gentle nudge.
 2. **15 Mins:** Asks them to wrap it up and restart.
 3. **25 Mins:** Asks them to cancel.
 4. **30 Mins:** If allowed, the supervisor safely takes their paperwork, closes their frozen project, and opens a fresh one for them. This unlocks the backlog and allows the office cleaning crew to shred the unneeded historical lines, shrinking the binders back down.

Quick Summary

- **Standard WAL** is like a traditional ledger. It's great for quick, short changes, but it relies on erasing data in-place and can cause lines to form when the office gets busy.
- **ScratchBird MGA** is like a historical diary. It takes a little more paper because it keeps a history of changes directly on the page, but it ensures that nobody ever blocks anyone else from reading, and it has a smart supervisor to keep the clutter under control.

Beyond Simple MGA

1. No More "Spy Cameras" (Eliminating CDC Records)

In a traditional database, if you want to copy data to another system or run complex reports (ETL/Recursion), you have to install a "spy camera" called CDC (Change Data Capture). This camera watches the database constantly, writing down every single move a user makes into a separate, heavy stack of paperwork just so you can replay it later.

How ScratchBird Does It: The "Whodunit" Journal

Because ScratchBird's LiveJournal page *already* preserves every version of a row with the writer's name (Transaction ID) stamped on it, you don't need a spy camera.

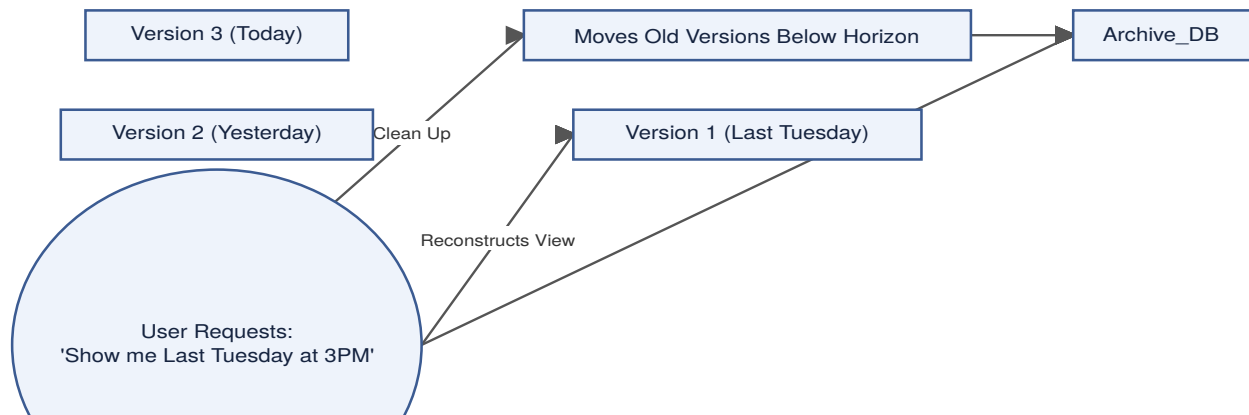
- The database record itself tells you exactly who has touched it since the last time you checked.
- Instead of sorting through a giant separate mountain of spy logs, a reporting tool can just look directly at the page and say: "*Show me lines written by anyone who arrived after Transaction 500.*" ---

2. The Database Time Machine (Point-in-Time Queries)

Normally, as we discussed, the office cleaning crew (Garbage Collection) shreds old lines once everyone goes home, to keep the binders from getting too thick. But what if you need to know exactly what the database looked like last Tuesday at 3:00 PM?

How ScratchBird Does It: The Archive Room

Instead of throwing old row versions into a paper shredder, ScratchBird's cleaning crew safely packs those old historical lines into **Archive Storage Binders** (Archive Filespaces).



When you ask ScratchBird to travel back in time:

1. You give it a date and time (e.g., *Last Tuesday at 3:00 PM*).
2. The database manager flips through its master schedule and figures out exactly which transaction bubble was active at that exact minute.
3. It opens up the main binders *and* the archive binders, ignores everything written after that deadline, and lets you see the world exactly as it looked at that moment.

3. DNA Tracking Across the Whole Building (Cluster-Wide UUIDs)

In a giant enterprise system, data isn't kept in just one binder; it's spread across a **Cluster** of different computers (like an archive office with 24 different rooms).

In a standard setup, if a row gets copied or moved to another room, it gets a new local line number, and its history is lost. It's like a person changing their identity every time they cross state lines.

How ScratchBird Does It: The Unchangeable Passport (UUIDs)

ScratchBird gives every single piece of data its own universal, lifelong fingerprint: a **UUID**.

- No matter which computer room a record moves to, splits into, or clones into across the entire global cluster, its UUID passport stays exactly the same.
- Because the historical journal tracks changes by UUID rather than by local page numbers, ScratchBird can follow the full family tree and history of a single record across the entire network of computers seamlessly.

Summary of the "Next-Level" Upgrade

By anchoring everything to permanent **UUID fingerprints** and moving old data to **Archive Filespaces** instead of destroying it, ScratchBird transforms the database from a simple storage box into a living history book. It gives you all the benefits of data tracking and time travel automatically, without slowing down the daily business of the office.

Engine Parser Boundary

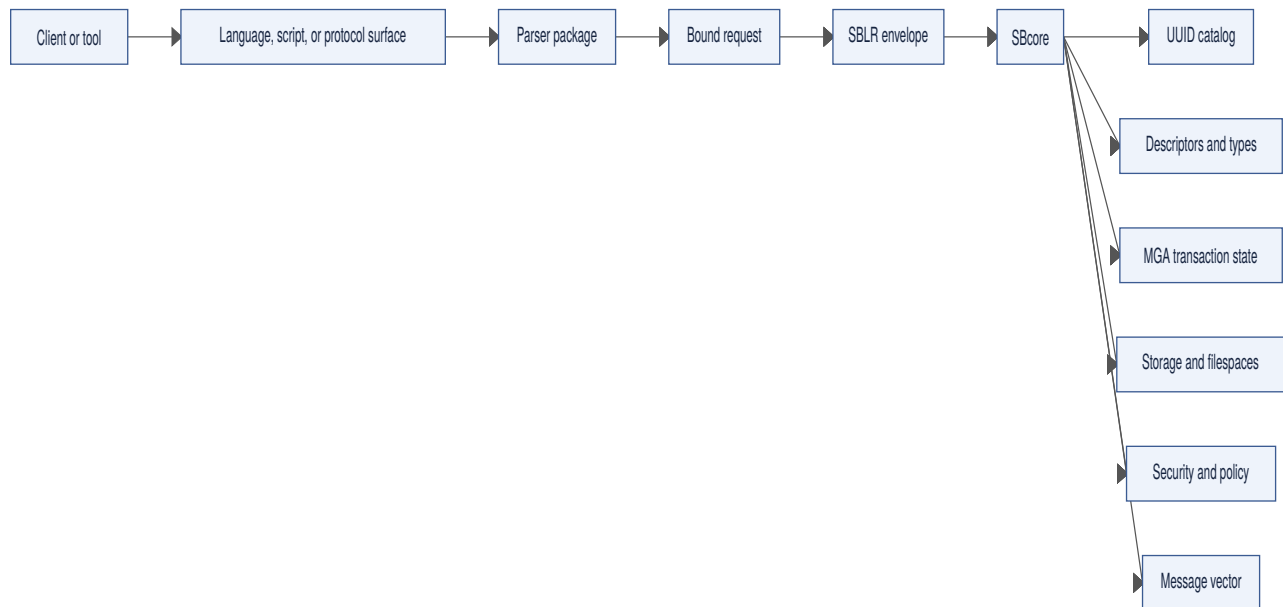
Purpose

After reading this page you will understand why ScratchBird draws a sharp line between the component that accepts your commands and the component that actually changes data. Understanding this boundary will help you reason about why the same SQL statement might be accepted by a parser but still refused by the engine, and why different client tools can share the same storage and transaction rules.

ScratchBird separates client language handling from durable database authority. A parser understands a command language, script format, or wire protocol. SBcore (the database engine) owns durable database behavior: catalog identity, descriptors (the engine's metadata records for objects and types), storage, transactions, authorization, diagnostics, and recovery decisions.

This page explains the boundary at an end-user level. It does not claim that every parser route or command surface is complete in every build.

The Boundary In One Diagram



The client sends text, frames, parameters, or tool commands. The parser decides whether that surface is accepted. The engine decides whether the resulting bound request — expressed as an SBLR envelope (the structured, engine-facing form of an accepted statement) — is valid, authorized, transactionally safe, and executable.

Why The Boundary Exists

Without a strict parser/engine boundary, every accepted language surface could become a separate database engine. That would mean transaction behavior, catalog identity, security, recovery, and diagnostics could each drift independently depending on which parser a client happened to use — the same underlying data could behave differently depending on how you connected to it.

ScratchBird avoids that by making parsers translators, not storage authorities. MGA (ScratchBird's transaction and visibility authority model) and all catalog state remain in SBcore regardless of which parser route a session uses.

Concern	Parser Package	SBcore
Syntax	Accepts or refuses the language or protocol it supports.	Does not treat raw client text as authority.
Names	Binds visible user names according to parser and session context.	Resolves admitted work to durable UUID-backed catalog identity.
Types	Parses literals and applies parser-visible type spelling.	Owns type descriptors, storage encoding, comparison, and coercion authority.
Defaults	Applies parser-specific defaults explicitly.	Stores explicit durable object descriptors.
Security	Carries session identity and visible context toward the engine.	Materializes authorization and policy before execution.
Transactions	Requests begin, commit, rollback, savepoint, or statement work.	Owns final visibility, cleanup, and recovery state.
Diagnostics	Renders client-shaped messages where implemented.	Produces authoritative message vectors for engine admission and execution.

Parser Responsibilities

A parser package is the client-facing component: it understands a specific language or protocol, applies its own syntax rules and defaults, and translates accepted work into an engine request. A parser also binds visible names through the session's schema root (the branch of the schema tree that this session is permitted to see and navigate). What a parser must never do is write database files, decide final transaction visibility, bypass catalog identity, or grant access outside the session's authority.

Specifically, a parser package is responsible for:

- accepting only the language or protocol surface it is designed to support;
- rejecting unrelated dialects or malformed protocol input;
- tokenizing and parsing accepted input;
- applying parser-specific identifier, literal, and default rules;
- binding visible names through the session's schema root and search context;
- lowering admitted work into SBLR or another engine-facing request shape;
- rendering rows, status, and diagnostics in the client-facing shape where implemented;
- refusing unsupported, denied, unsafe, or unavailable behavior clearly.

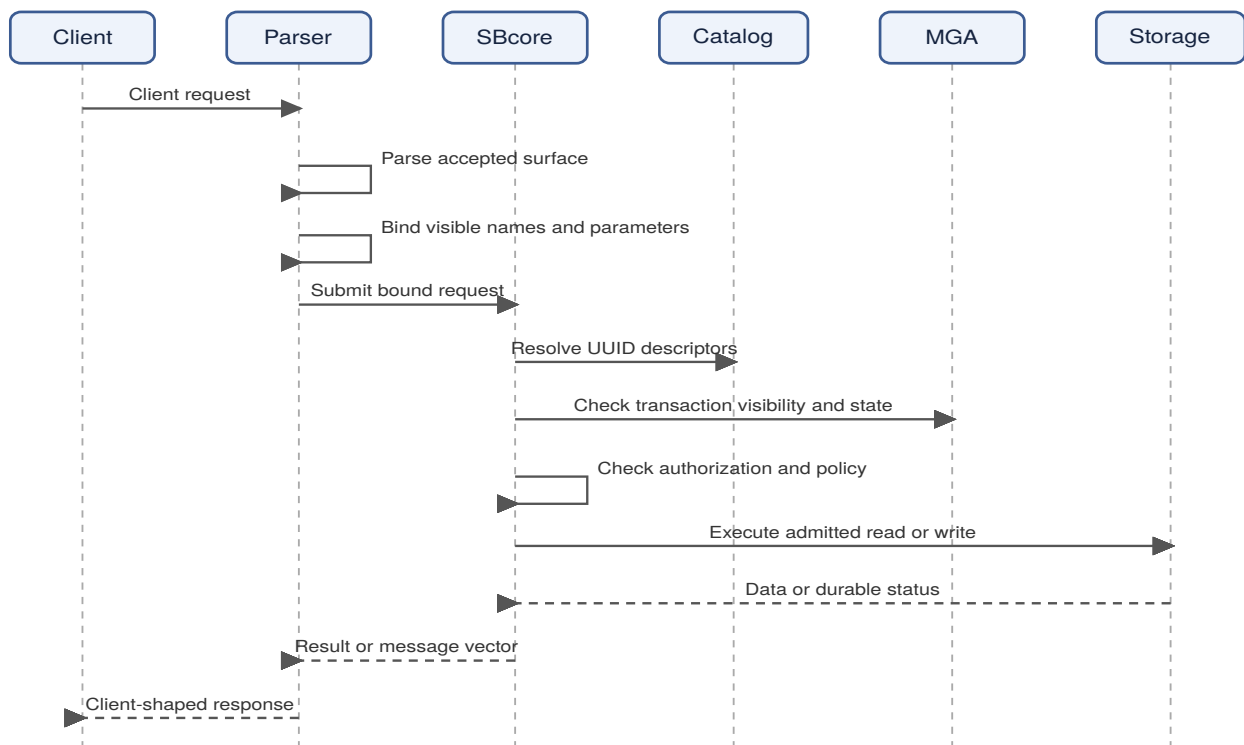
Engine Responsibilities

SBcore is responsible for:

- database create, open, close, and reopen behavior;
- durable object UUIDs and catalog state;
- descriptor validation and type authority;
- transaction begin, commit, rollback, savepoint, visibility, cleanup, and recovery;
- storage pages, row storage, overflow storage, and filespace state;
- index identity and index maintenance;
- materialized authorization and policy checks;
- support-bundle evidence and engine diagnostics;
- final admission, execution, or refusal of a bound request.

The engine may refuse a request even after the parser accepts the syntax. That is expected behavior: parser acceptance and engine admission are independent decisions.

Statement Lifecycle



This lifecycle is the same architectural pattern whether the client uses native SBsql or a compatibility parser route.

Parser-Specific Defaults

Different parser routes may expose similar objects with different default behavior. Examples include:

- identifier folding;
- default schema selection;
- datatype precision or scale defaults;
- string literal interpretation;
- index null ordering;
- generated constraint names;
- catalog projection shape;
- diagnostic wording.

Those defaults must be lowered into explicit engine requests. The engine should not have to guess which parser default created a durable object later.

Sandboxing

The parser boundary is also a sandbox boundary.

A session has a visible schema root and authorization context. A compatibility parser may present one schema branch as the connected database. Native SBsql may expose broader tree navigation when the identity is authorized.

The parser cannot simply spell a path outside the visible root and gain authority. Name resolution must end in a visible object identity that the engine admits for the session.

Refusal Behavior

Knowing where a refusal comes from helps you diagnose problems. A failure at the binder means a name was not visible; a failure at engine admission means the operation was unauthorized or unsupported by this build; a failure at execution means a runtime constraint was violated. The message vector (ScratchBird's structured diagnostic carrier) should tell you which stage refused and why.

Refusal can happen at several points.

Location	Example Refusal
Listener or entry point	Unknown route, unavailable parser, authentication failure.
Parser	Syntax not accepted, malformed protocol input, unsupported parser feature.
Binder	Name not visible, ambiguous object, invalid type coercion.
Engine admission	Unauthorized object, denied policy, unsupported engine operation.
Execution	Constraint violation, transaction conflict, storage refusal, recovery-required state.

Good refusal is part of compatibility. The system should not return success for work it did not implement or admit.

What This Boundary Does Not Mean

The parser/engine boundary does not mean:

- all parser packages are complete;
- all parser packages support the same commands;
- parser syntax is native SBsql syntax;
- the engine executes raw SQL text;
- syntax acceptance guarantees execution;
- a compatibility route can bypass ScratchBird storage or transaction rules.

Always check the parser documentation, build output, tests, and release notes for the specific route you intend to use.

Where To Go Next

- [SBsql And SBLR](#)
- [Recursive Schema Tree](#)
- [Identity, Authentication, And Authorization](#)
- [Storage, Transactions, And Recovery](#)
- [Reference-System Compatibility](#)
- Parser To SBLR Pipeline (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)

SBsql And SBLR

Purpose

After reading this page you will understand what SBsql is, what happens to your statements before they reach the engine, and why the text you write is not the same thing as the durable record the database keeps.

SBsql is the native ScratchBird command language. SBLR (the Bound Request form) is the structured, engine-facing representation of an accepted and name-resolved statement — it is what the parser hands off to SBcore (the engine) after it finishes parsing and binding. You write SBsql; the engine receives SBLR.

The practical rule is:

SBsql text is user input.

SBLR is the engine-facing request.

SBcore state is the authority.

This page explains the relationship at a high level. Use the Language Reference for exact syntax.

Where SBsql Fits

SBsql is intended to be the native user language for ScratchBird. Rather than copying the exact syntax of another database system, it is designed to express ScratchBird concepts — like the recursive schema tree, UUID catalog identity, MGA (the engine's transaction and visibility authority model), and workarea sandboxing — directly and without workarounds.

SBsql can be used for:

- creating and changing database objects;
- querying data;
- inserting, updating, deleting, merging, and streaming data where implemented;
- controlling transactions;
- managing schemas, privileges, policies, and diagnostics where authorized;
- defining routines, triggers, domains, types, and sequences where implemented;
- inspecting catalog and operational state;
- running scripts.

Native SBsql should express ScratchBird concepts directly instead of copying a reference-system language.

Where SBLR Fits

SBLR is not ordinary end-user SQL. It is the structured request form emitted after a parser accepts and binds a request. Think of it as the stage where user-facing names (like `app.notes`) have been resolved to their durable catalog identities and where parser-specific defaults have been made explicit — so that SBcore can execute the work without having to know anything about which parser produced it.

SBLR carries information such as:

- operation kind;
- object identity or name bindings;
- datatype descriptors;
- expression trees;
- parameter bindings;
- transaction context;
- routine bodies or callable operations where supported;
- diagnostics and refusal routing;
- policy-relevant context.

Users normally do not write SBLR directly in an interactive session.

Statement Pipeline



Each stage can refuse the request. For example, parsing can reject malformed syntax, binding can reject an invisible name, and engine admission can reject unauthorized work.

Example: Create A Table

The examples below show how the same text you type is translated into engine-owned metadata. The original SQL text is source material; the catalog — not the text — is what persists after the transaction commits.

An SBsql statement might look like this:

```
create table app.notes (  
    note_id bigint not null,  
    note_text text not null,  
    created_at timestamptz not null,  
    constraint pk_notes primary key (note_id)  
);
```

The parser does not store that text as the durable table. Instead, the accepted request is bound into engine-owned metadata:

- parent schema identity;
- table object identity;
- column descriptors;
- datatype descriptors;

- constraint descriptors;
- default and nullability rules;
- transaction visibility;
- authorization checks.

The original text can be useful as source material or reference, but the engine catalog is the durable authority.

Example: Query Data

An SBsql query:

```
select note_id, note_text
from app.notes
where note_id > 10
order by note_id;
```

The parser and binder must determine:

- which object `app.notes` refers to;
- whether the session can see it;
- which columns are projected;
- which operator and type rules apply to `note_id > 10`;
- what ordering is requested;
- which parameters, collations, or functions are involved;
- whether the engine admits the resulting plan.

The result is returned as rows or as a controlled diagnostic.

Context-Sensitive Language

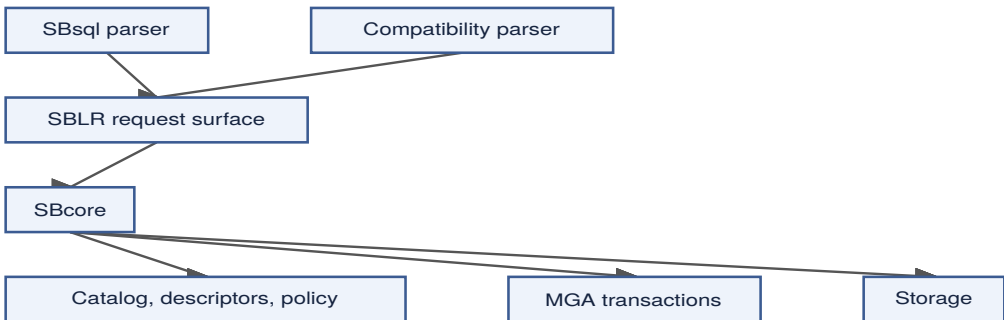
SBsql is intended to remain context-sensitive, with as few reserved words as practical. That means a word may be usable as an identifier in one context and a command word in another context.

Practical guidance:

- prefer clear object names;
- avoid names that look like common command words;
- quote identifiers only where the Language Reference says to do so;
- qualify names in administrative scripts;
- do not assume another parser's keyword rules apply to SBsql.

SBLR And Parser Packages

Native SBsql is not the only possible parser source. A compatibility parser can accept its own client language or protocol and lower accepted work to the same engine authority model.



This lets parser packages preserve their client-facing syntax and defaults without becoming independent engines.

SBsql language profiles use the same boundary. A localized SBsql profile may change spelling, phrase order, diagnostics, completion hints, and rendering templates, but it is normalized into canonical parser elements before UUID binding and SBLR lowering.

Diagnostics

SBsql and SBLR participate in structured diagnostics. When something goes wrong, the message vector (ScratchBird's structured diagnostic carrier) identifies which stage refused the request and why.

Stage	Example Diagnostic
Lexing	Invalid token.
Parsing	Statement shape not accepted.
Binding	Name not visible, ambiguous name, invalid argument count.
Type checking	Unsupported coercion or operator/type combination.
Engine admission	Authorization denied or feature unsupported by the build.
Execution	Constraint violation, transaction conflict, storage refusal.

The user sees the diagnostic through the client or tool. The underlying message vector should preserve enough structure for support and automation where available.

What SBsql/SBLR Does Not Mean

An SBsql grammar entry or an SBLR operation name does not, by itself, prove:

- the implementation is complete;
- every platform build includes it;
- every parser can call it;
- every operation is enabled by policy;
- the surface is production-ready;
- the operation bypasses engine authorization.

Availability must be checked against the current build, tests, configuration, and release notes.

Where To Go Next

- [Engine Parser Boundary](#)
- [First SBsql Session](#)
- SBsql Language Profiles (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)
- Script Tokens And Identifiers (SBsql Language Reference — Syntax, page XXX)
- Operators (SBsql Language Reference — Syntax, page XXX)
- Procedural SQL (SBsql Language Reference — Syntax, page XXX)
- Language Reference (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)

Storage, Transactions, And Recovery

Purpose

After reading this page you will understand how ScratchBird thinks about storage, transactions, and what happens when a database is reopened after shutdown or interruption. This matters because the same rules apply regardless of which parser or client tool you use — SBcore (the database engine) owns these decisions, and understanding that boundary will help you write safer scripts and migrations.

ScratchBird stores data through SBcore. Parser packages, client tools, and scripts can request changes, but they do not own storage finality, transaction visibility, or recovery decisions.

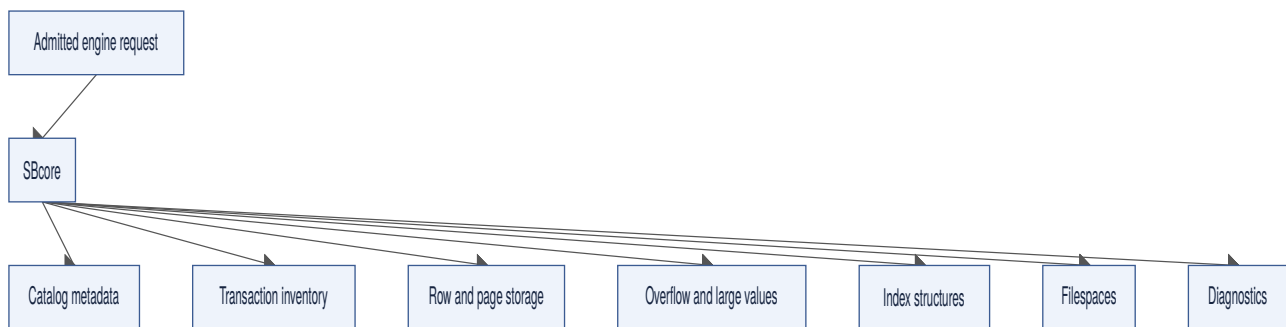
This page explains the high-level model for users. It is not a durability certification or crash-safety claim for every build and platform. Read release-specific test results before relying on a deployment for real data.

Engine-Owned State

A common misconception is that the SQL text you write, or the session state in a client tool, is the "real" database. It is not. The durable database exists inside SBcore, and that is the authoritative record that persists across session disconnects, process restarts, and recovery events.

SBcore owns durable database state such as database header and create/open state, filespace metadata (the descriptions of storage areas the database uses), page and row storage, catalog rows, object UUIDs, type and domain descriptors, index metadata and index contents, overflow or large-value storage metadata, transaction inventory, visibility and cleanup state, materialized authorization state, recovery-required or refusal state, and support-bundle evidence. Client-visible SQL text and parser state are not the durable database.

Storage Model At A High Level



The exact on-disk format belongs to implementation and release documentation. The user-facing rule is that SBcore is the component that understands and maintains durable storage.

Filespaces

A filesystem is a named storage area known to the engine — it is how SBcore tracks where on disk (or in another storage medium) durable data lives. A database can use filesystem metadata to describe multiple storage areas, separate primary data from secondary structures, and control how storage is organized.

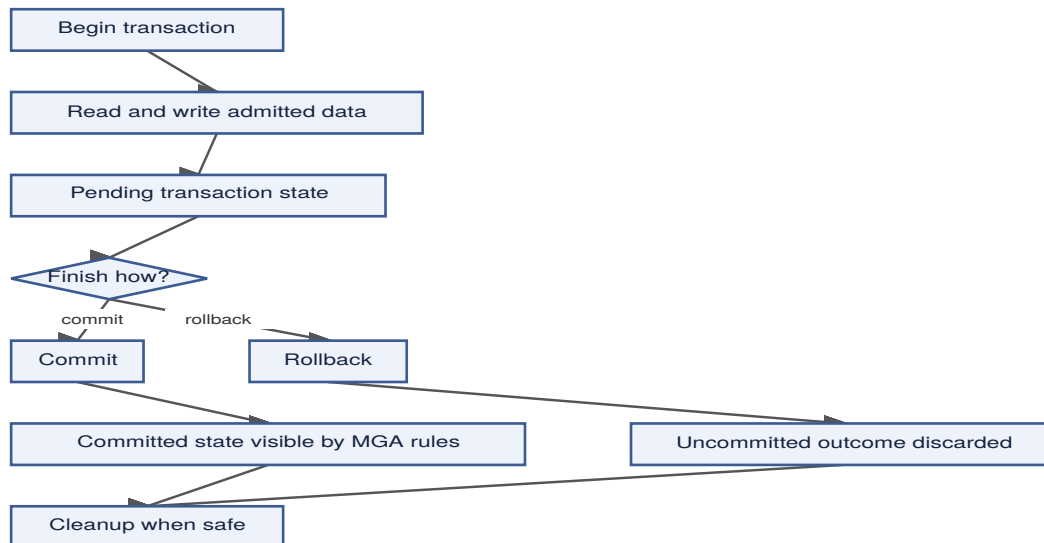
At a high level, filesystem behavior should answer:

- which storage areas belong to the database;
- which filesystem is primary or special for database bootstrap;
- whether a filesystem can be attached, detached, promoted, moved, or removed in the current release;
- what diagnostics are returned when storage is unavailable or unsafe;
- how recovery determines whether filesystem metadata is consistent.

Use the Language Reference for exact filesystem commands and supported lifecycle operations.

Transactions

A transaction is a boundary around work: changes you make within a transaction either all become durable (commit) or all disappear (rollback). A session can request transaction actions, but the engine owns final visibility — your parser or client tool asks; SBcore decides.



ScratchBird documentation refers to the transaction and visibility authority model as MGA. MGA governs which committed or uncommitted versions a transaction can see, when cleanup can happen, and how conflicts are resolved. For an end user, the core point is simple: commit, rollback, visibility, and cleanup are engine decisions. See [Understanding MGA](#) for a plain-language explanation of how MGA works with analogies.

Transaction Actions

Common transaction actions include:

Action	Meaning
Begin	Start a transaction context where required by the session mode.
Commit	Request that admitted changes become final according to engine rules.
Rollback	Request that uncommitted changes be discarded.
Savepoint	Mark a point within a transaction that can be rolled back to where supported.
Release savepoint	Remove a savepoint marker where supported.
Autocommit	Let the session or tool commit statement work automatically according to documented rules.
Prepare	Enter a prepared transaction state where supported by the engine and selected route.

Exact syntax and availability are described in the Language Reference.

Visibility

Visibility decides which committed or uncommitted versions a transaction can see. This is not an abstract concern: if two sessions are running concurrently, each needs a consistent view of the data — one session should not see half-written rows from another session that has not yet committed. Visibility rules handle this consistently across all parser routes and client tools.

Visibility rules affect newly inserted rows, updated rows, deleted rows, catalog objects created or dropped inside transactions, index contents, cleanup decisions, long-running readers, and recovery after reopen. Visibility is not decided by the parser. A parser can ask for work; SBcore determines the transactionally valid view.

Commit And Reopen

A practical first durability check is a commit-and-reopen test:

1. Create or open a disposable database.
2. Begin a session.
3. Create a schema and table.
4. Insert rows.
5. Commit.
6. Detach.
7. Stop the runtime if the selected mode uses one.
8. Reopen the same database.
9. Query the committed rows.

```
select count(*) as note_count from app.notes;
```

This proves more than an in-memory result. It confirms that the selected mode can reopen the database and see committed state for that basic workflow.

Recovery

Recovery is the engine's process for determining a safe state after normal shutdown, interruption, or uncertain durable state. When a database is reopened, SBcore inspects its own metadata to decide whether the stored state is consistent enough to admit normal work. The goal is to never silently serve data from an inconsistent state — it is safer to refuse access and require operator action than to pretend uncertain data is valid.

Recovery can lead to different outcomes:

Outcome	Meaning
Open normally	Durable metadata and transaction state are consistent enough to admit normal work.
Open read-only or restricted	The engine can expose limited access while preventing unsafe writes where supported.
Recovery required	The engine requires a recovery path before ordinary work proceeds.
Operator action required	The engine refuses to decide silently and requires administrative intervention.
Fail closed	The engine refuses access because safe state cannot be determined.

Silent inconsistency is the state to avoid. A clear refusal is safer than pretending that uncertain data is valid.

Parser Boundary And Storage

Parser packages can request storage-changing operations. They do not write pages directly.

That matters for compatibility routes:

- a parser can accept client syntax;
- a parser can lower admitted work to SBLR;
- the engine still enforces transaction, storage, authorization, and recovery rules;
- physical page-copy formats or low-level repair commands should not bypass SBcore through a parser route;
- logical streams must be interpreted as admitted operations, not as direct file edits.

Diagnostics

Storage and transaction diagnostics should identify the kind of problem without leaking protected material. The message vector (ScratchBird's structured diagnostic carrier) is what reaches the client or tool when something is refused or fails.

Useful diagnostic categories include: database open refused, unsupported filespace operation, storage path unavailable, transaction conflict, transaction state invalid for the command, recovery required, authorization denied, policy denied, unsupported physical operation through a parser route, and message vector redacted.

What This Page Does Not Claim

This page does not claim:

- a specific crash matrix is complete;
- every filespace lifecycle operation is implemented;
- every parser route supports every transaction action;
- every platform build has the same durability proof;
- a logical backup is the same as a physical page copy;
- client tools can repair storage directly.

Use the current build, tests, and Language Reference for exact behavior.

Where To Go Next

- [Understanding MGA](#) — plain-language explanation of how MGA transaction visibility works, with analogies
- [First Database](#)
- Transaction Control (SBsql Language Reference — Syntax, page XXX)
- Filespace (SBsql Language Reference — Syntax, page XXX)
- Database (SBsql Language Reference — Syntax, page XXX)
- Backup, Restore, Replication, Migration (SBsql Language Reference — Syntax, page XXX)
- [Diagnostics And Support Bundles](#)

Recursive Schema Tree

Purpose

After reading this page you will understand why ScratchBird organizes schemas as a nested tree rather than a flat list, and why that choice matters for security, compatibility, and database organization.

In many database systems, all schemas share a single level — every schema is directly visible to every connected user (subject to grants). ScratchBird takes a different approach: schemas form a tree, so an application can live under its own branch, a compatibility parser can see only its assigned workarea (a schema branch presented as the client's visible database root), and system objects are separated from user objects by design. The recursive schema tree is central to native SBsql administration, compatibility workareas, sandboxing, catalog projections, and durable UUID-backed object identity.

Basic Shape

The names below are explanatory labels, not a required database layout.

```
database_root
|-- system
|   |-- catalog
|   |-- security
|   |-- diagnostics
|   `-- storage
|-- users
|   |-- public
|   `-- home
|-- applications
|   |-- app
|   |   |-- tables
|   |   |-- routines
|   |   `-- policy
|   `-- audit
`-- workareas
    |-- compatibility_area_a
    `-- compatibility_area_b
```

Engine identity is UUID-based. The visible names are labels resolved by the session.

Why Recursive Schemas Exist

Recursive schemas let ScratchBird represent several ideas without flattening them into one global namespace.

Need	How The Tree Helps
Application organization	Application objects can live under a branch.
Administrative separation	System, security, diagnostics, and storage metadata can be separated from user objects.
User home areas	A user's default namespace can be a branch rather than a single global schema.
Compatibility workareas	A parser can present one branch as the client's database root.
Catalog projections	Metadata views can be placed where the intended users can see them.
Policy scoping	Policy can be attached or reasoned about by branch.
Migration staging	Imported or converted objects can be staged in a separate branch before promotion.

Durable Identity Versus Path Names

Path names like `applications.app.tables.notes` are convenient for humans, but they are not what SBcore uses as the authoritative record. Every durable object is identified by a UUID, and the path name is just a label that resolves to that identity during name binding. This separation is what makes renames safe: the object's identity does not change, only its visible label does.

A path such as `applications.app.tables.notes` is user-facing. The durable object is represented by catalog identity and descriptors (the engine's metadata records for objects, types, and constraints).

That distinction matters because:

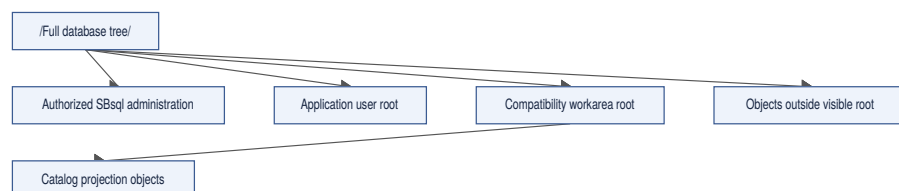
- an object can be renamed;
- a branch can be moved or reorganized where supported;
- parser routes can render names differently;
- dependencies should follow object identity;
- grants should apply to the intended object;
- transaction visibility controls whether a catalog change is visible.

Names are necessary for users. UUID identity is necessary for durable engine authority.

Session Views

One of the most practical consequences of the tree model is that different sessions can be shown different portions of it. An administrative user may navigate the full tree. An application user may only see the branch their application lives under. A compatibility parser session may see its workarea as if it were the entire database. This is part of the security model, not a display feature — the visible root limits which objects a session can name and access.

Different sessions can see different roots.



An authorized administrative SBsql session may see broad portions of the tree. A normal application user may see an application branch. A compatibility parser session may see its workarea as the root. The visible root is part of the security model, not just a display preference.

Schema Context Variables

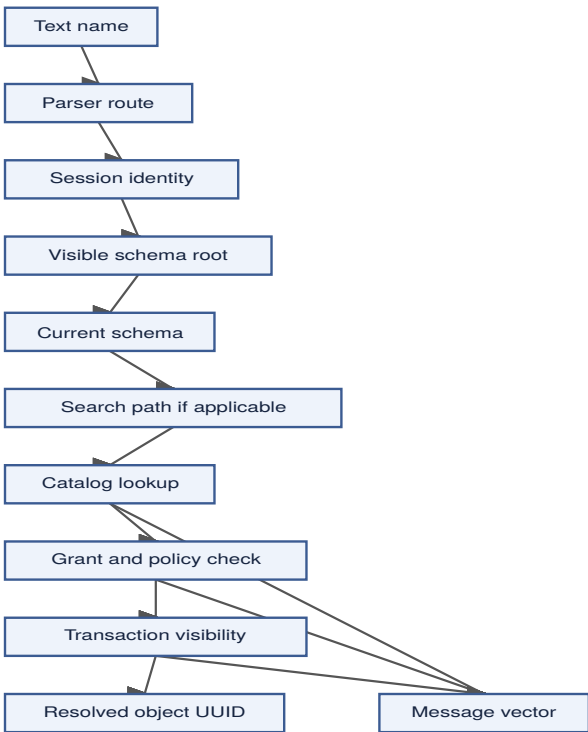
ScratchBird documentation uses several schema-context ideas.

Concept	Meaning
Database root	The top of the durable database tree.
Parser-visible root	The root presented to the selected parser route.
Home schema	The schema associated with a user, service, or configured workarea.
Current schema	The default schema for unqualified names in the current session.
Search path	An ordered set of schemas used by commands that allow path-based lookup.
Object parent schema	The schema that owns a specific object's local name.

The exact inspection and assignment syntax belongs in the Language Reference.

Name Resolution

Name resolution turns text into object identity.



An unqualified name such as `notes` may resolve through the current schema. A qualified name such as `app.notes` gives more path information. Neither form bypasses visibility, grants, policy, or transaction state.

Compatibility Workareas

A compatibility workarea is a schema branch presented to a parser route as the client-visible database root. The parser route sees a familiar namespace; the rest of the ScratchBird tree remains outside the client's visible scope. This is how ScratchBird can host multiple parser routes — each with its own default namespace expectations — on the same underlying database without those routes interfering with each other.

This lets the parser show a client a familiar namespace without giving that client direct access to the entire ScratchBird tree.

```
database_root
|-- workareas
|   |-- accounting_compat
|       |-- catalog_projection
|       |-- tables
|       |-- views
|       |-- routines
|-- internal
    |-- not_visible_to_that_client
```

A catalog projection can expose selected metadata if the projection object has authority. That is different from giving the connected user direct access outside the workarea.

Current Schema Examples

An SBsql session resolves unqualified names through the current schema path. The command that changes the current schema is release-specific; use `show schema path;` to inspect the active path.

```
create schema app;
show schema path;

create table app.notes (
    note_id bigint not null,
    note_text text not null
);

select note_id, note_text
from notes
order by note_id;
```

The unqualified name `notes` resolves relative to the current schema path. A more explicit script can use qualified names:

```
select note_id, note_text
from app.notes
order by note_id;
```

Use qualified names for administrative scripts and migrations when ambiguity would be expensive.

Object Lifecycle In The Tree

Object lifecycle operations interact with the tree.

Operation	Tree Effect
Create schema	Adds a branch under a parent schema.
Create object	Adds an object under a parent schema.
Rename object	Changes a visible label while preserving durable identity where supported.
Move object	Changes parent context where supported and authorized.
Comment on object	Adds descriptive metadata without changing authority.
Drop object	Removes or marks the object according to transaction visibility and dependency rules.
Describe or show	Presents visible metadata through the current parser route.

Dependency and authorization checks should prevent unsafe changes.

Practical Guidance

For new SBsql work:

- create application schemas deliberately;
- avoid placing application objects at the database root;
- qualify names in migrations;
- avoid names that differ only by case;
- do not rely on catalog projections as direct access authority;
- document the intended schema root for each application or parser route;
- verify name resolution after renames;
- include explicit `order by` in result checks when order matters.

Where To Go Next

- [Schemas, Objects, And Names](#)
- [Identity, Authentication, And Authorization](#)
- [Engine Parser Boundary](#)
- Schema Tree And Name Resolution (SBsql Language Reference — Syntax, page XXX)
- Schema Statements (SBsql Language Reference — Syntax, page XXX)

Identity, Authentication, And Authorization

Purpose

After reading this page you will understand why ScratchBird treats identity, authentication, and authorization as distinct stages — and why that separation matters for both security and flexibility.

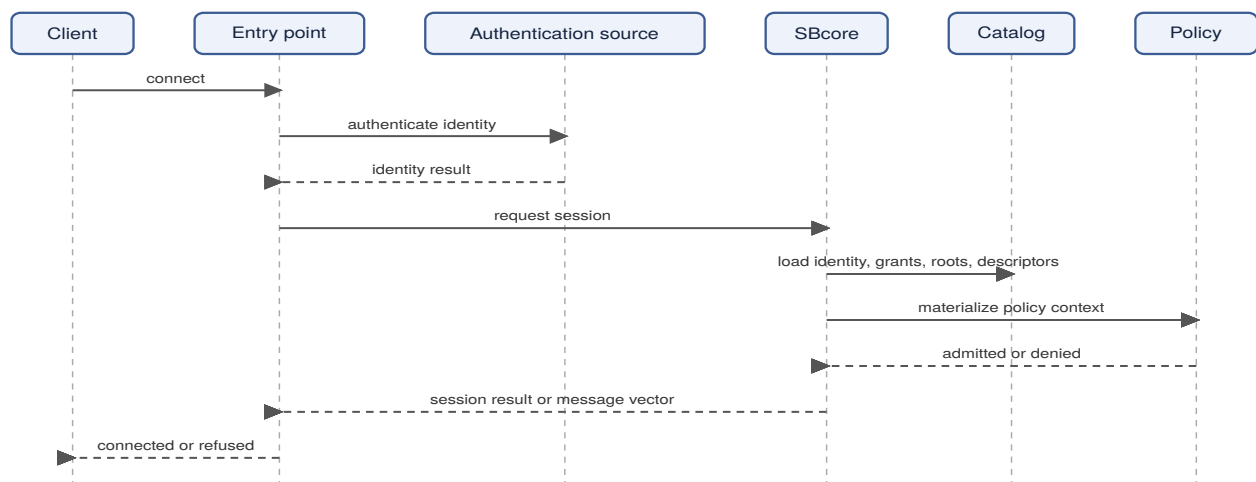
Many systems conflate these ideas: if a client can reach the server and supply a password, it is "in." ScratchBird draws sharper lines. Reaching a listener does not establish identity. Establishing identity does not guarantee authorization. Being authorized for some objects does not mean access to all objects. Each stage is a distinct check, and failure at any stage returns a controlled diagnostic rather than silently reducing access.

This page explains the high-level model. Exact providers, commands, and policy options depend on the current build and configuration.

Core Terms

Term	Meaning
Identity	The user, service, agent, or principal UUID associated with a session or operation.
Authentication	The process of proving the identity.
Authorization	The grants, roles, schema roots, object visibility, and policy rules available to that identity.
Session	A connected execution context with identity, parser route, transaction state, and schema context.
Principal	A user, role, group, service, or other authority-bearing entity.
Grant	Permission given to a principal or object.
Policy	A rule that can admit, deny, mask, restrict, or require diagnostics for an operation.
Sandbox	The visible namespace boundary for a session or parser route.
Protected material	Sensitive values that must be referenced, stored, redacted, and used through controlled mechanisms.

Connection Flow



Authentication proves who the session claims to be. Authorization decides what that identity may do after it is known. Notice that the engine loads the identity's grants, schema roots (the branches of the schema tree visible to this identity), and policy context before deciding whether to open the session — authorization is materialized at session start, not looked up on each individual request.

Identity

Identity should be durable and unambiguous. ScratchBird documentation commonly describes authority-bearing identities as UUID-backed.

An identity may represent:

- an interactive user;
- a service account;
- an administrative principal;
- a background agent;
- a group or role relationship;
- a parser or tool route acting on behalf of a user.

The identity attached to a session matters for:

- current and home schema;
- visible schema root;
- object grants;
- row-level policy;
- protected-material access;
- external access policy;
- support-bundle redaction;
- diagnostic detail.

Authentication

Authentication establishes the identity. Depending on build and configuration, authentication may be local, shared, delegated, or tool-mediated.

The documentation should be read cautiously:

- a named provider must exist in the build;
- configuration must admit it;
- the target platform must support it;
- diagnostics must make failures clear;
- secrets must not be passed as ordinary parser text unless a documented mechanism allows it.

Authentication failure should return a controlled diagnostic and should not open a database session.

Authorization

Authorization controls what the authenticated identity can do.

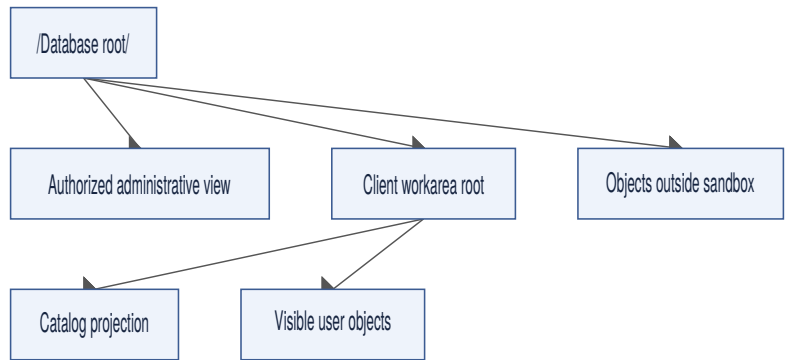
Authorization can include:

- database attach rights;
- schema visibility;
- object privileges;
- routine execution rights;
- catalog projection visibility;
- parser route admission;
- external access policy;
- protected-material use;
- row-level security;
- masks and filtered views;
- administrative operation rights.

Authorization is materialized before engine execution. A parser can present the request, but SBcore owns final admission.

Schema Roots And Sandboxes

Authorization includes namespace scope.



A native administrative SBsql session may see broad parts of the schema tree when authorized. A compatibility parser session normally sees its assigned workarea as the root. The client should not be able to name arbitrary objects outside that root.

Catalog projection objects can be special: they may expose selected metadata from outside a user's direct sandbox if the projection object itself has been granted that authority. That does not give the user direct access to the underlying objects.

Grants, Roles, And Policies

Grants and roles describe permissions. Policies can add contextual rules.

Mechanism	Typical Use
Grant	Allow a principal to select, insert, update, delete, execute, create, alter, or administer a specific object or object class.
Role	Group privileges so they can be assigned and activated consistently.
Schema root	Limit what namespace branch a session can see.
Row-level policy	Limit which rows are visible or changeable.
Mask	Transform protected output values.
External access policy	Control whether a session or routine may reach files, network routes, or other external resources.
Protected-material policy	Control whether a session may reference, unwrap, rotate, or use sensitive values.

Policy should fail closed when a safe result cannot be determined.

Parser Route And Authority

Parser routes do not grant authority by themselves.

A parser route can affect:

- language accepted;
- catalog projection shape;
- default schema behavior;
- visible workarea;
- diagnostic rendering;
- feature refusal.

The authenticated identity and engine authorization still decide whether the resulting operation is admitted.

Message Vectors

ScratchBird uses message-vector diagnostics to represent failures and refusals. A message vector (the structured diagnostic carrier that SBcore produces when something is refused or fails) should tell you which stage denied the request — authentication, session setup, object visibility, privilege, policy, or sandbox. This matters because the remediation differs: a missing privilege requires a grant, while a sandbox denial requires checking the session's visible root.

Common identity and authorization diagnostics include: authentication failed, identity provider unavailable, session denied, parser route denied, object not visible, privilege missing, policy denied, sandbox denied, protected material unavailable, external access denied, and diagnostic redacted. The exact rendering can differ by parser or tool, but the refusal should be controlled.

Operational Guidance

For early deployments and tests:

- use explicit identities instead of anonymous defaults;
- keep parser routes scoped to what is being tested;
- avoid broad grants in examples;
- qualify names in administrative scripts;
- verify denied cases as well as allowed cases;
- confirm support-bundle redaction before sharing diagnostics;
- keep raw secrets out of scripts and ordinary configuration files;
- document which identity source a build is using.

What This Page Does Not Claim

This page does not claim:

- a particular authentication provider is available;
- every policy surface is complete;
- every parser renders the same diagnostics;
- all external access is allowed;
- a connected client can administer every database object;
- sandboxed users can inspect unrelated schema branches directly.

Check the current build, configuration, and Language Reference before relying on a security surface.

Where To Go Next

- [Recursive Schema Tree](#)
- [Engine Parser Boundary](#)
- [Schemas, Objects, And Names](#)
- [Configuration Basics](#)
- Security And Privilege Statements (SBSql Language Reference — Syntax, page XXX)
- Policy, Mask, And RLS (SBSql Language Reference — Syntax, page XXX)

Git-Oriented Workflows

Purpose

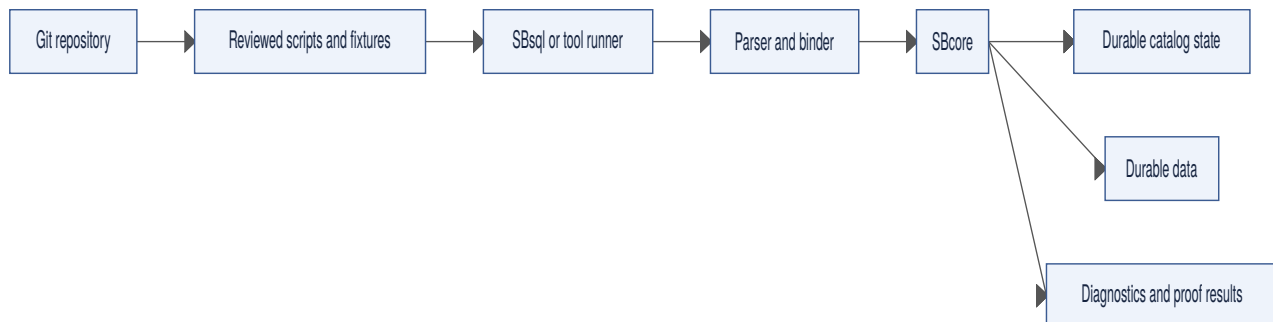
After reading this page you will understand what kinds of database-related files belong in a Git repository, what does not belong there, and — crucially — why Git and the database engine are different systems that should not be confused with each other.

Git is a natural fit for the source assets you write and review before running them against a database: scripts, fixtures, configuration templates, and expected results. What Git cannot do is replace the database. It does not own transaction history, catalog state, backup durability, authorization, or recovery decisions. This page describes the boundary between those two worlds.

Git is useful for managing scripts, configuration templates, fixtures, expected results, and documentation. Git does not replace database backups, recovery, transaction history, authorization, support bundles, or engine-owned catalog state.

ScratchBird also provides a distinct, opt-in capability — **external Git catalog versioning** — that lets you export a snapshot of the catalog into a form an external Git repository can track, diff the live catalog against a tracked snapshot, and produce a rollback *plan*. This is a controlled review-and-versioning convenience, **not** a change of authority: Git never executes against the database, and applying a planned change still flows through the engine's authorized catalog API. That capability is described in its own section below; the rest of this page covers the everyday source-control workflow.

The Core Distinction



Git tracks the request material and expected proof artifacts. SBcore owns the durable result after admitted execution.

External Git Catalog Versioning

Beyond tracking hand-written scripts, ScratchBird can export the **catalog itself** as a set of content-hashed artifacts that an external Git repository can version, review, and diff over time. This lets a team keep a Git-tracked history of catalog structure and compare or reconcile a live database against a committed snapshot — while the engine remains the sole authority over what actually changes.

This capability is **opt-in and policy-gated**. The engine refuses external-git operations unless the request carries `external_git_policy:enabled` (or `allow_external_git_versioning:true`); without it the operation is refused with `external_git_policy_required`.

What The Engine Provides

Operation	Surface	Role
EXPORT CATALOG ARTIFACT	SBsql statement (requires <code>right.catalog_read</code>)	Exports catalog objects as <code>sb.catalog.artifact.v1</code> rows, recorded under <code>sys.catalog.artifacts</code> .
IMPORT CATALOG ARTIFACT	SBsql statement (requires <code>right.catalog_mutate</code>)	Applies an authorized catalog artifact through the engine — the only admitted way to <i>apply</i> a change.
Export external Git snapshot	Engine artifact API / SBLR opcode <code>artifact.external_git.export_snapshot</code>	Emits a <code>sb.external_git.catalog_snapshot.v1</code> manifest plus one content-hashed object row per catalog object.
Diff external Git snapshot	SBLR opcode <code>artifact.external_git.diff_snapshot</code>	Compares the live catalog to a candidate snapshot, classifying each object as <code>unchanged</code> , <code>modified</code> , <code>added_in_candidate</code> , or <code>removed_from_candidate</code> .
Plan external Git rollback	SBLR opcode <code>artifact.external_git.rollback_plan</code>	Produces a rollback <i>plan</i> (actions such as <code>restore_current_catalog_artifact</code> , <code>reject_candidate_only_object_until_authorized_catalog_create</code> , or <code>no_action_required</code>) — it does not apply anything.

The three `external_git.*` operations are exposed through the engine artifact API and SBLR opcodes, not as typed SBsql statements; the EXPORT/IMPORT CATALOG ARTIFACT statements are the SBsql-level entry points. Each object row carries a stable `content_hash`; the engine recomputes it and rejects a snapshot whose supplied hash does not match (`external_git_snapshot_hash_mismatch`), is missing object identity (`external_git_snapshot_object_required`), or repeats a UUID (`external_git_snapshot_duplicate_uuid`).

The Authority Boundary (Why This Is Safe)

Every external-git result carries explicit authority evidence, and the boundary is enforced, not advisory:

- `external_git_versioning = convenience_snapshot_review_only` — versioning and review only.
- `git_runtime_authority = false` and `external_git_repository_authority = false` — Git never executes and is never an authority.
- `catalog_runtime_authority = ScratchBird_catalog_api` and `mga_transaction_authority = local_mga_transaction_inventory` — the engine catalog API and MGA keep authority.
- A diff row is marked `requires_authorized_catalog_import = true`, and the rollback apply route is `authorized_catalog_api_not_git_repository` — to actually apply a reconciliation you use `IMPORT CATALOG ARTIFACT` through the engine, never a direct Git apply.
- Any attempt to claim direct authority — `git_runtime_authority: true`, `external_git_direct_authority: true`, or `external_git_direct_apply: true` — is refused with `external_git_authority_forbidden`.

In short: Git can hold a versioned, diffable history of your catalog and you can plan a rollback from it, but the engine is the only thing that ever changes the database, through its authorized, MGA-governed catalog API. Object identity in these artifacts is UUID-based (`identity_authority = uuid`), so a snapshot tracks durable identity rather than just names.

For the operator workflow and the full operation/format reference, see the Operations and Administration and Language Reference manuals linked at the end of this page.

What Belongs In Git

Git is appropriate for human-reviewed and reproducible artifacts such as:

- schema creation scripts;
- migration scripts;
- rollback scripts where the project maintains them;
- seed data for development or tests;
- test fixtures;
- expected result files;
- parser compatibility inputs;
- configuration templates;
- policy templates without secrets;
- documentation;
- generated proof summaries that are intended for review;
- release notes and upgrade notes.

These files should be written so another user can understand what they request before executing them.

What Does Not Belong In Git By Default

Git should not be treated as ordinary storage for:

- live database files;
- temporary database files;
- local build output;
- staged release artifacts unless the release process explicitly tracks them;
- raw support bundles that may contain sensitive operational evidence;
- secrets, passwords, keys, or tokens;
- local machine paths;
- generated caches;
- logs with protected material;
- physical backup page copies.

If a project intentionally tracks a generated artifact, document why it is tracked and how it is regenerated.

Recommended Repository Shape

A project using ScratchBird can keep database source assets organized without mixing them with live storage.

```
project_root
|-- database
|   |-- schema
|   |-- migrations
|   |-- seed
|   |-- policy_templates
|   `-- expected_results
|-- tests
|   |-- sbsql
|   |-- fixtures
|   `-- proof
`-- docs
    `-- database_notes
```

This is only an example layout. The important rule is to keep source material separate from live database files and local generated output.

Migration Scripts

A migration script is a reviewed request to change the database. It is not the durable change by itself — the change only becomes durable after the script is executed and the engine commits the transaction. This distinction matters because two teams could have identical migration scripts yet end up with databases that differ in catalog identity, UUID assignments, or row contents depending on the history of each database.

A good migration workflow records:

- the intended precondition;
- the requested schema or data changes;
- the expected postcondition;
- the transaction boundary;
- the verification query or proof;
- the rollback or recovery plan where applicable;
- the minimum ScratchBird build or feature surface required.

Example structure:

```
-- migration: add note status
-- intent: add a status column for application filtering

alter table app.notes
  add column note_status text not null default 'open';

select note_id, note_status
from app.notes
order by note_id;

commit;
```

Use exact syntax from the current Language Reference when writing production scripts.

Expected Results

Expected result files are useful when a script should produce deterministic output.

For example, a test can record:

- row count;
- column names;
- column types;
- ordered result rows;
- expected diagnostic class for an invalid request;
- expected refusal when policy denies an operation.

When row order matters, scripts should request it explicitly with `order by`.

Configuration Templates

Configuration templates can live in Git when they do not contain secrets and when environment-specific values are clearly separated.

Good template behavior:

- use template variables for local paths and secret references;
- keep raw secrets out of the file;
- document required resource files;
- make parser route admission explicit;
- make diagnostics and redaction policy explicit;
- keep platform-specific notes separate when needed.

Git And Database Identity

ScratchBird uses UUID-backed catalog identity. Git tracks text files. These are fundamentally different things, and confusing them leads to subtle problems in migrations and dependency tracking.

Concretely:

- a script can request `create table app.notes`;
- the engine creates or modifies durable catalog objects;
- a later rename may keep the same durable object identity;
- a Git diff of scripts is not the same as a catalog diff;
- replaying a script against a different database may not produce the same durable identity.

Treat Git as source control for requests, not as the catalog.

Git And Transactions

Git commit history is not database transaction history.

Git Concept	Database Concept
Git commit	Versioned source change in a repository.
Database commit	Engine admission of a transaction outcome.
Git revert	Source-level reversal of a repository change.
Database rollback	Discard uncommitted database work.
Git branch	Source-control line of development.
Schema branch	Database namespace branch.

The terms can sound similar, but they are different systems with different authority.

Git And Backups

Git is not a database backup system.

Git can help reproduce scripts and expected states, but a database backup must preserve the database state according to the documented backup and restore surface. Logical backup, logical restore, import, export, and migration behavior should be handled through the relevant ScratchBird tools or SBsql commands where implemented and admitted.

Review Checklist

Before merging database-related source changes, review:

- Does the script state its intent?
- Does it use qualified names where clarity matters?
- Does it avoid raw secrets?
- Does it include a transaction boundary?
- Does it include verification queries or expected diagnostics?
- Does it avoid relying on implicit row order?
- Does it avoid server-local paths unless the operation is explicitly intended and admitted?
- Does it avoid claiming feature availability that the current build does not prove?

Where To Go Next

- [SBsql And SBLR](#)
- [Storage, Transactions, And Recovery](#)
- [First SBsql Session](#)
- [Backup, Restore, And Data Movement Overview](#)
- Script Tokens And Identifiers (SBsql Language Reference — Syntax, page XXX)

Choosing A Mode Summary

Purpose

Before you configure anything, you need to know which shape fits your application. ScratchBird can run as a library inside your process, as a local service that several programs share, or as a network-accessible server — and those shapes involve meaningfully different components, lifecycle responsibilities, and security considerations.

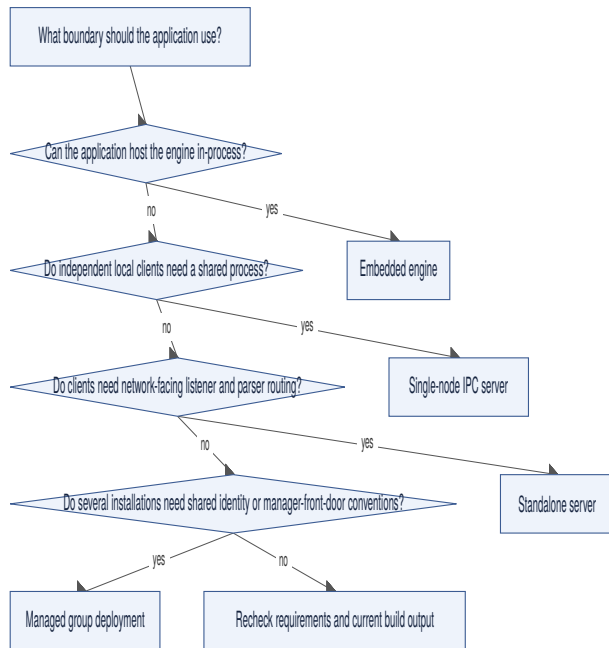
This page orients you to the four operating modes and helps you pick the right one to read first. It is not a sizing guide, benchmark, support statement, or deployment recommendation. The right mode depends on the current build output, target platform, configuration, parser packages, resource files, tests, and the application boundary you intend to use.

The Four Shapes

Mode	Short Description	Main Entry	Network Listener Required	Read More
Embedded engine	The application links to SBcore (the core database engine) and uses the engine in its own process.	SBcore library/API	No	Embedded Engine
Single-node IPC server	Local clients connect to SBsrv (the local server process) through a shared IPC endpoint.	SBsrv IPC endpoint	No	Single-Node IPC Server
Standalone server	Clients connect through SBgate (the listener and router) and a parser package that handles their protocol.	SBgate and parser packages	Yes	Standalone Server
Managed group deployment	Multiple installations use SBmgr (the manager front-door) for consistent identity and policy conventions.	SBmgr plus local services	Depends on local service shape	Managed Group Deployment

Decision Flow

The following flowchart starts from the boundary your application needs and works outward. Most new deployments land at embedded or single-node IPC; the more complex shapes add real value only when their specific capabilities are required.



Quick Recommendations

Need	First Mode To Evaluate
One application owns all access and can carry engine lifecycle responsibility.	Embedded engine.
Several local processes need a shared database service without accepting network traffic.	Single-node IPC server.
A client connects over a listener or requires parser routing for a client protocol.	Standalone server.
Several installations need consistent identity validation, policy conventions, and manager-mediated entry.	Managed group deployment.
You are only trying to learn SBsql syntax.	Use the simplest mode available in your build, then read First SBsql Session .
You are testing a compatibility parser.	Standalone server, because parser routing and protocol-facing behavior are part of the test.

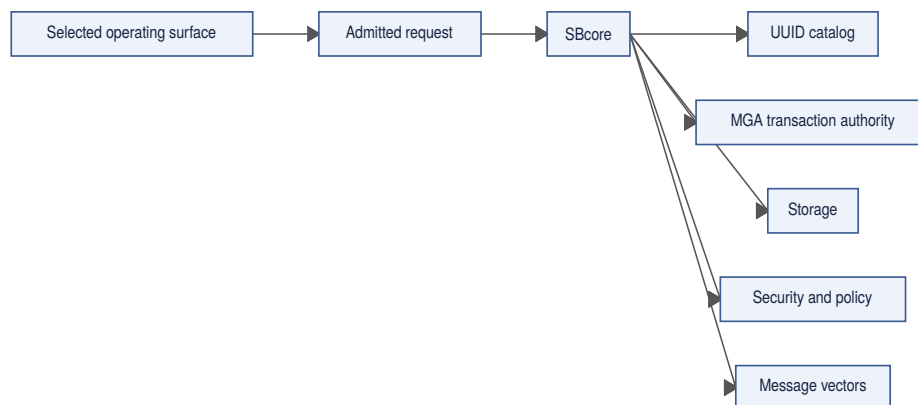
Comparison Table

The table below summarizes the key differences across modes to help you spot where your requirements fit.

Area	Embedded Engine	Single-Node IPC Server	Standalone Server	Managed Group Deployment
Process boundary	Same process as application.	Separate local server process.	Listener and server processes.	Manager-front-door convention over local services.
Client location	Application-local.	Same machine.	Network-facing client boundary where configured.	Operator-defined local installations.
Primary component	SBcore.	SBsrv.	SBgate, parser package, SBsrv, SBcore.	SBmgr plus configured local services.
Parser route	Optional, depending on application surface.	Depends on local client route.	Required for protocol or SQL client traffic.	Depends on the local service selected by SBmgr.
Lifecycle owner	Application.	Local service supervisor or operator.	Service supervisor or operator.	Operator-managed entry and local services.
Best first proof	Open database, run transaction, close cleanly.	Start server, attach local clients, run transaction, detach.	Connect through listener, route parser, run transaction, disconnect.	Authenticate through manager, open local route, run scoped session.
Main risk to understand	Application crash and engine lifecycle are tied together.	Local IPC configuration and service lifecycle.	Listener, parser, authentication, and routing configuration.	Shared identity and policy expectations across installations.

What All Modes Share

Regardless of which mode you pick, the same engine authority model applies underneath. The mode changes how a client reaches the engine; it does not move durable object identity, final transaction authority, recovery decisions, or materialized authorization out of SBcore.



What To Verify Before Choosing

Before settling on a mode for anything beyond exploration, confirm these points — discovering a gap after you have started configuring is more disruptive than checking first.

- The required binaries or libraries exist in the build output.
- Required parser packages are staged and registered.
- Resource files are present.
- Configuration files are explicit and valid.
- Authentication and authorization behavior are understood.
- Diagnostics can be collected and redacted.
- Start, stop, attach, detach, and restart tests pass for the target platform.
- The selected mode has proof coverage for the workflow you need.

Conservative Mode Selection

Use the smallest mode that satisfies the application boundary. Adding unnecessary layers increases configuration surface, failure modes, and security exposure without providing benefit.

- Do not add a listener if the application only needs embedded access.
- Do not expose network-facing routes when local IPC is enough.
- Do not use a compatibility parser for native SBsql work unless that parser route is the thing being tested.
- Do not treat a managed group deployment as shared storage or distributed query behavior.
- Do not infer availability from a diagram; check the current build output and tests.

Where To Go Next

- [Embedded Engine](#)
- [Single-Node IPC Server](#)
- [Standalone Server](#)
- [Managed Group Deployment](#)
- [First Database](#)
- [Configuration Basics](#)

Embedded Engine

Purpose

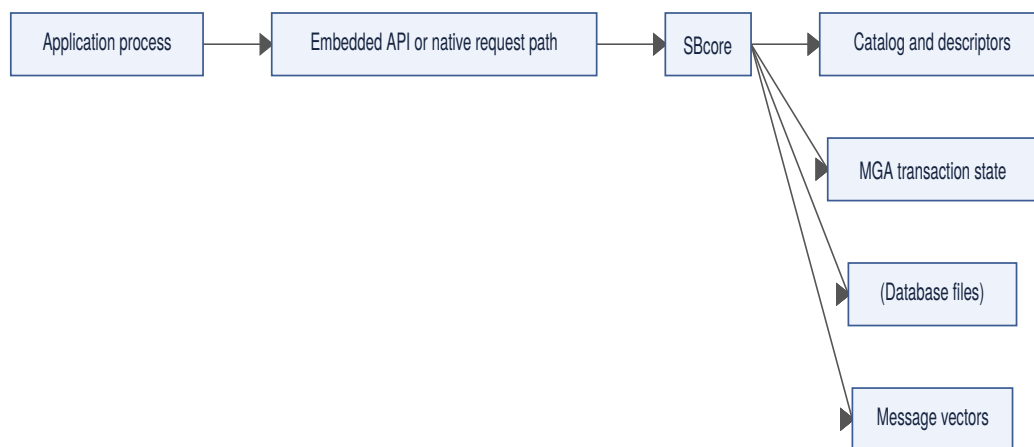
Embedded mode is the simplest deployment shape: your application links directly to SBcore (the core database engine) and the engine runs inside the same process. There is no separate server to start, no network listener to configure, and no IPC endpoint to manage. If you are building an application where a single program owns the database, or if you want the most direct path to understanding the engine itself, embedded mode is usually the right place to start.

This page explains the shape, responsibilities, and boundaries of embedded mode. It does not claim that every API, platform, or feature is available in every build.

Who this is for: developers embedding a database into an application, test authors who need direct engine access, and anyone who wants to understand SBcore before adding server-shaped complexity.

How it differs from adjacent modes: unlike Single-Node IPC Server, there is no separate server process — the application itself is the process boundary. Unlike Standalone Server, there is no network listener or parser pool.

High-Level Shape



The application and engine share one process boundary. That is the defining characteristic of embedded mode.

What Embedded Mode Is For

Embedded mode is a fit to evaluate when the application can own the full database lifecycle. More specifically, it suits situations where:

- one application owns the database lifecycle;
- the application can manage attach, detach, open, close, and shutdown behavior;
- local in-process access is enough;
- a separate server process would add complexity without providing a needed boundary;
- tests or tools need direct engine access;
- the application can collect and handle engine diagnostics.

Embedded mode is often the easiest way to understand the engine itself because fewer runtime components sit between the application and SBcore.

What The Application Owns

In embedded mode, the application carries responsibilities that a server process would otherwise centralize. Understanding these responsibilities up front prevents surprises during testing.

Responsibility	Embedded Reading
Process lifetime	If the application starts, stops, or crashes, the embedded engine session is inside that same process boundary.
Configuration	The application must pass or locate the correct configuration, resource files, and database paths.
Authentication	The application must use the configured identity model correctly for the embedded route.
Authorization	The engine still enforces authorization, but the application must not bypass the intended session model.
Transactions	The application must begin, commit, roll back, and close transaction scopes intentionally.
Diagnostics	Message vectors (structured diagnostic records) and errors are returned to the application and must be logged or presented safely.
Resource cleanup	The application must detach sessions and close databases cleanly.

Engine Authority Still Applies

Embedded mode does not mean the application owns database finality. SBcore remains responsible for all durable decisions regardless of how it was reached:

- durable catalog identity;
- descriptor validation;
- storage and filespace (the physical file organization backing a database) state;
- transaction finality and visibility;
- recovery decisions;
- index maintenance;
- materialized authorization;
- diagnostic message vectors.

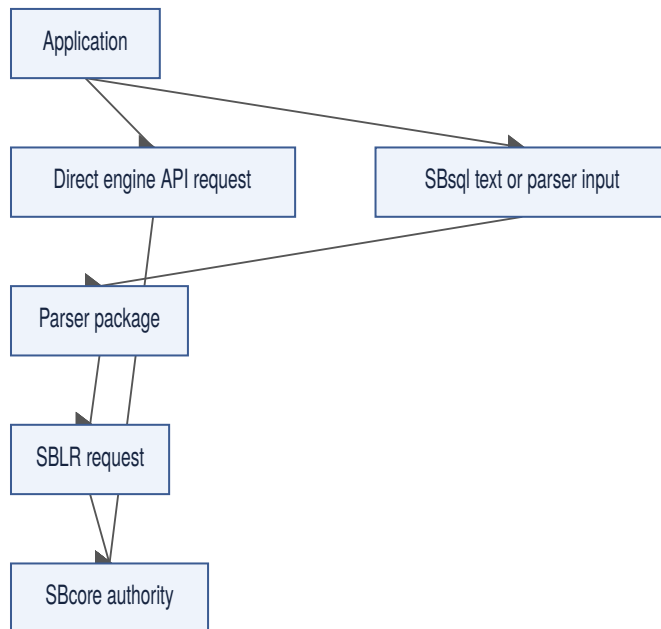
The application can request work. The engine admits, executes, or refuses that work.

Parser Use In Embedded Mode

An embedded application can use different request styles depending on what the build exposes:

- a direct embedded API surface;
- native SBsql through the SBsql parser (the native ScratchBird command language parser);
- another configured parser route if the application intentionally embeds that route.

The important boundary remains the same: SQL text or protocol-shaped input must be parsed and lowered to an internal representation before engine execution. Raw text is not durable engine authority.



First Embedded Smoke Test

A useful first embedded test validates the full lifecycle from open to close. Work through these steps in order:

1. The application can locate SBcore and required resources.
2. The application can create or open a disposable database.
3. A session can be established with the intended identity.
4. A transaction can create a schema and table.
5. Rows can be inserted and queried.
6. The transaction can commit.
7. The database can close and reopen.
8. A controlled invalid request returns a diagnostic.
9. The application detaches and exits cleanly.

Use a disposable database path until lifecycle behavior is understood. Do not run first tests against databases that contain real work.

Operational Boundaries

Area	What To Know
Crash isolation	An application crash affects the engine process because they are the same process. Recovery behavior is still engine-owned after reopen.
Local concurrency	Concurrency is limited by the embedded route and process design. Use a server mode when independent clients require a shared boundary.
Security boundary	Do not treat in-process access as a reason to skip identity, grants, policy, or protected-material rules.
Resource limits	The application must account for engine memory, file handles, timeouts, and cleanup policy.
Diagnostics	The application must handle message vectors clearly and redact protected material before logging or support collection.
Updates	Application and engine library versioning must be managed together.

What Embedded Mode Does Not Provide

It is worth being explicit about what embedded mode does not give you, so that moving to a more complex mode later is a deliberate choice rather than a surprise:

- network client access;
- a listener;
- parser pool management;
- a local multi-client service process;
- independent process supervision;
- compatibility with arbitrary external database tools;
- shared identity conventions across multiple installations;
- automatic operational packaging.

When To Choose Another Mode

Consider [Single-Node IPC Server](#) when several local clients need a shared service process — it adds a process boundary between clients and the engine without requiring a network listener.

Consider [Standalone Server](#) when clients must connect through listener and parser routing, such as when testing a compatibility parser or accepting network connections.

Consider [Managed Group Deployment](#) when several installations need consistent manager-mediated entry and shared identity or policy conventions.

For hands-on configuration and startup procedure, see the Operating Modes Runbook (ScratchBird — Operations, Security, and Autonomy, page XXX).

Where To Go Next

- [Choosing A Mode Summary](#)
- [First Database](#)
- [SBsql And SBLR](#)
- [Storage, Transactions, And Recovery](#)
- [Diagnostics And Support Bundles](#)

Single-Node IPC Server

Purpose

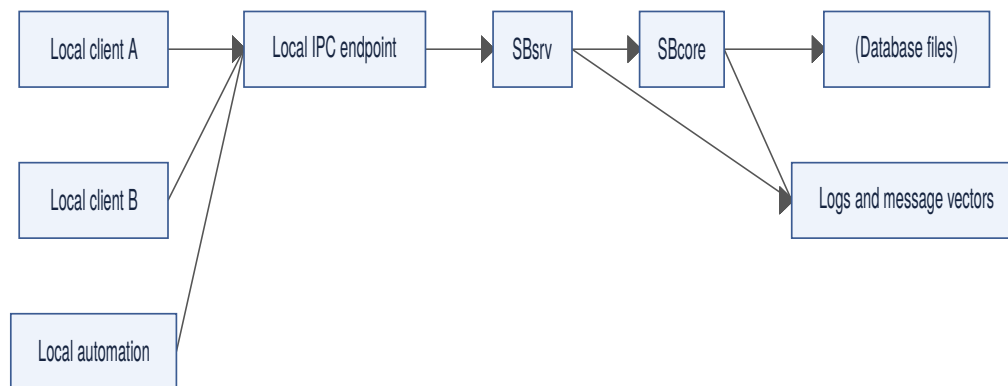
Single-node IPC server mode solves a specific problem: you have several local clients that all need to share one database, but you do not want or need a network-facing listener. SBsrv (the local server process) runs on the same machine and accepts connections from local clients through an IPC endpoint — a local communication channel that stays on-machine.

This is the mode to evaluate when moving up from embedded mode because you need a process boundary between clients and the engine, or because multiple local programs need to share database access concurrently.

Who this is for: developers with several local tools or services sharing one database, operators who want service-style lifecycle management without network exposure.

How it differs from adjacent modes: unlike Embedded Engine, the engine runs in a separate process rather than inside the client — client crashes do not terminate the server. Unlike Standalone Server, there is no network listener; clients must be on the same machine.

High-Level Shape



What This Mode Is For

Single-node IPC server mode is the right starting point when you are evaluating:

- local multi-client access to a shared engine;
- a process boundary between client applications and SBcore;
- local automation that should not embed SBcore directly;
- service-style lifecycle management on one machine;
- smoke tests that need attach, detach, and restart behavior;
- a setup where network listener behavior is not part of the requirement.

This mode is not a remote access mode. If a client needs to connect through a network-facing listener or parser route, read [Standalone Server](#).

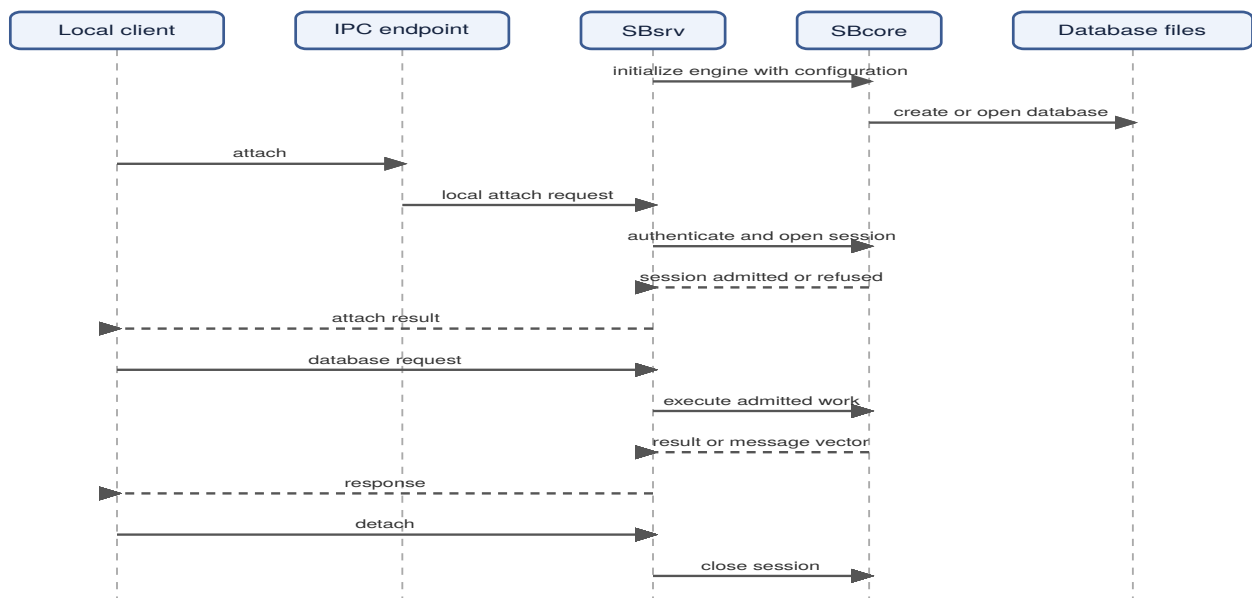
Component Responsibilities

Each component has a defined role. Understanding those roles helps you know where to look when something goes wrong.

Component	Responsibility In This Mode
Local client	Connects to the configured IPC endpoint and sends requests through the local route.
IPC endpoint	Provides the local communication boundary between clients and SBsrv.
SBsrv	Owns the local service process, opens engine sessions, routes admitted local requests, and returns results or diagnostics.
SBcore	Owns catalog identity, descriptors, transactions, storage, recovery, authorization, and engine diagnostics.
Configuration	Defines database paths, resource locations, authentication, authorization, IPC endpoints, diagnostics, and policy.

Request Flow

The following sequence shows a typical attach-work-detach cycle so you can reason about which component handles each step.



Parser Behavior

Single-node IPC mode may expose different local request surfaces depending on build and configuration. A local client might use native SBsql (ScratchBird's native command language), a direct local API, or another configured parser route.

The same rules still apply regardless of which surface is used:

- parser packages accept and lower client syntax to internal representations;
- SBcore owns durable authority;
- unsupported or denied behavior should return a controlled diagnostic;
- parser availability must be proven by the current build and tests.

First Local IPC Smoke Test

A first IPC test proves that the server lifecycle and client attach/detach cycle work end to end before you build anything more complex on top.

1. SBsrv starts with the intended configuration.
2. Required resource files are available.
3. A disposable database can be created or opened.
4. A local client can attach through the IPC endpoint.
5. Authentication and authorization produce the expected session.
6. A simple create, insert, select, and commit cycle succeeds.
7. A second local client can attach if the scenario requires it.
8. A controlled invalid request returns a message vector.
9. Clients detach cleanly.
10. SBsrv stops cleanly and can restart.
11. The database reopens with committed data visible.

Local Isolation

The server process provides isolation that embedded mode does not have — but that isolation is local process isolation, not network security or a replacement for authentication and authorization.

- Client crashes do not automatically terminate the server process.
- The engine lifecycle can be supervised separately from clients.
- Multiple clients can use a shared local service boundary.
- Diagnostics can be collected centrally by the server process.

Configuration Checklist

Before using this mode, verify these items are explicitly set rather than assumed:

- IPC endpoint location and permissions;
- database path and storage permissions;
- resource file locations;
- authentication provider or local identity configuration;
- grants, schema roots, and policy;
- diagnostic output location;
- service start and stop behavior;
- stale endpoint cleanup behavior after abnormal termination;
- parser or local API route configuration.

Diagnostics To Expect

Knowing what normal failure messages look like helps you distinguish configuration problems from software defects. Useful diagnostics in this mode include:

- configuration validation errors;
- IPC endpoint unavailable or permission denied;
- database open refused;
- authentication failure;
- authorization denied;
- parser route missing or unavailable;
- transaction state diagnostic;
- controlled shutdown or drain messages;
- stale endpoint or stale process state.

What This Mode Does Not Provide

- network listener access;
- parser pool management for network clients;
- remote client compatibility;
- shared identity conventions across installations;
- distributed query behavior;
- automatic backup or repair behavior;
- proof that every parser package is available.

When To Choose Another Mode

If you need clients to connect over a network, or if you are testing a compatibility parser that requires listener and parser routing, read [Standalone Server](#) instead.

If several installations need to share identity conventions and consistent policy admission, read [Managed Group Deployment](#).

For hands-on startup and configuration procedure, see the Operating Modes Runbook (ScratchBird — Operations, Security, and Autonomy, page XXX).

Where To Go Next

- [Choosing A Mode Summary](#)
- [Standalone Server](#)
- [First Database](#)
- [Configuration Basics](#)
- [Identity, Authentication, And Authorization](#)
- [Diagnostics And Support Bundles](#)

Standalone Server

Purpose

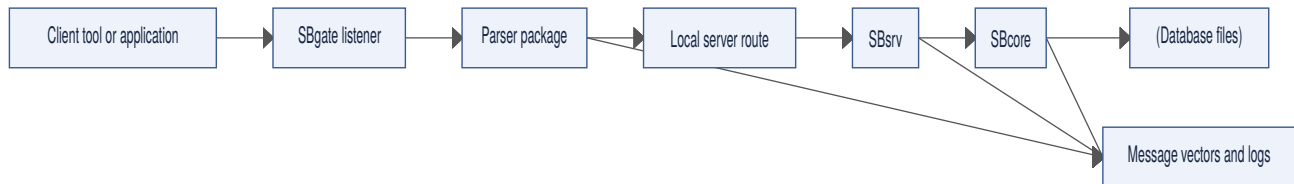
Standalone server mode is the full client/server shape where clients connect over a network through SBgate (the listener and router) and a parser package that speaks the client's protocol. This is the mode to evaluate when a client needs a network-facing entry point, protocol negotiation, a compatibility parser, or a test of end-to-end client/server behavior.

The defining boundary is that clients do not reach SBcore (the core database engine) directly. They connect to SBgate, which routes them to a parser package (a component that accepts one client language or protocol family), and that parser lowers admitted work to an internal representation before it reaches the engine.

Who this is for: operators validating network-facing deployments, developers testing compatibility parsers, and anyone whose client tool must connect through a listener rather than a local IPC channel.

How it differs from adjacent modes: unlike Single-Node IPC Server, standalone server adds a network listener (SBgate) and requires a parser package for every client connection. Unlike Managed Group Deployment, it handles one installation rather than coordinating identity and policy across several.

High-Level Shape



What This Mode Is For

Standalone server mode is the right page to read when you are evaluating:

- network-facing client access;
- listener startup and shutdown;
- parser selection and parser pool behavior;
- compatibility client or tool experiments where a parser exists;
- native SBsql over a listener route;
- protocol negotiation and refusal behavior;
- end-to-end client/server smoke tests.

Actual suitability depends on the current release, target platform, parser status, configuration, and proof results.

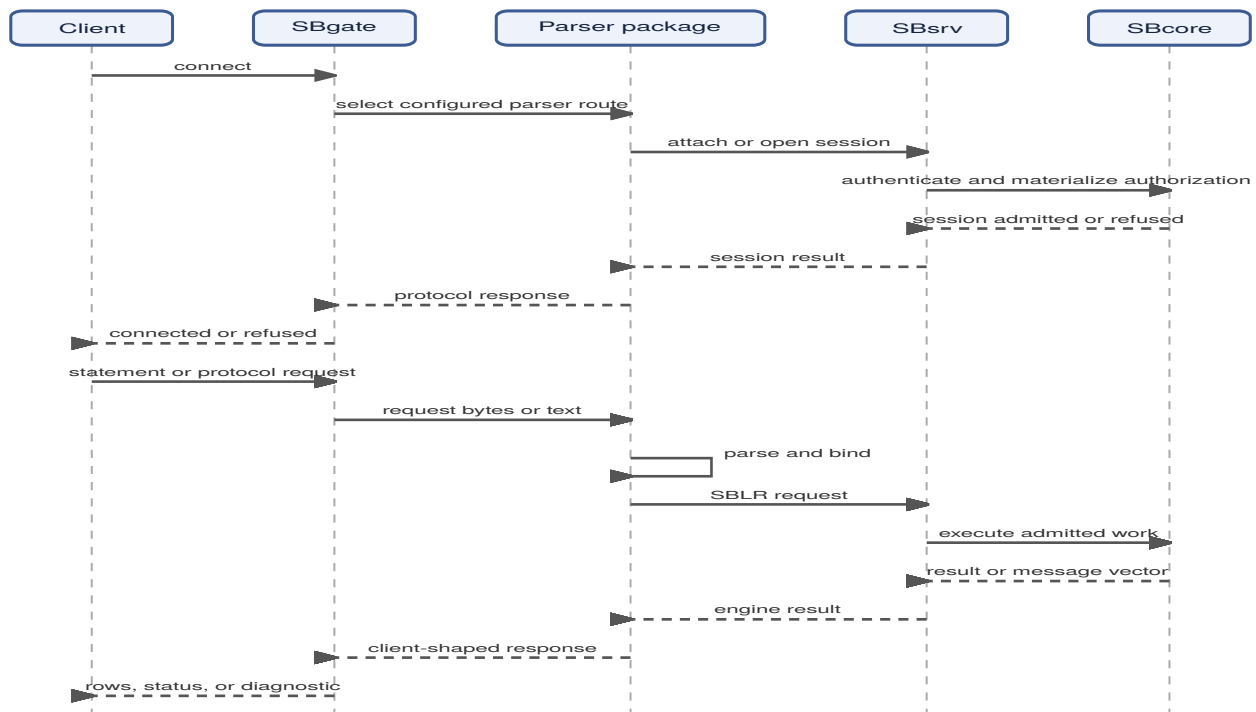
Component Responsibilities

Understanding the role of each component makes it easier to diagnose problems at the right layer.

Component	Responsibility In This Mode
Client	Connects through the configured listener route and sends language or protocol requests.
SBgate	Accepts client connections, performs listener-level routing, and hands work to the selected parser path.
Parser package	Accepts one client language or protocol family, binds visible names, lowers admitted work to SBLR (the internal request representation passed to the engine), and renders client-shaped results or diagnostics.
SBsrv	Provides the local service route to SBcore where configured.
SBcore	Owns durable catalog identity, descriptors, transactions, storage, recovery, authorization, and engine diagnostics.
Configuration	Defines listener endpoints, parser registration, identity sources, database routes, resource files, policy, and diagnostics.

Request Flow

The following sequence shows what happens from the moment a client connects to the moment it receives a result. Notice that SBcore is reached only after the connection has been authenticated and the parser has lowered the request.



Parser Routing

The listener selects a configured parser path — it does not make syntax into engine authority on its own. The parser accepts or refuses the client surface, then submits a bound request to the engine path.

Parser routing should be explicit enough that you can answer these questions for any connected session:

- which parser handled the connection;
- which database or workarea (the namespace root visible to a compatibility client) the session entered;
- which identity was authenticated;
- which schema root the session sees;
- which unsupported or denied requests are refused by the parser;
- which requests reach engine authority.

Compatibility Parser Boundaries

A compatibility parser is scoped to its own client family. Being explicit about what a parser must not do prevents accidental bypass of engine authority.

It should not:

- accept unrelated dialects silently;
- bypass engine transactions;
- write storage directly;
- grant access outside its configured workarea;
- treat physical page-copy data as logical restore input;
- perform low-level repair or verification through a compatibility route;
- claim unsupported features by returning success without doing the work.

It should:

- accept the supported client surface;
- lower supported work to SBLR;
- apply parser-specific defaults explicitly;
- return controlled diagnostics for unsupported, denied, unsafe, or unavailable behavior;
- keep catalog projections within the configured authority model.

First Standalone Server Smoke Test

With more components involved than in embedded or IPC mode, a first standalone test needs to verify each layer in turn before declaring success.

1. Required binaries, parser packages, and resource files are staged together.
2. Configuration validates before accepting clients.
3. SBsrv can open the database route.
4. SBgate starts and listens on the intended endpoint.
5. The selected parser package is available and registered.
6. A client can connect and authenticate.
7. The parser opens the expected schema root or workarea.
8. A create, insert, select, and commit cycle succeeds.
9. A controlled invalid request returns the expected diagnostic.
10. The client disconnects cleanly.
11. Listener drain or stop behavior completes.
12. The database reopens with committed data visible.

Diagnostics To Collect

Standalone server mode has more moving parts than embedded or IPC mode. When something goes wrong, capturing diagnostics at each layer points to the right fix faster.

- configuration validation result;
- listener endpoint and route selection;
- parser registration and version;
- authentication result;
- session identity and schema root;
- database open result;
- transaction state;
- message vectors for unsupported or denied requests;
- parser-to-engine request identifiers where available;
- clean shutdown, drain, and restart evidence.

Diagnostics should be redacted before sharing outside trusted support channels.

Security And Exposure

Network-facing entry points require explicit configuration — do not rely on defaults to provide security. Before allowing access beyond a local test environment, verify:

- only intended endpoints are listening;
- authentication is configured;
- authorization and schema roots are explicit;
- parser routes are limited to the needed surfaces;
- diagnostics do not expose protected material;
- server-local file access is denied unless an explicit documented policy admits a safe operation;
- unsupported management or low-level actions refuse clearly.

This guide describes the concepts to verify, not a certified deployment shape.

What This Mode Does Not Provide

- implementation of every command in every parser package;
- compatibility with every external client tool;
- shared identity conventions across separate installations;
- cross-installation query planning;
- automatic data movement;
- physical backup or repair through parser routes;
- production readiness without release-specific proof.

When To Choose Another Mode

If several installations need consistent manager-mediated entry, shared identity, or coordinated policy admission, read [Managed Group Deployment](#).

For hands-on listener startup, parser registration, and configuration procedure, see the Operating Modes Runbook (ScratchBird — Operations, Security, and Autonomy, page XXX).

Where To Go Next

- [Choosing A Mode Summary](#)
- [Single-Node IPC Server](#)
- [Managed Group Deployment](#)
- [Reference-System Compatibility](#)
- [Engine Parser Boundary](#)
- [Configuration Basics](#)
- [Diagnostics And Support Bundles](#)

Managed Group Deployment

Purpose

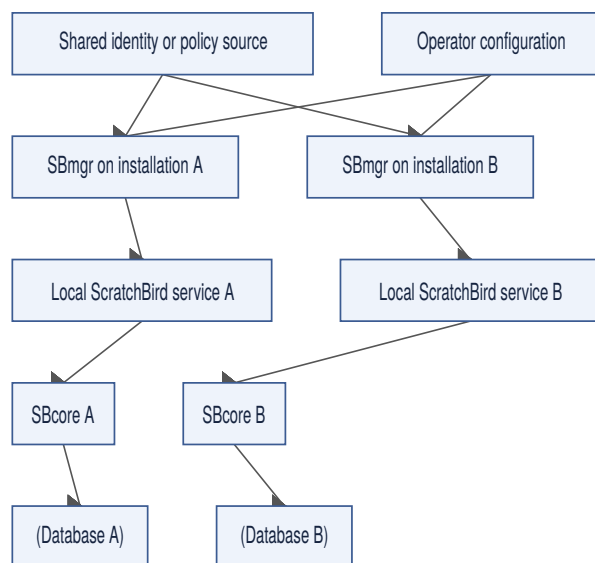
Managed group deployment is for organizations running more than one ScratchBird installation and needing those installations to share consistent identity conventions, policy rules, and diagnostic practices. SBmgr (the manager front-door component) provides a controlled entry point that validates identity and routes sessions to the appropriate local service before a client reaches any database.

This page is important to read carefully because managed group deployment is frequently misunderstood: it is an **operational consistency pattern**, not a distributed database. Each installation keeps its own database files, its own transaction authority, and its own recovery state. SBmgr coordinates admission — it does not merge databases or enable cross-installation queries.

Who this is for: operators managing multiple ScratchBird installations who need common identity validation, consistent policy admission, and shared diagnostic conventions.

How it differs from adjacent modes: unlike Standalone Server, managed group adds SBmgr as a front-door layer above one or more local services, and it assumes more than one installation is involved. Unlike the other modes, this shape is primarily about operational consistency, not a new technical capability of the engine.

High-Level Shape



What Is Managed

The value of a managed group comes from making these operational concerns consistent across installations, rather than configuring each one independently.

Area	Meaning
Entry convention	SBmgr provides a consistent front-door pattern before traffic reaches local services.
Identity validation	Installations can use a shared identity source or shared identity policy where configured.
Admission policy	Operators can apply consistent expectations for who may connect and which routes may be used.
Parser route policy	Parser availability can be managed deliberately instead of being assumed.
Diagnostics	Logs, message vectors (structured diagnostic records), support-bundle expectations, and redaction policy can be made consistent.
Configuration style	Installation configuration can follow the same layout and validation rules.

What Remains Local

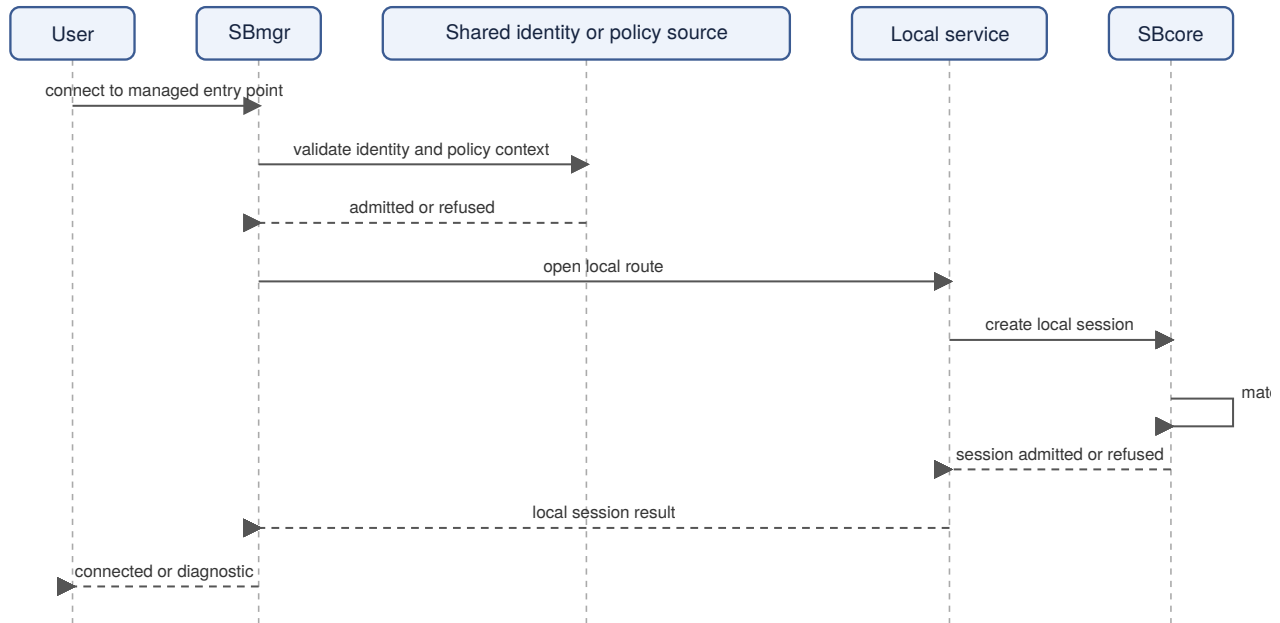
Even inside a managed group, each database retains full authority over its own durable state. Operational consistency at the SBmgr layer does not change what each engine owns.

Area	Local Authority
Database files	Each database has its own storage and filespace (the physical file organization backing a database) state.
Transactions	Each database keeps its own MGA transaction authority (the multi-generational transaction model).
Recovery	Each database reopens, recovers, or refuses according to its own durable state.
Catalog identity	Object UUIDs and descriptors belong to the database that owns them.
Grants and schema roots	Authorization is evaluated for the session and database context.
Parser workarea	A compatibility session sees the workarea (the namespace root visible to a compatibility client) assigned by its local route.

Managed group deployment is therefore an operating pattern, not a promise that all databases become one database.

User Connection Flow

The sequence below shows how a session flows through SBmgr before reaching the local engine. The key insight is that the manager validates identity first; the local engine materializes authorization second.



The user receives a session inside a specific local database context. Grants, parser route, schema root, workarea, and policy determine what the user can see and do.

When To Evaluate This Mode

Managed group deployment is worth evaluating when:

- several installations should use the same identity conventions;
- users or agents should connect through a manager-controlled front door;
- operators need consistent diagnostics and support-bundle expectations;
- parser routes should be centrally described or consistently admitted;
- policy defaults need to be coordinated;
- installations need a common operational runbook.

If there is only one local application process, read [Embedded Engine](#). If there are several local clients on one machine, read [Single-Node IPC Server](#). If the requirement is listener and parser routing for clients at one installation, read [Standalone Server](#).

Configuration Checklist

Before using a managed group shape, explicitly define each of these items. Gaps discovered after connecting real clients are more disruptive than verifying them beforehand.

- installation identity;
- local database routes;
- SBmgr endpoint and local service routes;
- identity source and authentication behavior;
- authorization mapping for users, roles, groups, or agents;
- parser packages admitted at each installation;
- default schema roots or workareas;
- diagnostics, support-bundle, and redaction policy;
- start, stop, drain, and restart behavior;
- refusal behavior when the identity source or local service is unavailable.

Refusal Cases

Controlled refusal is a core part of the managed group operating model — the manager should never silently pass a request it cannot validate.

SBmgr should refuse clearly when:

- identity validation fails;
- the identity source is unavailable and policy does not allow fallback;
- the requested installation or database route is unknown;
- the local service is unavailable;
- the parser route is not admitted;
- the user is not authorized for the requested schema root or workarea;
- configuration validation fails;
- diagnostics cannot be safely produced.

Data Movement Is Separate

Managed group deployment may coexist with backup, restore, migration, import, export, CDC, or replication features where those features are implemented and admitted. Those are separate data movement surfaces.

Do not infer data movement, shared storage, or automatic query routing from the presence of SBmgr. The manager front-door role is about connection admission and local route control.

Diagnostics To Collect

When troubleshooting managed group issues, collect evidence at both the SBmgr layer and the local service layer to understand which step refused or failed.

- SBmgr configuration validation;
- identity source selection;
- authentication result;
- local route selected;
- parser selected;
- database opened;
- session identity and schema root;
- authorization result;
- refusal message vector when connection is denied;
- local service health;
- clean drain and shutdown behavior.

Diagnostics must follow the configured redaction policy.

What This Mode Does Not Provide

- one shared database across installations;
- distributed query planning;
- automatic replication;
- automatic failover;
- shared storage;
- cross-installation transaction finality;
- compatibility parser completeness;
- production suitability without release-specific proof.

Those behaviors require separately documented and proven surfaces.

For hands-on configuration and startup procedure for managed group deployments, see the Operating Modes Runbook (ScratchBird — Operations, Security, and Autonomy, page XXX).

Where To Go Next

- [Choosing A Mode Summary](#)
- [Standalone Server](#)
- [Choosing A Deployment Mode](#)
- [Configuration Basics](#)
- [Identity, Authentication, And Authorization](#)
- [Diagnostics And Support Bundles](#)

First Database

Purpose

Creating your first ScratchBird database is less about running a single command and more about verifying that several things work together: the right build output, the right operating mode, the right resource files, and a working transaction cycle. This page gives you a safe path through that verification.

The goal is modest but meaningful: prove that the selected build can create or open a database, connect through the intended mode, run a small transaction, return diagnostics, and shut down cleanly. Everything that comes after — schema design, compatibility parsers, administration — builds on this foundation.

This page is an orientation guide rather than a fixed command transcript, because command names, binary locations, configuration defaults, and release packaging can vary by build target.

Before You Start

Before creating a database, confirm these items. Each one affects whether your first test works, and discovering a gap mid-test is more disruptive than checking first.

Item	Why It Matters
Build output exists	The tools, engine library, parser packages, and resource files must come from the same build.
Target platform is known	A Linux, Windows, or BSD output tree may have different file names and service behavior.
Operating mode is chosen	Embedded, local IPC, standalone server, and managed group deployments have different startup paths.
Resource files are staged	Character sets, collations, time zones, policy defaults, and configuration files are part of a usable deployment.
Authentication path is understood	Even a first test should use a known identity and expected authorization behavior.
Storage location is deliberate	Put test databases somewhere disposable until you understand cleanup and backup behavior.

Do not create first-test databases in directories that also hold release binaries or source files.

Choose A Mode

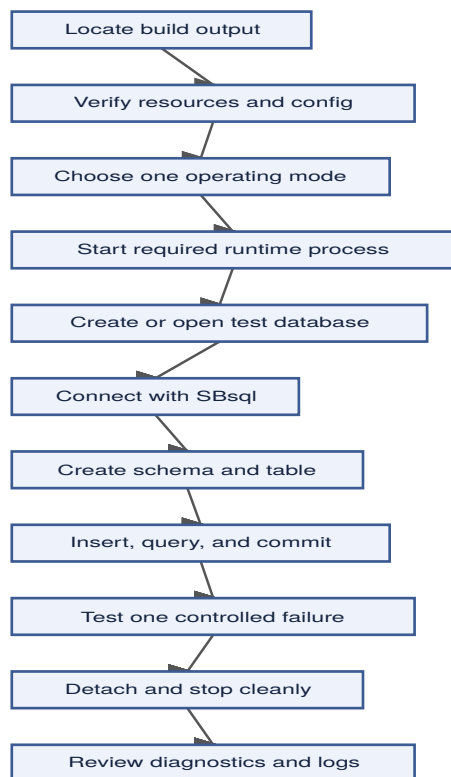
ScratchBird can be approached through more than one mode. For a first test, pick one path and follow it all the way through rather than mixing modes.

Mode	First-Test Shape	Use When
Embedded engine	Application or test tool opens SBcore (the core database engine) directly.	You are validating library embedding or an application-local database.
Single-node IPC server	Local client talks to SBsrv (the local server process).	Several local clients need one server process without a network listener.
Standalone server	Client enters through SBgate (the listener and router) and parser routing.	You are validating network-facing listener and parser behavior.
Managed group deployment	Client enters through SBmgr (the manager front-door), then listener and parser routing.	You need managed entry points and shared identity or policy integration.

For a first user workflow, the single-node IPC or standalone server modes are often easiest to reason about because they show a clear client/server boundary. Embedded tests are better when the application itself is the product boundary.

First Database Flow

The flow below shows a safe sequence. Each step builds on the previous one, so work through them in order rather than jumping ahead to the SQL.



Pick A Test Database Name

Use a name that clearly identifies the database as disposable, for example:

- `scratchbird_getting_started`
- `first_database_test`
- `sbsql_smoke_test`

Keep the database in a temporary test location until you know how the selected operating mode handles database paths, configuration, identity, and cleanup.

Create Or Open The Database

The exact command depends on the selected binary and mode. Whatever that command is, the operation should establish:

- database file or database resource location;
- initial catalog;
- initial filesystem (the physical file organization backing the database);
- initial character set and collation behavior;
- security and policy baseline;
- parser route for the first session;
- diagnostics location.

In a first test, avoid advanced options. Do not test backup, restore, repair, import, replication, or compatibility parser behavior until a basic create/open/connect/query cycle works.

First SBsql Workload

Once connected with SBsql (ScratchBird's native command language), run a small transaction that exercises names, types, inserts, selects, and commit. The example below uses native SBsql syntax with standard types.

```
create schema app;

create table app.notes (
    note_id bigint not null,
    note_text text not null,
    created_at timestamptz not null,
    constraint pk_notes primary key (note_id)
);

insert into app.notes (note_id, note_text, created_at)
values
    (1, 'first note', current_timestamp),
    (2, 'second note', current_timestamp);

select note_id, note_text, created_at
from app.notes
order by note_id;

commit;
```

This example exercises table creation, column descriptors, scalar datatypes, a named constraint, schema-qualified

name resolution, and a transaction commit. If a release changes spelling, available types, or built-in function names, follow the Language Reference for that release.

Verify A Controlled Refusal

A first database test should include one intentional failure. The goal is not to break the system; it is to confirm that invalid work returns a controlled diagnostic rather than a confusing crash or silent success.

```
select *  
from app.table_that_does_not_exist;
```

Expected behavior:

- the session remains alive;
- the transaction state is understandable;
- the diagnostic says what failed;
- the diagnostic does not expose protected material;
- the client can continue or detach cleanly according to transaction state.

Confirm The Database Reopens

After the first transaction, verify that the data was actually written to durable storage, not just held in memory. This rules out a silent failure in the commit or reopen path.

1. Detach the client.
2. Stop the server process if the selected mode uses one.
3. Start the same mode again.
4. Reopen the same test database.
5. Query the rows inserted earlier.

```
select count(*) as note_count  
from app.notes;
```

```
select note_id, note_text  
from app.notes  
order by note_id;
```

If the rows are not present, check whether the transaction was committed, whether the same database was reopened, and whether the current schema is the one you expect.

What Success Looks Like

A successful first database run proves:

- the selected output tree is internally consistent;
- required resource files can be found;
- the chosen operating mode starts;
- the database can be created or opened;
- SBsql can connect;
- simple DDL and DML execute;
- commit makes data visible after reconnect;
- an invalid query returns a controlled diagnostic;
- the runtime detaches and shuts down without leaving an obvious stuck state.

It does not prove that every parser, datatype, administrative command, or compatibility surface is complete.

Common Early Problems

Symptom	Likely Area To Inspect
Binary starts but cannot find resources	Output staging, configuration paths, character set or collation resources.
Client cannot connect	Operating mode, socket or port configuration, listener route, authentication settings.
Parser not found	Parser package output, parser registration, configuration.
Create database fails	Storage directory permissions, existing file state, configuration, initial security policy.
Query fails with syntax error	SBsql syntax version or command spelling.
Data disappears after restart	Transaction not committed, wrong database path, temporary test database, failed reopen.
Shutdown leaves state behind	Runtime lifecycle handling, stale process, stale socket, or configuration issue.

Cleanup

When the first test is complete:

- detach all clients;
- stop server or manager processes started for the test;
- keep logs if a diagnostic needs review;
- delete only disposable test databases that you created for this purpose;
- do not delete shared resource files from the output tree.

Where To Go Next

With the database working, the natural next step is to explore the SQL language more fully. [First SBsql Session](#) walks through a guided session covering schema context, transaction control, and how to read diagnostics.

- [First SBsql Session](#)
- [Schemas, Objects, And Names](#)
- [Choosing A Mode Summary](#)
- [Configuration Basics](#)
- Language Reference (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)

First SBsql Session

Purpose

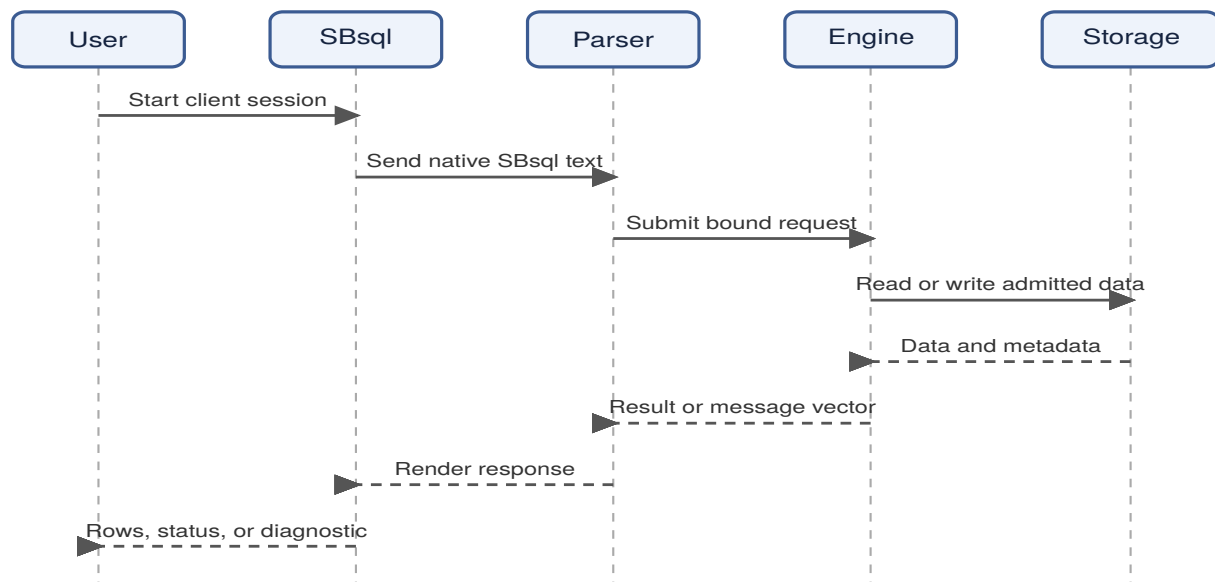
Once your database is running (see [First Database](#)), the next step is learning how to work in a session: understanding where you are, running transactions deliberately, and reading what the engine tells you when something goes wrong.

SBsql is the native ScratchBird command language. A first SBsql session should prove that you can connect to the intended database, understand your schema context, run a small transaction, inspect results, and detach cleanly. This page walks through that arc step by step.

It does not replace the full Language Reference, and it should not be read as a complete list of supported statements.

What A Session Is

A session is the authenticated conversation between a client and the database through a selected operating mode and parser route. Understanding what a session tracks helps you reason about why commands succeed or fail.



Within one session, the engine tracks these things on your behalf:

- **identity:** who the engine thinks you are;
- **current schema:** where unqualified names resolve;
- **transaction state:** whether work is pending, committed, or rolled back;
- **parser route:** whether the request is going through native SBsql;
- **diagnostics:** how success and failure are reported.

Session Checklist

Before running commands, know the answers to these questions — they explain most early session problems:

- which database you are connecting to;
- which operating mode is running;
- which identity is being used;
- whether autocommit is enabled by default for the selected tool;
- where diagnostics and logs can be reviewed;
- whether the database is disposable or persistent.

Start With Context

Begin every new session by inspecting where you are. Exact output formatting can vary by build.

```
show schema path;  
show search path;  
select current_user;  
show transaction;
```

These commands confirm that you are in the database and schema context you intended. If a command is not available in the current build, use the equivalent context-inspection command documented for that release. Starting without checking context is the most common cause of "objects appear missing" confusion.

Create A Working Schema

Rather than placing test objects at the database root, create a schema to contain them. This mirrors real application practice and makes cleanup straightforward.

```
create schema app;  
  
show schema path;
```

When the session's current schema is `app`, unqualified names such as `notes` can resolve relative to `app` when visible and unambiguous. The command for changing the current schema can vary by release, so the examples below use qualified names that do not depend on session schema state — that makes the examples more portable and clearer.

Create A Table

The following table creation exercises several basic behaviors at once, which makes it a useful first test.

```
create table app.notes (  
    note_id bigint not null,  
    note_text text not null,  
    created_at timestamptz not null,  
    constraint pk_notes primary key (note_id)  
);
```

This example demonstrates:

- table creation and column descriptor registration;
- scalar datatypes (`bigint`, `text`, `timestamptz`);
- a named primary key constraint;
- schema-qualified name resolution;
- catalog transaction behavior (the table is not visible until committed).

Insert Rows

Insert more than one row so that ordering and row counts are easy to inspect in later steps.

```
insert into app.notes (note_id, note_text, created_at)  
values  
    (1, 'created from the first SSql session', current_timestamp),  
    (2, 'second row in the same statement', current_timestamp),  
    (3, 'third row for ordering checks', current_timestamp);
```

Multi-row `values` input is useful for smoke tests because it proves that the parser and executor are not limited to one row per insert statement.

Query Rows

Query the data using an explicit column projection and stable ordering. This example demonstrates a basic `select` with explicit output columns and an `order by` clause.

```
select note_id, note_text, created_at  
from app.notes  
order by note_id;
```

Avoid `select *` in documentation examples unless the point is to inspect all columns. Explicit projection makes examples clearer and avoids hiding column-order assumptions.

Commit Or Roll Back Intentionally

End the transaction deliberately rather than relying on implicit behavior.

```
commit;
```

If you are experimenting and want to discard the work instead:

```
rollback;
```

For a first persistence test, commit the transaction, detach, reconnect, and query the table again. If the rows are not

present after reconnect, check whether the commit ran and whether you reopened the same database.

Test A Controlled Error

Run one statement that should fail. This teaches you what the engine's diagnostic output looks like before you encounter a real error under pressure.

```
select note_id
from notes_that_do_not_exist;
```

Expected behavior:

- the client receives a message vector (a structured diagnostic record) explaining the failure;
- the session remains controlled;
- protected details are not leaked;
- the next allowed command behaves according to transaction state.

Successful systems explain failures clearly. A controlled refusal is the right behavior here.

Reconnect And Verify Persistence

After commit, verify that the data was actually written to durable storage.

1. Detach the SBsql client.
2. Stop and restart the selected runtime if appropriate.
3. Connect again.
4. Qualify names with the schema.
5. Query the committed rows.

```
select count(*) as note_count
from app.notes;
```

```
select note_id, note_text
from app.notes
order by note_id;
```

If the rows are not present, check whether the transaction was committed, whether the same database was reopened, and whether the current schema is the one you expect.

Clean Up Test Objects

For a disposable first session, remove the test objects after verifying the workflow.

```
drop table app.notes;
drop schema app;
commit;
```

Only drop objects that you created for the test. Do not run cleanup examples against a database that contains real work.

Reading Result Sets

A result set has column names, column order, datatypes, nullability behavior, and row order. For early tests, keep these habits:

- name the columns you want explicitly;
- include `order by` when row order matters;
- test null values intentionally;
- test type conversion intentionally;
- keep result sets small enough to inspect by eye.

Reading Message Vectors

ScratchBird diagnostics communicate structured refusal or error information through message vectors. A user-facing rendering may include text, code, class, source component, object name, or policy information depending on the command and build.

For a first session, it is useful to categorize failures you see. Different categories require different fixes:

- syntax errors: the statement text needs correction;
- missing object errors: check the current schema and whether the object was committed;
- authorization denials: check grants and session identity;
- unsupported feature refusals: the feature may not be available in this build;
- configuration problems: check resource files, parser registration, or route configuration;
- runtime availability problems: check whether the required process is running.

Common Session Mistakes

Mistake	What Happens
Connecting to the wrong database path	Objects appear missing or changes appear to disappear.
Forgetting to commit	Reconnect tests may not show expected rows.
Using the wrong current schema	Unqualified names resolve somewhere else or fail.
Relying on implicit row order	Result rows may not appear in insertion order.
Mixing parser expectations	Native SBsql examples should be run through the SBsql parser route.
Ignoring diagnostics	A refused command may leave the session in a state that requires commit, rollback, or detach.

Where To Go Next

With a working session under your belt, [Schemas, Objects, And Names](#) explains how ScratchBird stores durable object identity separately from the names you type — which matters as soon as you start renaming things, working with compatibility parsers, or writing migration scripts.

- [First Database](#)
- [Schemas, Objects, And Names](#)
- Schema Tree And Name Resolution (SBsql Language Reference — Syntax, page XXX)
- Table Statements (SBsql Language Reference — Syntax, page XXX)
- Insert (SBsql Language Reference — Syntax, page XXX)
- Select (SBsql Language Reference — Syntax, page XXX)
- Transaction Control (SBsql Language Reference — Syntax, page XXX)

Schemas, Objects, And Names

Purpose

In most databases, a name is how you find a thing. In ScratchBird, names are user-facing labels, but the engine stores durable identity separately — objects are tracked by UUID-backed catalog entries, not just by the text you type. That distinction matters the moment you rename an object, work through a compatibility parser, or write a migration script.

This page explains how to think about schemas, objects, qualified names, current schema, home schema, compatibility workareas, and recursive schema trees. For complete syntax, use the Language Reference. This page is the end-user orientation.

This page continues the tutorial arc from [First SBsql Session](#), where you created a schema and a table. Now you will understand why those names worked the way they did.

Names Are User-Facing Labels

A name is what a user types or sees in a tool:

```
select note_id, note_text
from app.notes;
```

In that example, `app` is a schema name, `notes` is an object name inside that schema, and `app.notes` is a qualified name. The engine does not rely only on text names for durable identity — objects are represented by catalog identity, descriptors, parent schema identity, grants, and transaction visibility.

Durable Identity

ScratchBird uses UUID-backed catalog identity for durable database objects. Understanding this distinction prevents confusion when names change but references should not.

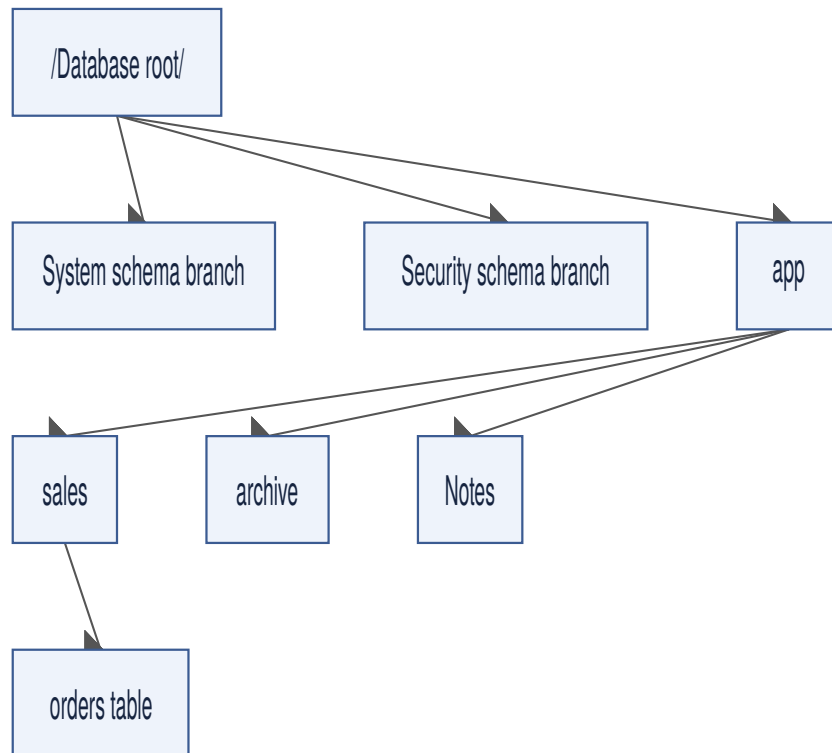
A visible name can change:

- an object can be renamed;
- an object can be displayed through a parser-specific catalog projection;
- a session can resolve an unqualified name through a current schema;
- a compatibility workarea (the namespace root presented to a compatibility client) can make one schema branch look like the client-visible root;
- a dependency can continue to point at the same object after a rename.

Durable identity lets the engine answer the deeper question: "Which object is this?" Names answer the user-facing question: "How does this session spell it?"

Recursive Schema Tree

ScratchBird schemas can contain child schemas and database objects. This creates a schema tree rather than one flat namespace. A path through the tree gives context — `app.sales.orders` and `app.archive.orders` are different objects even though both end with `orders`.



Schema Context

A session can have several schema-related concepts at once. The relationship between them is what determines how an unqualified name like `notes` resolves to a specific object.

Concept	Meaning
Database root	The top of the durable database tree. Not every session can see it directly.
Parser-visible root	The root of the namespace presented to the selected parser route.
Home schema	The schema associated with the connected identity or configured workarea.
Current schema	The default schema used for unqualified names in the current session.
Search path	An ordered list of schema locations used by commands that allow path-based lookup.
Object parent schema	The schema that owns an object's name within the tree.

The exact variables and commands used to inspect these values are described in the Language Reference for the current release.

Current Schema

The current schema is the default location for unqualified object names. Knowing your current schema is essential before running any statement that relies on unqualified names.

```
show schema path;
```

```
select note_id, note_text
from notes;
```

If the current schema is `app` and `notes` is visible in `app`, the unqualified name can resolve to `app.notes`. The command used to change the current schema is release-specific; consult the Language Reference for the current build.

Unqualified names are convenient in interactive sessions, but they can also hide mistakes. When writing administrative scripts, migrations, or examples intended for other users, prefer qualified names where clarity matters.

Qualified Names

A qualified name includes path information, which makes it clear where the object is expected to live regardless of session schema state.

```
select note_id, note_text
from app.notes;
```

In a recursive schema tree, deeper paths may be used where supported:

```
select order_id, order_status
from app.sales.orders;
```

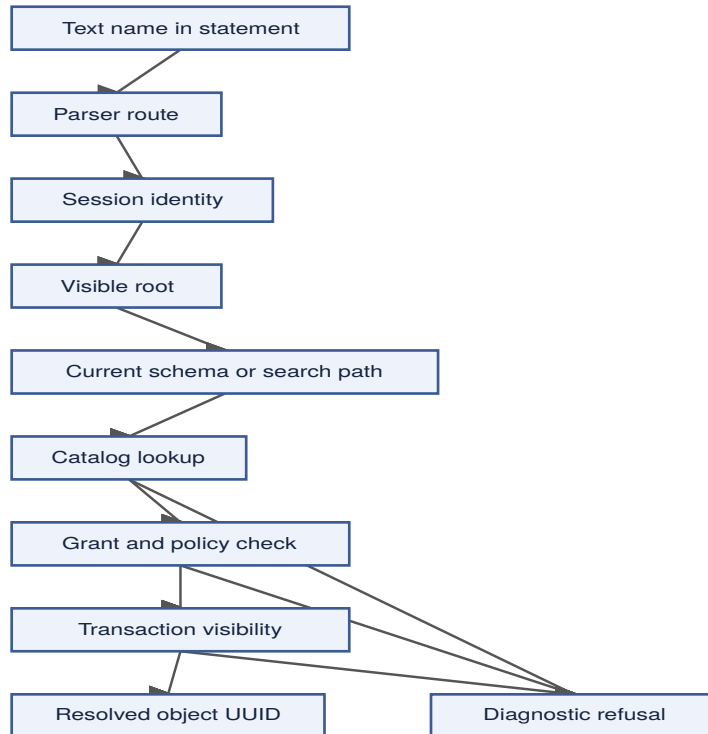
The binder still has to resolve the visible path to a real object identity. A qualified name is not a bypass around security, sandboxing, or transaction visibility.

Name Resolution

Name resolution is the process of turning user-visible text into engine object identity. Understanding the steps helps you predict where a name lookup will fail and why.

At a high level, name resolution considers:

1. the parser route;
2. the authenticated identity;
3. the parser-visible schema root;
4. the current schema;
5. the search path where applicable;
6. explicit qualification in the statement;
7. grants, policy, and object visibility;
8. transaction visibility for recently created, changed, or dropped objects.



If any step fails, the result should be a diagnostic rather than an accidental lookup outside the intended scope.

Common Object Types

End users will commonly encounter these object categories. Not every parser route exposes every object category in the same way; native SBsql is the reference language for ScratchBird object administration.

Object Type	What It Represents
Schema	A branch in the namespace tree.
Table	Stored rows and column descriptors.
Temporary table	A table whose data lifetime is scoped by session or transaction according to the table definition.
View	A named query projection.
Materialized view	A stored projection that must have refresh and dependency behavior defined by the implementation.
Index	A search structure over table or expression data.
Constraint	A rule attached to a table, column, or domain.
Domain	A reusable constrained type definition.
Type descriptor	A named type or type shape known to the engine.
Sequence	A database object that generates ordered values according to its definition.
Procedure	A stored routine that can perform controlled work.
Function	A stored or built-in routine that returns a value or result.
Package	A named grouping of routine definitions where supported.
Trigger	Routine behavior tied to table, database, transaction, or event-style actions where implemented.
Policy	A named rule for row access, masking, external access, or operational admission.
Role and privilege	Security objects that describe who can do what.
Comment	User-facing descriptive metadata attached to an object.

Object Lifecycle

Most schema objects follow a lifecycle from creation through use to eventual removal. Some object types also support `recreate`, `create` or `alter`, `refresh`, `attach`, `detach`, `validate`, or other object-specific actions. The object page in the Language Reference is the authoritative place for supported lifecycle syntax.



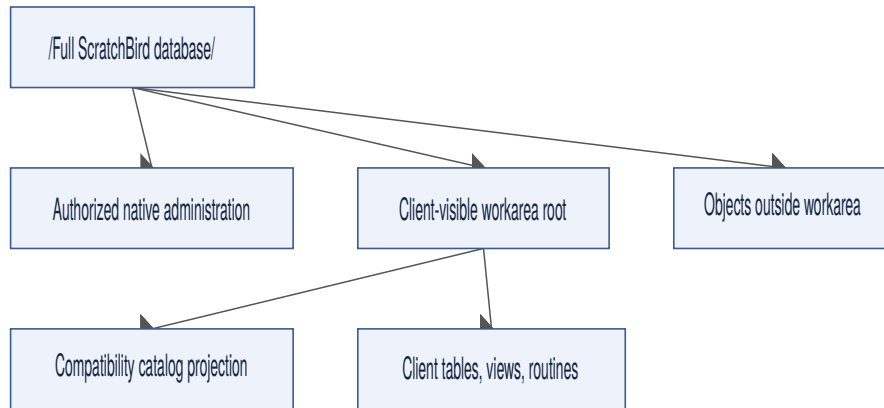
Comments And Descriptions

Comments are descriptive metadata. They help tools and users understand objects, but they do not grant access and do not change object identity.

Use comments for purpose, ownership notes, migration context, operational warnings, column meaning, and expected units for values. Avoid putting secrets, passwords, tokens, or protected operational details in comments.

Compatibility Workareas

A compatibility parser session normally sees a workarea as its root. That means a client can experience the connected workarea as "the database" even though ScratchBird may store it as a branch inside a larger recursive schema tree.



The client cannot simply name `Outside` and access it. If a catalog projection shows selected metadata, that projection is an object with its own authority — it is not proof that the connected user can directly query the underlying object.

Case, Quoting, And Identifiers

Identifier behavior can vary by parser route. Native SBsql is intended to remain context-sensitive with as few reserved words as practical, but scripts should still be written carefully:

- use clear object names;
- avoid names that differ only by case;
- avoid names that look like built-in functions or command keywords;
- use qualified names in administrative scripts;
- quote identifiers only when the language reference says quoting is required or intended;
- keep migration scripts consistent about naming style.

Object Defaults Can Be Parser-Specific

Two parser routes can expose similar object concepts while applying different defaults. Examples include index null behavior, identifier folding, default schema selection, default datatype precision, string literal handling, generated name formatting, catalog projection rows, and diagnostic wording. The engine still owns the durable object; the parser is responsible for mapping its client-facing defaults into an explicit engine request.

Practical Naming Guidance

For new ScratchBird-native work:

- create an application schema instead of placing user objects at the root;
- qualify object names in migration and administration scripts;
- choose stable names that describe the object's purpose;
- use comments for human-facing meaning;
- avoid secrets in object names and comments;
- avoid relying on implicit search paths in automated scripts;
- test rename behavior before using it in migration tooling;
- verify grants after moving or renaming important objects.

Example Schema Layout

A small application might start with this shape:

```
database_root
|-- app
|   |-- notes
|   |-- note_tags
|   |-- active_notes
|   `-- routines
|-- audit
|   `-- note_events
`-- policy
    `-- application_policies
```

That tree separates application data, audit records, and policy-related objects. The exact layout for a real application depends on authorization, operational needs, and migration strategy.

Where To Go Next

This completes the core tutorial arc: you have created a database, worked through a session, and now understand how names and objects relate to each other. The next topic in depth is how compatibility parsers work and what it means for a ScratchBird database to expose a reference-system surface — read [Reference-System Compatibility](#) for that.

- [First SBsql Session](#)
- [Reference-System Compatibility](#)
- [Recursive Schema Tree](#)
- Schema Tree And Name Resolution (SBsql Language Reference — Syntax, page XXX)
- Schema Statements (SBsql Language Reference — Syntax, page XXX)
- Table Statements (SBsql Language Reference — Syntax, page XXX)
- Script Tokens And Identifiers (SBsql Language Reference — Syntax, page XXX)

Reference-System Compatibility

Purpose

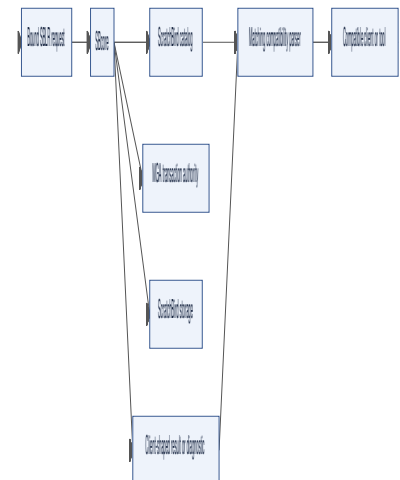
ScratchBird can serve clients that speak the language and protocol of another database system, through a compatibility parser package. This matters when you have existing applications, tools, or workflows built for another system and you want them to connect to a ScratchBird database without being rewritten.

A compatibility parser is a standalone adapter for one client family. It lets a client speak that parser's language and protocol shape while ScratchBird keeps storage, transactions, identity, security, and recovery inside the ScratchBird engine. The key word is "scoped": the presence of a parser package means there is a route for that client family; it does not mean every command, tool behavior, catalog row, wire-protocol edge case, or administrative feature from that source ecosystem is complete in the current build.

This page should be read after [Schemas, Objects, And Names](#), because compatibility parsers surface a workarea as the client-visible root — which only makes sense once you understand the recursive schema tree.

The Compatibility Model

A compatibility parser sits between a client and SBcore (the core database engine). The parser is responsible for accepting the client surface; the engine remains responsible for durable authority.



What The Parser Owns

A compatibility parser can own client-facing behavior. Understanding this boundary helps you know what to test per-parser and what remains consistent across all parsers.

Parser-owned behavior includes:

- protocol negotiation for its client family;
- syntax accepted by that parser;
- parser-specific object defaults;
- parser-specific type spelling and literal handling;
- catalog projections that make ScratchBird metadata visible in the expected shape;
- diagnostic rendering for that client family;
- logical backup, restore, replication, or import/export streams where those surfaces are implemented and admitted by policy;
- mapping accepted work into SBLR (the internal representation passed to the engine).

The parser does not own final transaction authority, storage recovery, durable catalog identity, authorization finality, object UUID assignment, cleanup and garbage collection decisions, page format interpretation outside the engine, or low-level repair or verification authority.

Parser Isolation

Each compatibility parser is isolated from the others. This separation is intentional — it keeps compatibility behavior explicit and prevents a client from accidentally entering a different dialect.

Rule	Meaning
One parser, one client family	A parser should accept only the syntax and protocol shape it is built for.
No cross-dialect fallback	A parser should not silently accept unrelated language forms because another parser supports them.
No implicit dependency	Installing one compatibility parser must not imply that any other parser is installed.
Parser-local defaults	Object defaults, name folding, null handling, index defaults, and diagnostics are parser-specific where implemented.
Engine authority remains shared	Once accepted and lowered, engine execution still uses ScratchBird catalog, security, transaction, and storage rules.

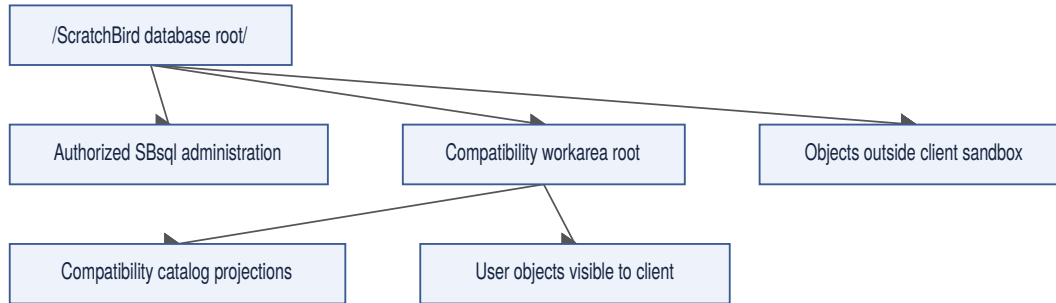
Compatibility Areas

Compatibility is not a single thing — a parser can be strong in one area and incomplete in another. Read compatibility claims by specific surface rather than treating "has a parser package" as blanket compatibility.

Area	What To Check
Connection behavior	Authentication route, session state, database selection, client options, and refusal behavior.
SQL syntax	Accepted statements, expressions, functions, procedural blocks, comments, identifiers, and script tokens.
Type system	Native type spelling, value ranges, coercions, binary encodings, domain behavior, and null handling.
DDL	Create, alter, drop, recreate, rename, comment, show, describe, and object-specific lifecycle behavior.
DML and query behavior	Insert, update, delete, merge, upsert, select, joins, grouping, ordering, windowing, recursive common table expressions, and result ordering.
Transactions	Autocommit, begin, commit, rollback, savepoints, retain or chain behavior where applicable, and prepared transactions where supported.
Procedural SQL	Stored procedures, functions, triggers, packages, events, cursors, result sets, and routine metadata.
Catalog projection	System tables, views, metadata functions, dependency rows, privileges, indexes, constraints, and generated objects.
Backup and restore	Logical stream support, denied physical copy behavior, server-local file policy, and diagnostics.
CDC, replication, and ETL	Direction, ordering, transaction grouping, record identity, quarantine, cutover, and idempotency behavior where implemented.
Diagnostics	Error codes, message text, refusal classes, unsupported behavior, denied behavior, and policy failures.

Sandboxed Workareas

A compatibility client normally connects to a compatibility workarea (the namespace root visible to the client). That workarea is the root of the namespace the client can see. Objects outside the workarea are not directly accessible to the client, even if a catalog projection displays selected metadata about them.



Some catalog projection objects may display selected metadata from outside the sandbox when the projection object itself has been granted that authority. That does not give the user unrestricted direct access outside the workarea.

Logical Streams And File Access

Compatibility commands that move data require careful handling. The key distinction is between logical remote streams and server-local or physical storage operations.

Allowed by design when implemented and policy admits it:

- a client sends a logical restore stream;
- a client receives a logical backup stream;
- a tool imports rows or records through an admitted route;
- a tool exports rows or records through an admitted route;
- a parser-support routine streams logical CDC or ETL records.

Denied or restricted by design:

- server-local file open or manipulation by a compatibility parser unless policy explicitly admits a safe surface;
- physical page-copy backup or restore formats;
- low-level repair, verification, or page maintenance through compatibility parser routes;
- operations targeting objects outside the connected workarea without explicit engine authority.

A logical stream can be interpreted as client work. A physical page copy or repair command attempts to bypass engine authority.

Refusals Are Compatibility Behavior

A compatibility parser that fails clearly on unsupported behavior is doing its job correctly. A parser that silently accepts unsupported commands and returns success without doing the work is a defect, not a feature.

Common refusal classes:

- syntax not accepted by the selected parser;
- feature unsupported by the parser;
- feature unsupported by the engine build;
- operation denied by sandbox policy;
- operation denied because it requires server-local file access;
- operation denied because it is physical backup, restore, repair, or verification;
- unavailable parser UDR or bridge package;
- missing capability for CDC, replication, import, export, or migration;
- ambiguous ordering or transaction grouping in a stream.

Reading Compatibility Claims

When evaluating a parser, look for proof of the exact surface you need rather than inferences from directory names or package manifests.

1. The parser package exists in the build output.
2. The parser can be configured and selected.
3. Connection and authentication tests pass.
4. The SQL or protocol surface you need is implemented.
5. Type, index, transaction, and catalog behavior match the parser's documented compatibility target.
6. The behavior is covered by reference-system regression tests or parser proof gates.
7. Unsupported cases return the documented message vector (a structured diagnostic record).

Do not infer readiness from file names, directory names, or generated manifests alone.

For procedures to configure and test compatibility parsers, see the full Operations Administration (ScratchBird — Operations, Security, and Autonomy, page XXX) chapters.

Where To Go Next

- [First Database](#)
- [Schemas, Objects, And Names](#)
- [Engine Parser Boundary](#)
- [SBsql And SBRL](#)
- Language Reference (SBsql Language Reference — Foundations, Data Types, and Catalog, page XXX)

Choosing A Deployment Mode

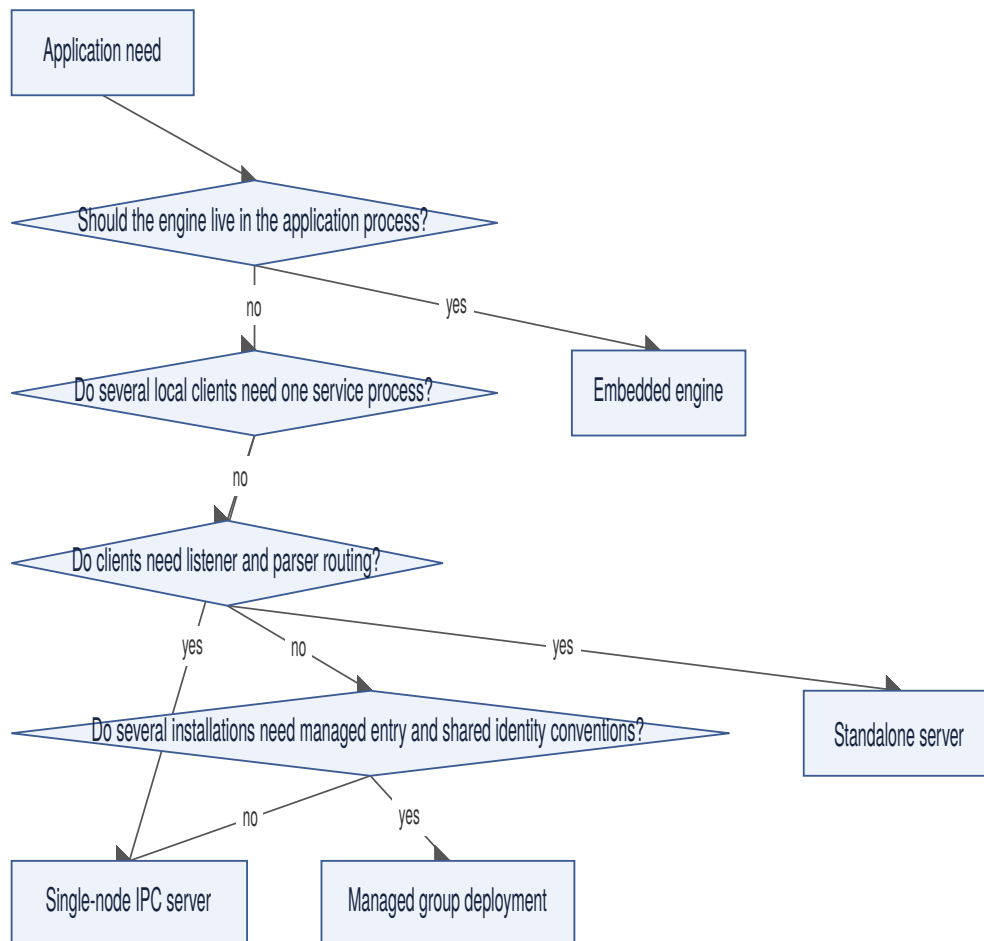
Purpose

This page is an administrator's companion to [Choosing A Mode Summary](#). Where that page helps you identify which mode to read about, this page helps you translate application and operational requirements into a concrete deployment decision and a first proof plan.

It is not a performance guide, sizing guide, support statement, or production-readiness claim. A deployment mode is usable only when the current build output, target platform, configuration, tests, and release notes prove the required behavior. For the hands-on step-by-step procedures, see the Operations Administration chapters.

Start With The Boundary

The first deployment decision is the process and trust boundary. Begin by asking what separation the application genuinely needs — adding listener, parser, manager, and shared identity layers is only worthwhile when their specific capabilities are part of the requirement.



Deployment Comparison

Mode	Process Boundary	Client Scope	Main Components	Administrative Focus
Embedded engine	Same process as the application.	Application-local.	SBcore (the core database engine).	Application lifecycle, resource discovery, transaction handling, diagnostics returned to the application.
Single-node IPC server	Separate local server process.	Same-machine clients.	SBsrv (the local server process) and SBcore.	IPC endpoint permissions, local service lifecycle, attach/detach behavior, local diagnostics.
Standalone server	Listener and parser route to a local service.	Network-facing client boundary where configured.	SBgate (the listener and router), parser package, SBsrv, SBcore.	Listener endpoints, parser registration, authentication, workarea routing, diagnostics, controlled drain/stop.
Managed group deployment	Manager-front-door convention over local services.	Operator-defined local installations.	SBmgr (the manager front-door) plus configured local services.	Shared identity conventions, route admission, policy consistency, local service health, diagnostics.

Questions To Ask

These questions surface the requirements that determine mode selection. Work through them before writing any configuration.

Question	Why It Matters
Does one application own all access?	Embedded mode may be enough if the application can carry engine lifecycle responsibility.
Do independent local clients need access?	A local server process gives clients a shared boundary without a network listener.
Does any client require a network-facing entry point?	Standalone server mode is the listener and parser-routing shape.
Is a compatibility parser part of the requirement?	Parser packages require explicit registration, routing, and proof.
Are several installations managed under common identity rules?	SBmgr can provide a consistent front-door convention where configured.
Do operators need service supervision independent of applications?	Prefer a server process over embedded mode.
Is the database disposable, test, or persistent?	Storage paths, backup behavior, diagnostics, and cleanup rules differ by intent.
What should happen when startup is uncertain?	The deployment should fail closed with useful diagnostics.

Administrative Worksheet

Before configuring a deployment, write these down so the configuration can be reviewed without having to reconstruct decisions later. This worksheet should be reviewable without exposing secrets.

- target platform and architecture;
- intended mode;
- database path or database selection mechanism;
- output tree location for binaries and resources;
- required parser packages;
- authentication source;
- authorization and default schema root policy;
- listener endpoint or IPC endpoint;
- expected users, services, or agents;
- diagnostic and support-bundle location;
- redaction policy;
- backup and restore expectation;
- start, stop, drain, and restart procedure;
- proof tests required before the deployment is trusted.

Mode-Specific First Proof

Do not broaden a deployment until the first proof is repeatable. These are the minimum acceptance tests for each mode.

Mode	Minimum First Proof
Embedded engine	Application opens a disposable database, runs create/insert/select/commit, closes, reopens, and handles one controlled diagnostic.
Single-node IPC server	SBsrv starts, a local client attaches, runs create/insert/select/commit, detaches, server restarts, and committed data is still visible.
Standalone server	SBgate starts, a parser route accepts a client, a session authenticates, work reaches SBcore, diagnostics return through the client, and drain/stop behaves cleanly.
Managed group deployment	SBmgr validates identity or policy context, opens a local route, admits or refuses the session clearly, and local service diagnostics identify the route used.

Parser And Tool Availability

Parser and tool availability must be checked per build — assumptions here are a common source of failed deployments.

- A parser does not exist because a directory exists.
- A parser is not installed because another parser is installed.
- A parser does not support another parser's dialect.
- A management command is not admitted because a tool name exists.
- A compatibility surface is not complete without proof.

Each parser should be configured and proven as its own standalone capability.

Security Considerations

For every mode, verify these items before accepting any real work. Security should be explicit — do not rely on default behavior for anything that protects real data.

- identity source and authentication behavior;
- default grants and roles;
- schema root or workarea assignment;
- protected-material handling;
- external access policy;
- parser route admission;
- diagnostic redaction;
- refusal behavior for denied or unsupported requests.

Operational Considerations

Administrators should have clear answers for these operational questions before the deployment is used for real work.

- who starts and stops the service;
- how clean shutdown is requested;
- how forced shutdown is handled;
- where logs and message vectors are collected;
- how support bundles are generated and reviewed;
- how stale local endpoints are detected;
- how configuration changes are validated before restart;
- how backup and restore drills are performed;
- how a failed open or recovery-required state is escalated.

What This Page Does Not Claim

This page does not claim:

- a deployment is production-ready;
- a mode is faster or safer than another mode for all workloads;
- every parser route is available;
- every administrative tool exists in every build;
- managed group deployment makes separate databases share storage or transactions;
- backup, restore, or repair behavior is complete without release-specific proof.

Where To Go Next

For detailed operating procedures for each mode, see the Operations Administration chapters, particularly the Operating Modes Runbook (ScratchBird — Operations, Security, and Autonomy, page XXX).

- [Choosing A Mode Summary](#)
- [Embedded Engine](#)
- [Single-Node IPC Server](#)
- [Standalone Server](#)
- [Managed Group Deployment](#)
- [Configuration Basics](#)
- [Diagnostics And Support Bundles](#)

Configuration Basics

Purpose

Getting ScratchBird configured correctly is not just a matter of setting a few flags. Configuration controls how components start, which databases they open, which parser routes are available, how identities are authenticated, what policy is applied, and where diagnostics are written. A deployment that passes all its proof tests but uses different resource files than the tested build is not the same deployment.

This page explains the concepts an administrator should verify before relying on a configuration. For hands-on setup procedures and configuration file formats, see the full Operations Administration chapters.

Exact file names, option names, and command-line flags can vary by build and packaging.

Configuration Is Part Of The Product

A ScratchBird output tree is not only binaries. A usable deployment may also need parser packages, character set resources, collation resources, time zone resources, policy defaults, configuration templates, security provider configuration, diagnostic and support-bundle settings, tool configuration, and tests or proof assets used to validate the output. If the runtime cannot find the resources it was tested with, the deployment is not the same as the tested build.

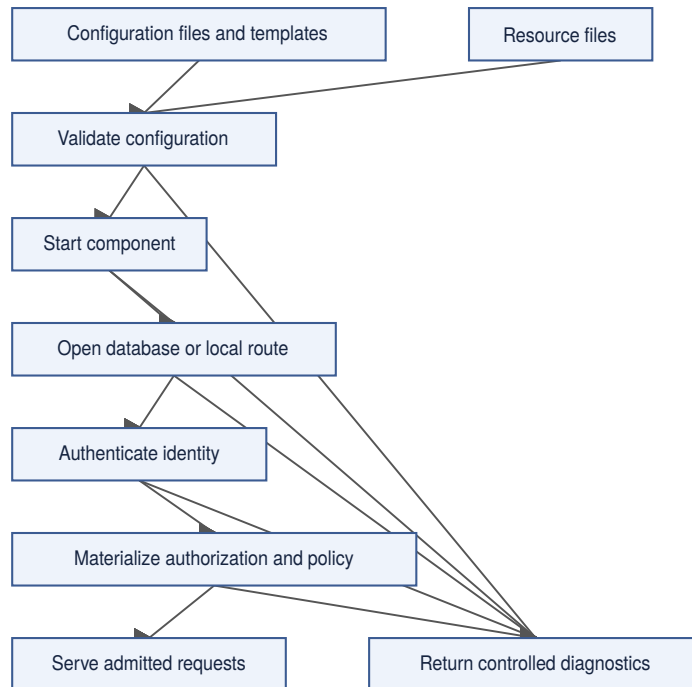
Configuration Areas

These are the areas an administrator must address. A gap in any area can cause failures that look confusing without this map.

Area	What It Controls
Operating mode	Embedded, single-node IPC, standalone server, or managed group deployment.
Database route	Which database is created or opened for a given request.
Storage paths	Where database files, filespace (the physical file organization backing a database), temporary files, and diagnostic files may live.
Resource files	Character sets, collations, time zones, policies, and other required data.
Parser routes	Which parser packages are available for which entry points.
Authentication	Which identity source and method are admitted.
Authorization	Grants, roles, schema roots, workareas, and policy.
External access	Whether file, network, or bridge-like external actions are admitted.
Runtime limits	Memory, file handles, frame sizes, timeouts, request envelope sizes, and backpressure.
Diagnostics	Log level, message vector (structured diagnostic record) detail, support-bundle scope, and redaction policy.
Lifecycle	Start, stop, drain, restart, stale endpoint handling, and refusal behavior.

Configuration Flow

Configuration is validated before the component accepts client work. A failed validation should produce a useful diagnostic rather than a running-but-broken deployment.



Output Tree Expectations

A release or test output tree should make it clear what belongs together. At a high level, administrators should be able to identify: runtime binaries, shared libraries, parser packages, command-line tools, configuration templates, resource files, test or proof material, platform-specific files, and documentation for the build.

Avoid mixing live database files with release binaries and resources. Databases should live in deliberate storage locations, not inside source directories or build output directories.

Parser Route Configuration

Parser routes should be configured explicitly rather than assumed. A well-configured parser route should answer these questions:

- which parser package is used;
- which client surface it accepts;
- which endpoint or mode can reach it;
- which database or workarea it attaches to;
- which identity rules apply;
- which schema root is visible;
- which unsupported surfaces are refused;
- where parser diagnostics are written.

Installing one parser should not imply that any other parser exists. A parser should not silently accept another parser's language.

Database Route Configuration

A database route should answer these questions before it is trusted with real data:

- whether the request creates a database or opens an existing database;
- where the database files live;
- which tablespaces are expected;
- which initial character set and collation are used;
- which security baseline applies;
- which parser-visible root or workarea is selected;
- what happens if the database is already open, missing, locked, or recovery-required.

Use disposable paths for early tests until the create/open/reopen lifecycle is understood.

Authentication And Authorization

Configuration should make identity behavior explicit — implicit defaults are a common source of security gaps. At minimum, know:

- which identity source is used;
- how a user, service, or agent authenticates;
- how failures are reported;
- which roles or grants are loaded;
- which schema root or workarea is assigned;
- whether protected material is referenced by secret reference rather than raw value;
- how policy denies are rendered.

Authentication proves identity. Authorization admits work. Both should be tested with both positive and negative cases before trusting the deployment.

Secrets And Protected Material

Do not put raw secrets into ordinary configuration examples, parser packets, scripts, or diagnostics. Use documented secret-reference mechanisms where available.

A configuration review should check:

- whether raw secrets appear in files;
- whether secret references resolve only for authorized identities;
- whether support bundles redact protected material;
- whether diagnostics avoid exposing credentials;
- whether configuration templates can be shared safely.

Resource Limits And Backpressure

Operational configuration should define limits before the system is under load — defaults should be treated as starting points, not as a substitute for workload-specific testing.

Examples of limits to configure deliberately:

- maximum request or frame size;
- maximum request envelope bytes;
- cursor or stream fetch size;
- transaction timeout;
- idle session timeout;
- memory budget;
- temporary storage policy;
- file handle limits;
- diagnostic retention;
- cancellation behavior.

Diagnostics Configuration

Diagnostics need their own configuration. Define these items before accepting real work.

- where logs go;
- which components log at which detail level;
- how message vectors are rendered;
- what support bundles include;
- what is redacted;
- how long diagnostic material is retained;
- who is allowed to generate or view support bundles.

Diagnostic configuration should be tested with both successful and refused requests.

Validation Checklist

Before opening a configured deployment to users, verify each of these items in order. Later steps depend on earlier ones.

1. Configuration parses successfully.
2. Required resource files are found.
3. Required binaries and parser packages are staged.
4. Database paths are deliberate and writable by the intended service identity.
5. Authentication succeeds for an allowed identity.
6. Authentication fails clearly for an invalid identity.
7. Authorization admits an expected request.
8. Authorization denies an expected forbidden request.
9. Parser routes accept only their intended surfaces.
10. Diagnostics are written and redacted.
11. Clean shutdown works.
12. Restart and reopen prove committed data remains visible.

Common Configuration Problems

Symptom	Likely Area
Component starts but cannot serve requests	Missing resource files, invalid route, unavailable database, or failed policy load.
Parser route unavailable	Parser package not staged, not registered, wrong route, or incompatible package version.
Authentication succeeds but queries fail	Authorization, schema root, grants, or policy.
Object appears missing	Current schema, visible root, workarea, transaction visibility, or wrong database route.
Support bundle contains too much detail	Redaction policy or diagnostic scope.
Service cannot restart cleanly	Stale endpoint, stale process state, storage lock, or database open refusal.
Restore or import refused	Policy, parser support, logical versus physical stream classification, or server-local file restriction.

Where To Go Next

For detailed configuration procedures, see the Operations Administration chapters.

- [Choosing A Deployment Mode](#)
- [Diagnostics And Support Bundles](#)
- [Identity, Authentication, And Authorization](#)
- [First Database](#)
- [Standalone Server](#)
- Refusal Vectors (SBsql Language Reference — Syntax, page XXX)

Backup, Restore, And Data Movement Overview

Purpose

Data movement in ScratchBird covers a range of related activities: protecting your data through backup and restore, importing or exporting records, streaming changes through CDC or replication, and migrating between systems. Understanding the distinctions between these surfaces — and between logical and physical operations — prevents both data loss and security mistakes.

This page is an orientation for administrators. It explains the concepts, the safety model, and what to verify before relying on any data movement workflow. It is not a complete data-protection policy, a release certification, or a step-by-step procedure. Any workflow that protects real data must be proven against the current build, target platform, configuration, parser route, and operational policy.

For detailed procedures, see the Operations Administration backup and restore chapters.

A backup is not useful until restore has been tested.

Core Distinction: Logical Versus Physical

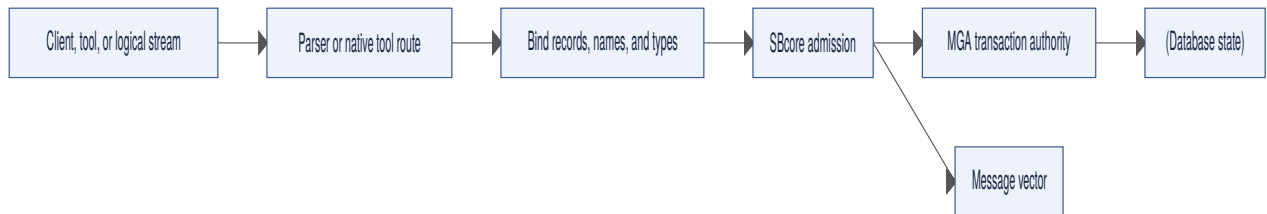
The most important distinction in data movement is whether an operation is logical or physical. Getting this wrong leads either to failed operations (physical operations rejected through parser routes) or to security problems (physical operations being admitted when they should not be).

Kind	Meaning	Administrative Reading
Logical export	Data and metadata are represented as statements, rows, records, events, or structured values.	Can be handled through parser, tool, transaction, authorization, and engine rules where implemented.
Logical import	A stream of statements, rows, records, or events is applied as database work.	Should pass through normal admission, type, transaction, and policy checks.
Logical backup	A backup stream that represents database content as metadata and data operations.	Restore can be treated as admitted logical work when the format is supported.
Logical restore	Applying a logical backup stream to a target database.	Should be tested into a disposable database before trust.
Physical copy	Database pages, files, or storage images are copied directly.	Requires engine-native storage rules; should not be treated as parser-level logical restore input.
Low-level repair or verification	Direct inspection or manipulation of storage internals.	Should be limited to documented native administrative surfaces where implemented and authorized.

Logical streams can be interpreted as work. Physical storage operations require storage authority.

ScratchBird Data Movement Model

All data movement — including logical backup and restore — passes through the same admission path as ordinary client work. It is not a shortcut around SBcore (the core database engine).



Backup

A backup workflow should define each of these items before it is run on anything important.

- source database;
- backup kind;
- scope: full, partial, schema, table, object, or filtered set where supported;
- consistency point;
- transaction handling;
- output destination;
- retention policy;
- encryption or protected-material handling where applicable;
- verification query or proof;
- restore drill plan.

For a first draft workflow, prefer a logical backup stream where the release documents and tests that surface.

Restore

A restore workflow should define:

- target database;
- whether the target is empty, existing, or a migration staging area;
- whether the stream is logical or physical;
- how object names and UUID-backed identity are handled;
- how security, grants, policies, and protected material are restored or remapped;
- transaction boundaries;
- conflict handling;
- validation queries;
- failure and rollback behavior.

Do not trust a backup strategy until restore into a non-production target has been tested.

Remote Logical Streams

Remote logical streams are the safest compatibility shape because the server receives logical work from a client or tool rather than being asked to open and manipulate arbitrary local files.

Allowed by design where implemented and policy admits it:

- a client sends a logical restore stream;
- a client receives a logical backup stream;
- a tool imports rows or records through an admitted route;
- a tool exports rows or records through an admitted route;
- a parser-support routine streams logical CDC or ETL records.

The stream still must pass authorization, type, transaction, and policy checks.

Server-Local File Access

Server-local file access is sensitive and deserves explicit policy treatment. A command that asks the server to open, read, overwrite, or repair a local file should be denied unless a documented native administrative surface and policy explicitly admits it.

For compatibility parser routes, the conservative rule is:

- remote logical streams can be supported where implemented;
- server-local file manipulation should be denied by default;
- physical page-copy backup or restore should be denied through parser routes;
- low-level repair and verification should not be available through compatibility parser routes.

This protects the engine authority boundary and reduces accidental file exposure.

CDC, Replication, And ETL

CDC, replication, and ETL are logical data movement surfaces. Each has specific requirements for correctness that must be defined before the workflow can be trusted.

Surface	Meaning
CDC	Change data capture: records changes as ordered logical events.
Replication	Applies change streams between systems according to ordering, identity, and conflict rules.
ETL	Extract, transform, load: reads data from one source, changes its shape, and writes it to a target.
Migration	Moves schema, data, routines, security, and operational behavior from one shape to another.

These surfaces need explicit rules for source identity, target identity, transaction grouping, record identity, ordering token or ordering evidence, idempotency, quarantine, cutover, retry, and refusal when order or identity is ambiguous.

Do not describe a replication or ETL route as available until the relevant parser, tool, engine path, and tests prove it.

Reference-System Compatible Data Movement

Some compatibility parser families expose logical backup, restore, CDC, replication, ETL, import, or export behavior. ScratchBird should support those surfaces only where they are implemented, safe, scoped to that parser, and admitted by policy.

The parser must classify each operation correctly:

Compatibility Request	Expected Classification
Client sends logical metadata and data stream.	Logical restore candidate, if implemented and admitted.
Client requests logical backup stream for the connected workarea.	Logical backup candidate, if implemented and admitted.
Client asks server to open an arbitrary local file.	Deny by default unless a safe native policy admits it.
Client submits physical page-copy format.	Deny through compatibility parser route.
Client asks for low-level repair or verification.	Deny through compatibility parser route.
Client requests CDC or replication stream.	Admit only if that parser route and engine surface implement it and policy allows it.

Compatibility does not mean bypassing ScratchBird security, storage, or transaction authority.

Migration

Migration is broader than import. A migration may need to handle schemas, tables and data, datatypes and domains, indexes and constraints, views and materialized views, stored procedures and functions, triggers and events, security grants and roles, policies, comments, sequences, backup and restore behavior, and application cutover.

Migration should be staged, tested, validated, and reversible where possible.

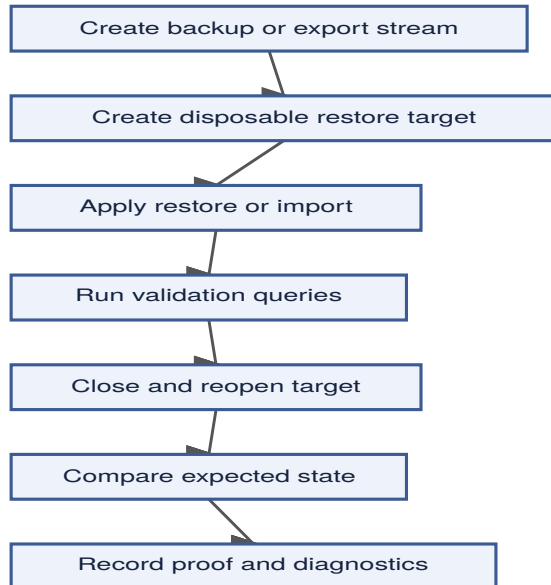
Validation Checklist

Before relying on a data movement workflow, verify each of these items.

1. The operation is classified as logical or physical.
2. The selected parser or native tool route supports the operation.
3. The source and target are explicit.
4. Authorization admits the operation.
5. External access policy admits any required external resource.
6. Protected material handling is defined.
7. Transaction boundaries are clear.
8. Restore or replay is tested into a disposable target.
9. Row counts, checksums, or validation queries are recorded where appropriate.
10. Expected refusals are tested.
11. Diagnostics are redacted.
12. Failure recovery is documented.

Restore Drill

A restore drill should be routine and repeatable. The drill is not complete until the restored target has been reopened and validated — closing and reopening proves the data was written durably, not just applied to a running engine.



What This Page Does Not Claim

This page does not claim:

- every backup or restore command is implemented;
- every parser supports logical backup or restore;
- physical page-copy operations are supported through parser routes;
- replication or CDC is complete for every parser;
- a backup is valid without restore testing;
- repair or verification is available through compatibility parser routes;
- data movement is safe without policy review.

Where To Go Next

For detailed procedures and format documentation, see the Operations Administration chapters.

- [Configuration Basics](#)
- [Diagnostics And Support Bundles](#)
- [First Database](#)
- [Reference-System Compatibility](#)
- Backup, Restore, Replication, Migration (SBsql Language Reference — Syntax, page XXX)
- Copy (SBsql Language Reference — Syntax, page XXX)
- Refusal Vectors (SBsql Language Reference — Syntax, page XXX)

Diagnostics And Support Bundles

Purpose

Good diagnostics make ScratchBird usable under real conditions: they tell you what the system did, refused, could not complete, or could not safely determine. A controlled refusal with a clear explanation is better than a confusing crash or silent success. Support bundles collect diagnostic evidence for review while keeping sensitive material out of places it should not go.

This page explains the expected administrative model for diagnostics and support bundles. It does not claim that every component emits complete evidence in every build. For detailed procedures on configuring diagnostic output and generating support bundles, see the Operations Administration diagnostics chapter.

Diagnostic Goals

Good diagnostics should answer:

- Which component handled the request?
- Which identity and parser route were involved, if policy allows that detail?
- Which database or route was selected?
- Was the request accepted, denied, unsupported, unavailable, or failed during execution?
- Was the refusal caused by syntax, binding, authorization, policy, storage, transaction state, or configuration?
- Can the user continue, retry, roll back, detach, or ask an operator to intervene?
- Was any protected material redacted?

Diagnostics are part of the product surface. A controlled refusal is better than an unclear crash or silent success.

Diagnostic Types

Type	Meaning
Error	A request failed during parsing, binding, admission, execution, storage, or transaction handling.
Refusal	A request was denied, unsupported, unavailable, unsafe, or outside policy.
Warning	Work completed but produced a condition the user or operator should inspect.
Info	Operational detail useful for understanding startup, shutdown, routing, or configuration.
Evidence	Structured context used to explain a decision, prove a route, or support review.
Proof	Test or validation output showing that a behavior was exercised.

Message Vectors

ScratchBird uses message vectors for structured diagnostics. A message vector is a structured diagnostic record that lets tools distinguish between failure categories rather than parsing only human-readable text. That distinction matters when you are trying to write automation that responds differently to "authorization denied" versus "database cannot open."

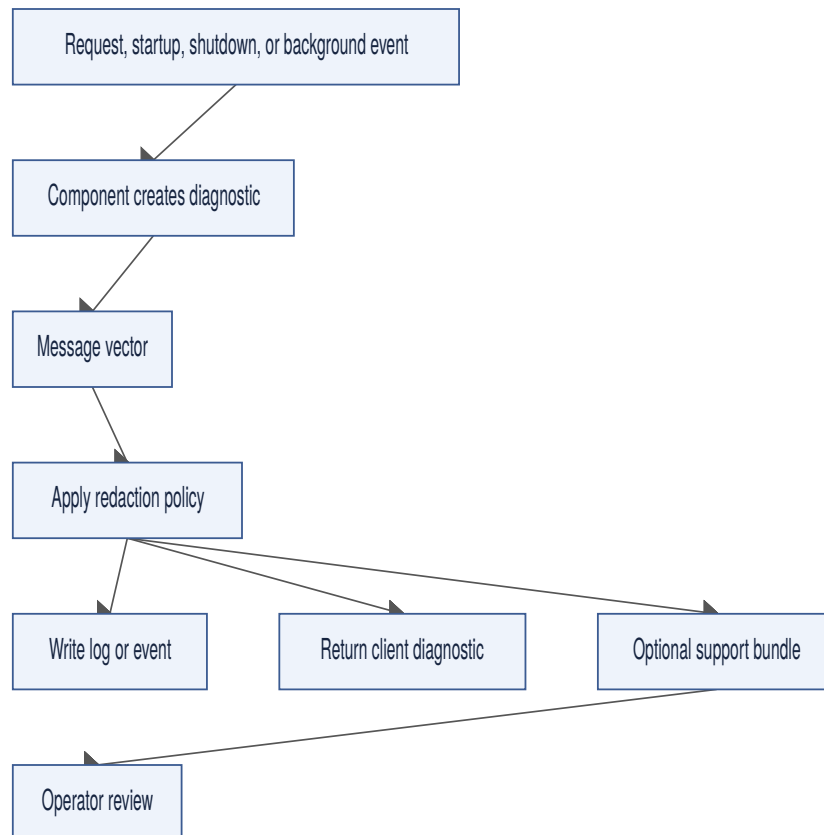
Common message vector categories include:

- invalid syntax;
- invalid token;
- parser route unavailable;
- parser feature unsupported;
- object not found or not visible;
- ambiguous name;
- invalid type coercion;
- authorization denied;
- policy denied;
- sandbox denied;
- protected material unavailable;
- external access denied;
- missing capability;
- database open refused;
- transaction state invalid;
- recovery required;
- storage unavailable;
- diagnostic redacted.

The exact rendering can differ by parser or tool. The underlying category should remain clear.

Diagnostic Flow

The flow below shows how a diagnostic moves from the event that triggered it through to logging, client delivery, and optional support bundle inclusion. Redaction must happen before sensitive material is shared outside the trusted boundary.



What To Include In Diagnostics

Where policy allows, useful diagnostics include:

- timestamp;
- component name;
- build or package version;
- operating mode;
- route selected;
- parser package selected;
- session identity or redacted identity reference;
- database route;
- schema root or workarea reference;
- transaction state;
- object name and object UUID where safe;
- message vector class;
- refusal reason;
- next recommended action;
- correlation identifier;
- support-bundle identifier.

Do not include raw secrets, credentials, protected values, or unnecessary local machine details.

Support Bundle Purpose

A support bundle is a controlled diagnostic package that helps an operator or support engineer understand a problem without requiring unrestricted access to the environment. It is not a raw log dump — it should be scoped to the problem and reviewed before being shared.

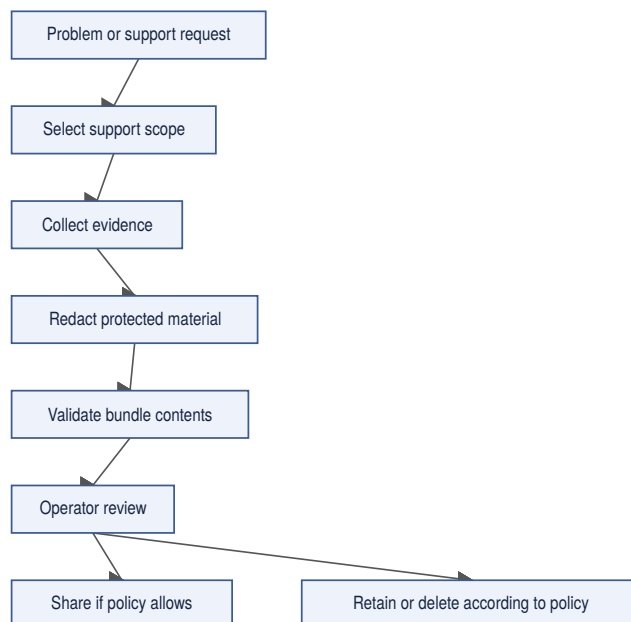
A support bundle may include:

- configuration summary;
- component versions;
- startup and shutdown evidence;
- parser registration summary;
- database open summary;
- identity provider summary;
- redacted session context;
- message vectors;
- selected logs;
- support-bundle generation metadata;
- resource availability summary;
- test or proof summaries where available.

It should not include raw secrets or protected material.

Support Bundle Flow

An operator should review the bundle before sharing it. Generating a bundle and sending it immediately — without review — risks disclosing information that should have been redacted.



Redaction Policy

Redaction should protect passwords, keys, tokens, raw protected material, unwrapped secret values, unnecessary local paths, credentials in connection strings, private user data not needed for diagnosis, and sensitive policy details where disclosure would create risk.

Redaction can preserve safe evidence such as hashes, UUIDs where policy allows, component names, version identifiers, timestamps, message vector classes, object names where safe, redacted route names, and counts and summaries.

Diagnostics By Area

When investigating a problem, collect diagnostics from the relevant area first. This table maps areas to the most useful evidence.

Area	Useful Evidence
Startup	Configuration validation, resource discovery, component version, route registration, database open result.
Authentication	Provider selected, identity result, refusal class, redacted principal reference.
Authorization	Grants loaded, schema root, policy outcome, denied object or route where safe.
Parser	Parser selected, accepted surface, unsupported feature refusal, binding failure.
Query execution	Statement class, transaction state, object visibility, result or failure class.
Storage	Database path class, filespace state, open refusal, storage unavailable, recovery-required state.
Transactions	Begin, commit, rollback, savepoint, conflict, invalid state, cleanup restriction.
Backup and restore	Logical stream classification, target database, policy result, denied physical operation.
Data movement	Source, target, ordering or record identity summary, quarantine or refusal state.

Refusal Examples

These examples show how a good diagnostic distinguishes between different failure causes, which matters for routing the fix to the right person.

User-Facing Situation	Diagnostic Should Distinguish
A parser command is not implemented.	Unsupported parser feature.
The engine build lacks the operation.	Missing engine capability.
The identity lacks permission.	Authorization denied.
The request targets another schema branch.	Sandbox denied or object not visible.
A command asks the server to open a local file.	External access denied or policy denied.
A physical page-copy restore is submitted through a parser route.	Unsupported physical operation through that route.
A database cannot safely open.	Recovery required or open refused.

Operator Review Checklist

Before sharing a support bundle:

1. Confirm the bundle was generated by the intended build.
2. Confirm the scope is limited to the problem.
3. Confirm raw secrets are absent.
4. Confirm protected material is redacted.
5. Confirm local paths are removed or minimized.
6. Confirm user data is not included unless required and authorized.
7. Confirm the message vector classes are preserved.
8. Confirm timestamps and component names are present.
9. Confirm the bundle can be retained or deleted according to policy.

What This Page Does Not Claim

This page does not claim:

- every component emits complete diagnostics;
- every support-bundle field exists in every build;
- redaction has been independently certified;
- diagnostics are a substitute for backups;
- support bundles are safe to share without operator review.

Verify support-bundle behavior with the current build before using it in a real support process.

Where To Go Next

For detailed diagnostic configuration procedures, see the Operations Administration chapters.

- [Configuration Basics](#)
- [Choosing A Deployment Mode](#)
- Refusal Vectors (SBsql Language Reference — Syntax, page XXX)
- Management And Operations (SBsql Language Reference — Syntax, page XXX)
- Security And Privilege Statements (SBsql Language Reference — Syntax, page XXX)

The Non-Programmer's Guide to MGA

1. Always in a Transaction (The Safety Bubble)

Whenever you open the database, you are automatically placed inside a **Transaction**. Think of this as your own private safety bubble. Anything you do inside this bubble isn't real to the rest of the world until you hit "Save" (Commit). If something goes wrong, the bubble pops (Rolls back), and it's like it never happened.

2. Always in Snapshot Mode (The Time-Freeze Photo)

The moment your safety bubble opens, the database takes a digital snapshot of the entire system.

- You only see data that was officially saved **before** your snapshot was taken.
- Even if another user changes a row while you are working, your view remains frozen in time. You will never experience data shifting under your feet.

3. Non-Destructive Changes (The LiveJournal Page)

When you **Delete** or **Update** a row, the database *never* overwrites or destroys the original data on the disk.

- **An Update:** Writes a brand new version of the row right next to the old one, with a note saying: *"If you want the version before this, look over there."*
- **A Delete:** Doesn't erase anything. It just attaches a "Dead" sticky note to the row.

How the Database Decides What You See

When a non-programmer reads the "LiveJournal" page, the database acts as a smart filter by asking three simple questions:

1. **Is the latest version officially saved?** * Looking at the diagram, **Tom's Version 3** isn't saved yet (it's Active). The database immediately ignores it for other users.
2. **Was it saved *before* my snapshot photo was taken?**
 - **User A** took their photo after Sarah saved. They see **Version 2 (\$12.00)**.
 - **User B** took their photo a long time ago, before Sarah even started writing. The database rolls them all the way back to **Version 1 (\$10.00)**.
3. **What happens to the old stuff?**
 - Eventually, when User B finishes their work and closes their bubble, *nobody* in the system will need Version 1 anymore. The next time the database reads this page, it will quietly clean up and remove Version 1 to save space. This is called **Garbage Collection**.